

CSD3009 - DATA STRUCTURES AND ANALYSIS OF ALGORITHMS

DR.S.DEVARAJU

Module 4

Searching and Sorting: Searching: Linear search, Binary search and Hashing. Algorithms and data structures for sorting: Insertion Sort, Bubble sort, Selection Sort, Merge sort, Quick Sort, Heap sort, Radix sort, Bucket sort. Algorithm design techniques: Divide and conquer, Greedy approach, dynamic programming.

<https://www.javatpoint.com/data-structure-tutorial>

Searching Techniques

The searching problem

- Searching is a process of finding a particular element among several given elements.
- The search is successful if the required element is found. Otherwise, the search is unsuccessful.

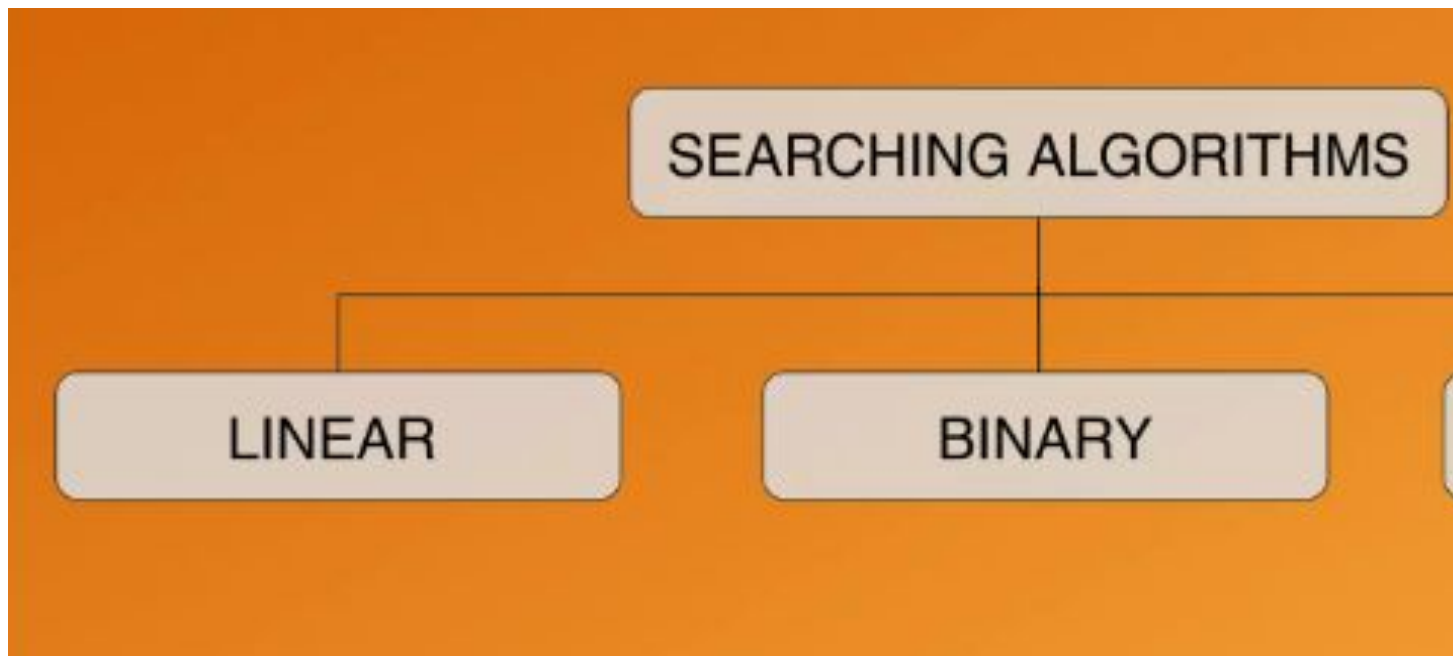
(OR)

- Problem is to retrieve an entire record based on the value of some key
- Find an index i such that $x = S_i$ if x equals one of the keys
- If x does not equal one of the keys, report failure

The searching problem

- Searching algorithm is an algorithm made up of a series of instructions that retrieves information stored within some data structure, or calculated in the search space of the problem domain.

-

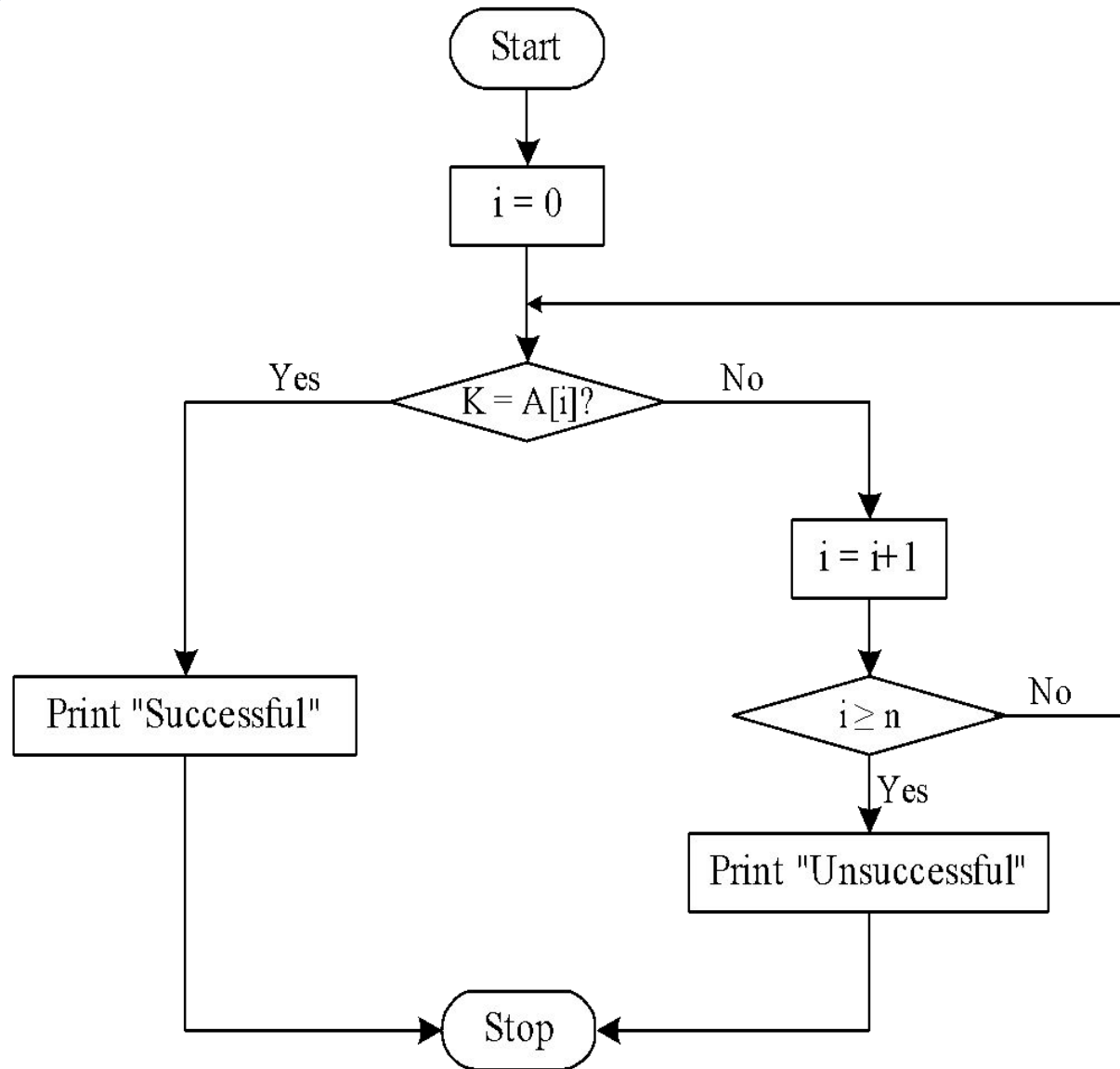


Linear Search

https://www.youtube.com/watch?v=9J5OTuk2_CI

Sequential Search with Arrays

Flowchart: Sequential Search with Array



Example: Sequential Search with Array

Linear Search

2 inputs

→ array
→ element

8	7	6	5
0	1	2	3

```
int search (int arr[], int x)
{
    for (int i = 0; i < arr.length; i++)
    {
        if (arr[i] == x)
        {
            return i;
        }
    }
    // x is not found
    return -1;
}
```

Example: Sequential Search with Array

```
int main()
{
    int A[10], i, n, K, flag = 0;
    printf("Enter the size of an array: ");
    scanf("%d",&n);

    printf("Enter the elements of the array: ");
    for(i=0; i < n; i++)
        scanf("%d",&A[i]);
    printf("Enter the number to be searched: ");
    scanf("%d",&K);
    for(i=0;i<n;i++){
        if(a[i] == K){
            flag = 1; break;
        }
    }
    if(flag == 0)
        printf("The number is not in the list");
    else
        printf("The number is found at index %d",i);
    return 0;
}
```

Complexity Analysis

- Case 1: The key matches with the first element
 - $T(n) = 1$
- Case 2: Key does not exist
 - $T(n) = n$
- Case 3: The key is present at any location in the array

$$T(n) = \sum_{i=1}^n p_i \cdot i$$

$$T(n) = \frac{1}{n} \sum_{i=1}^n i$$

$$T(n) = \frac{n+1}{2}$$

$$p_1 = p_2 = \cdots p_i = \cdots p_n = \frac{1}{n}$$

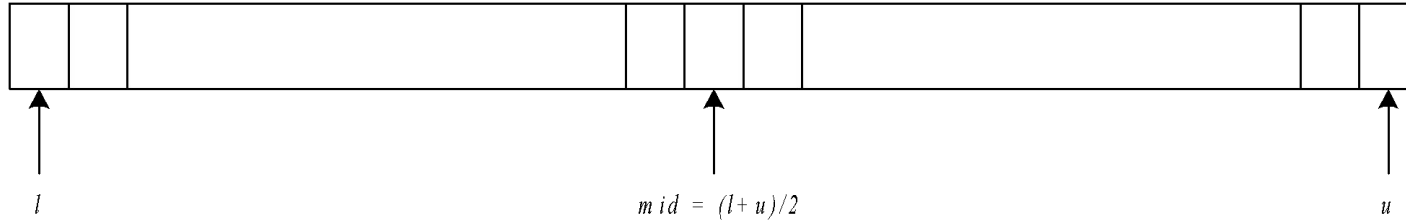
Complexity Analysis : Summary

Case	Number of key comparisons	Asymptotic complexity	Remark
Case 1	$T(n) = 1$	$T(n) = O(1)$	Best case
Case 2	$T(n) = n$	$T(n) = O(n)$	Worst case
Case 3	$T(n) = \frac{n+1}{2}$	$T(n) = O(n)$	Average case

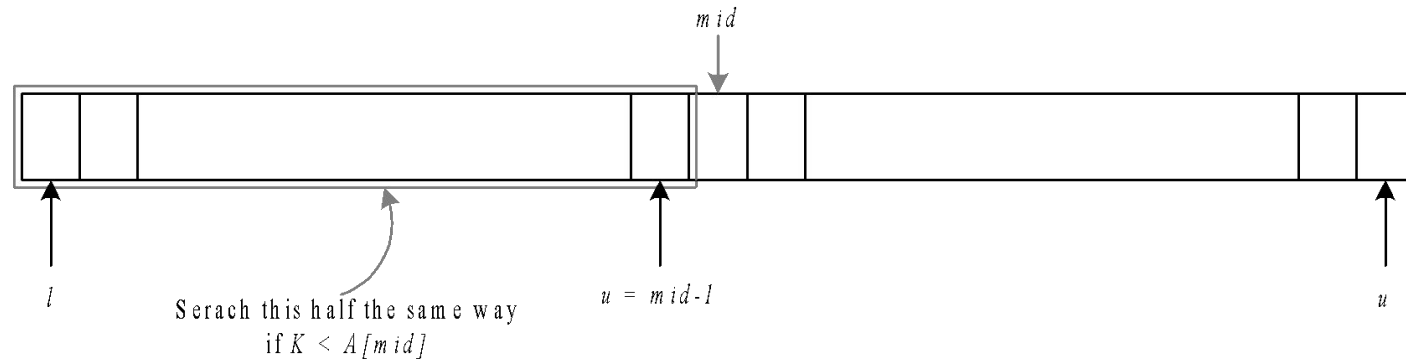
Binary Search

<https://www.youtube.com/watch?v=81QLBCW94Oo>

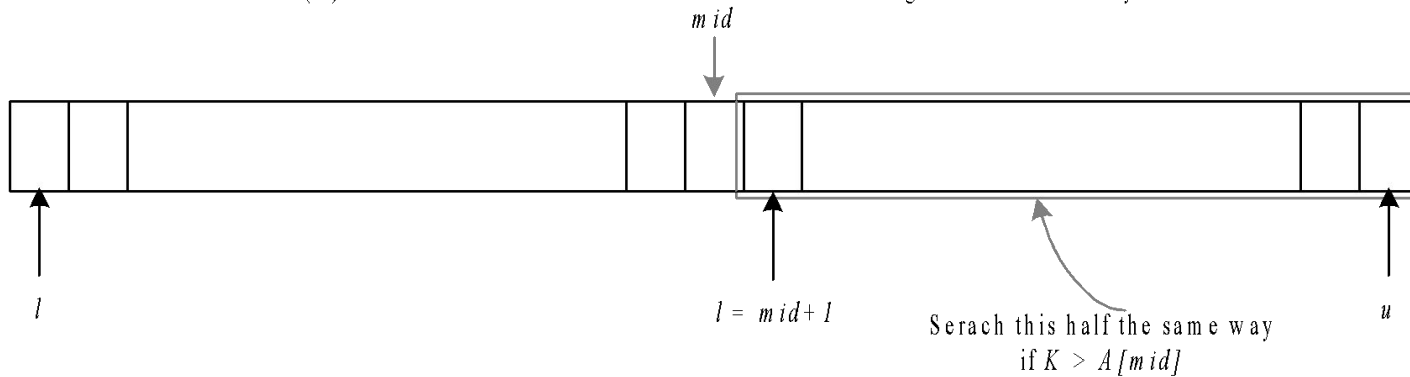
The Technique



(a) An ordered array of elements with index values l , u and mid

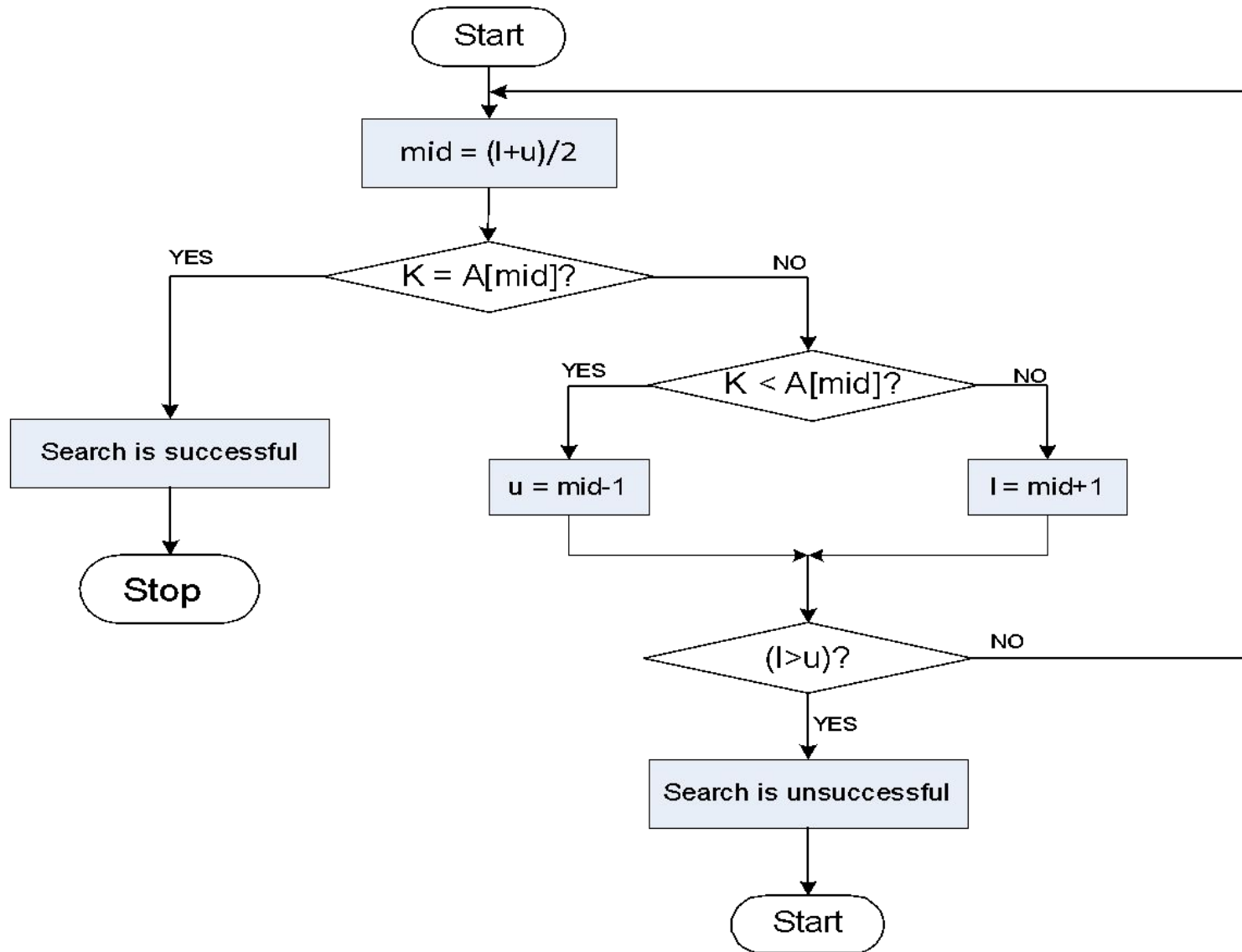


(b) Search the entire list turns into the searching of left-half only



(c) Search the entire list turns into the searching of right-half only

Flowchart: Binary Search with Array



Binary Search (with Iteration)

Example 4①

Successful search

Array

0	1	2	3	4	5	6	7	8	9
5	7	9	13	32	33	42	54	56	88

← skip → start → end

key = 33

$A[mid] = 32$

$$mid = \frac{(start + end)}{2} = \frac{(0 + 9)}{2} \approx 4$$

$$33 > A[mid]$$

$$33 > A[4]$$

$$start = mid + 1$$

5	6	7	8	9
33	42	54	56	88

start mid skip

$$mid = \frac{5 + 9}{2} = \frac{14}{2} = 7$$

5	6
33	42

$$mid = 5 + 6 \approx 5$$

$$33 = A$$

$$33 = A$$

$$33 < A[mid]$$

$$33 < A[7]$$

$$end = mid - 1$$

$$33 = A[mid]$$

$$33 = A[7]$$

$$33 = 33$$

Binary Search (with Iteration)

```
#include <stdio.h>

int main()
{
    int i, l, u, mid, n, K, data[100];

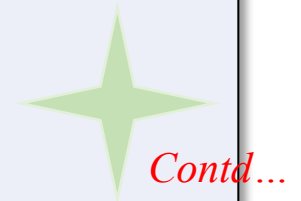
    printf("Enter number of elements\n");
    scanf("%d",&n);

    printf("Enter %d integers in sorted order\n", n);

    for (i = 0; i < n; i++)
        scanf("%d",&array[i]);

    printf("Enter value to find\n");
    scanf("%d", &K);

    l = 0;
    u = n - 1;
    mid = (l+u)/2;
```



Binary Search (with Iteration)

```
while (l <= u) {  
    if (data[mid] < K)  
        l = mid + 1;  
    else if (data[mid] == K) {  
        printf("%d found at location %d.\n", search, mid+1);  
        break;  
    }  
    else  
        u = mid - 1;  
  
    mid = (l + u)/2;  
}  
if (l > u)  
    printf("Not found! %d is not present in the list.\n", K);  
  
return 0;  
}
```

Binary Search (with Recursion)

```
#include<stdio.h>
int main(){

    int data[100],i, n, K, flag, l, u;

    printf("Enter the size of an array: ");
    scanf("%d",&n);

    printf("Enter the elements of the array in sorted order: " );
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    printf("Enter the number to be search: ");
    scanf("%d",&K);

    l=0,u=n-1;
    flag = binarySearch(data,n,K,l,u);
    if(flag==0)
        printf("Number is not found.");
    else
        printf("Number is found.");

    return 0;
}
```



Contd...

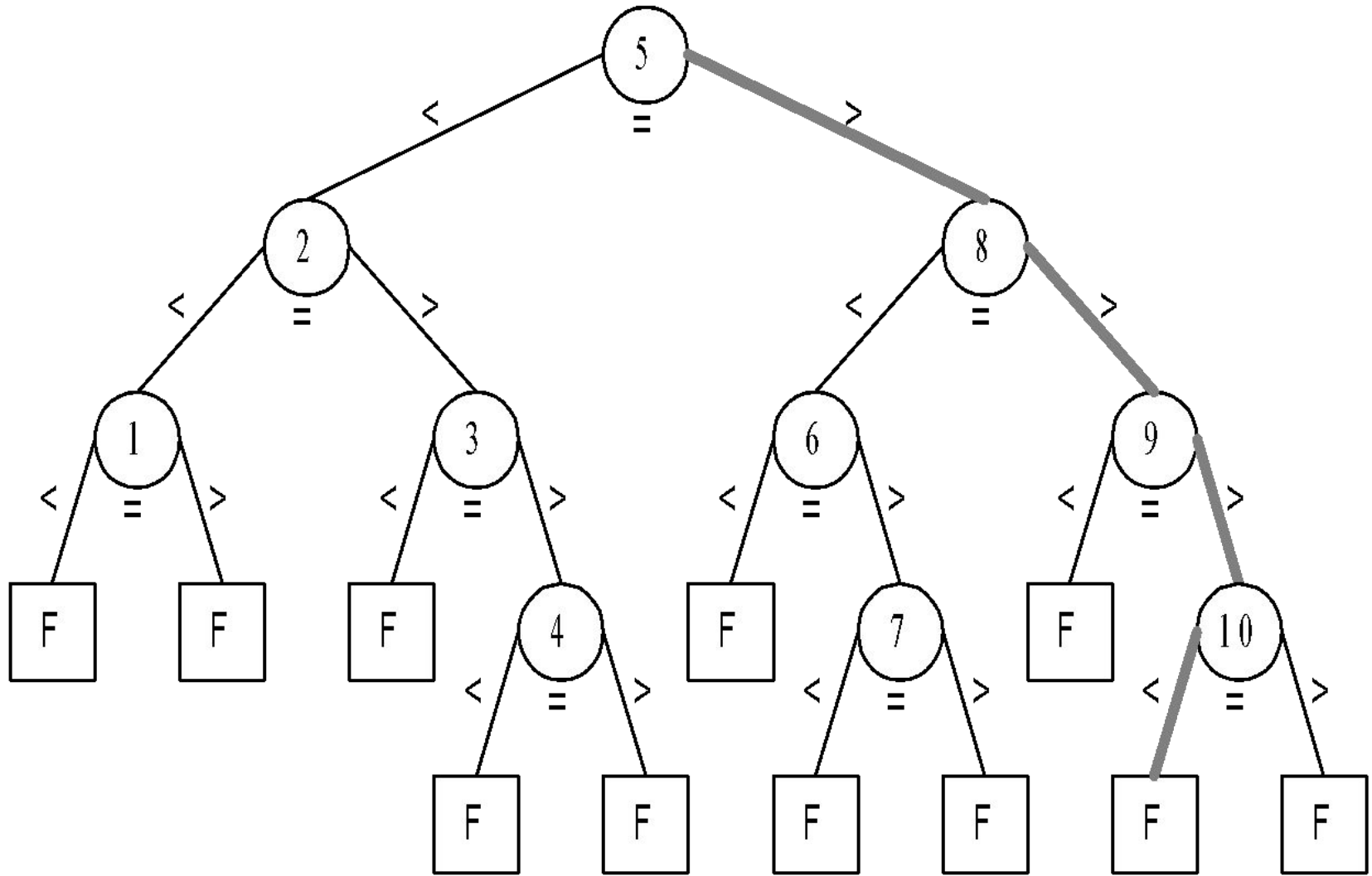
Binary Search (with Recursion)

```
int binary(int a[],int n,int K,int l,int u){

    int mid;

    if(l<=u){
        mid=(l+u)/2;
        if(K==a[mid]){
            return(1);
        }
        else if(m<a[mid]){
            return binarySearch(a,n,K,l,mid-1);
        }
        else
            return binarySearch(a,n,m,mid+1,u);
    }
    else return(0);
}
```

Complexity Analysis



Complexity Analysis: Binary Search

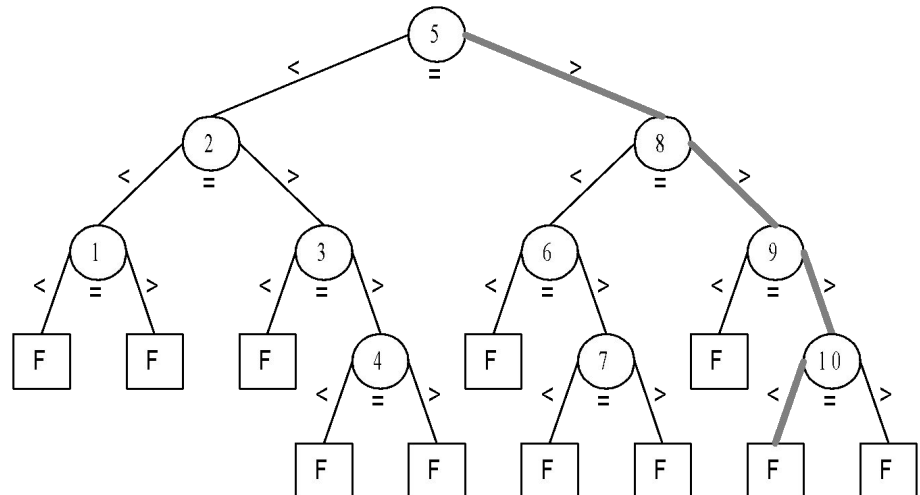
Let n be the total number of elements in the list under search and there exist an integer k such that:-

- For successful search:-

- If $2^{k-1} \leq n < 2^k$, then the binary search algorithm requires at least one comparison and at most k comparisons.

- For unsuccessful search:-

- If $n = 2^{k-1}$, then the binary search algorithm requires k comparisons.
- If $2^{k-1} \leq n < 2^k - 1$, then the binary search algorithm requires either $k-1$ or k number of comparisons.



Complexity Analysis: Binary Search

- Best case

$$T(n) = 1$$

- Worst case

$$T(n) = \lfloor \log_2(n + 1) \rfloor$$

Complexity Analysis: Binary Search

- Average Case

- Successful search:-

$$T(n) = \frac{I}{n} + 1$$

$$T(n) = \lfloor \log_2 n \rfloor + \frac{\lfloor \log_2 n \rfloor}{n} + \frac{2}{n} - 1$$

- Unsuccessful search:-

$$T'(n) = \frac{E}{n+1}$$

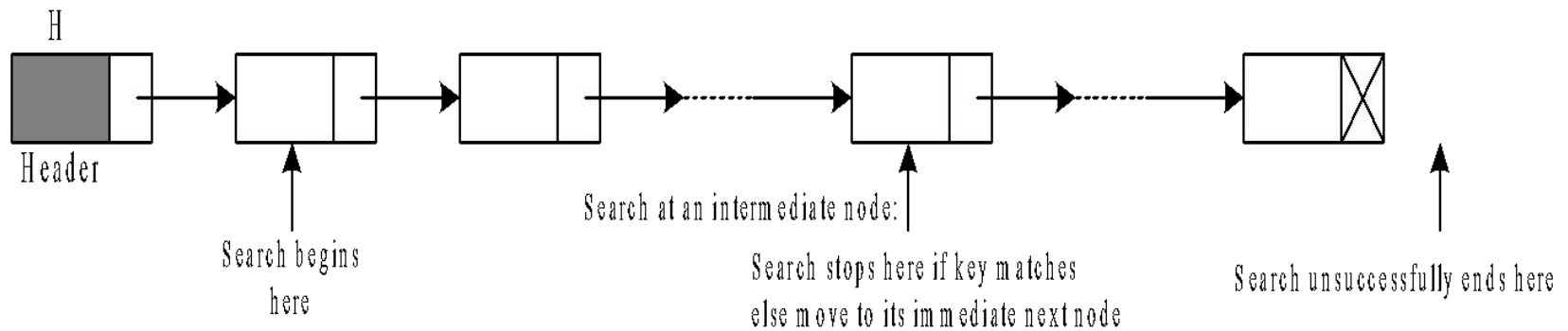
$$T'(n) = \lfloor \log_2 n \rfloor - \frac{2}{n+1}$$

Sequential Search with Linked List

Sequential Search with Linked List

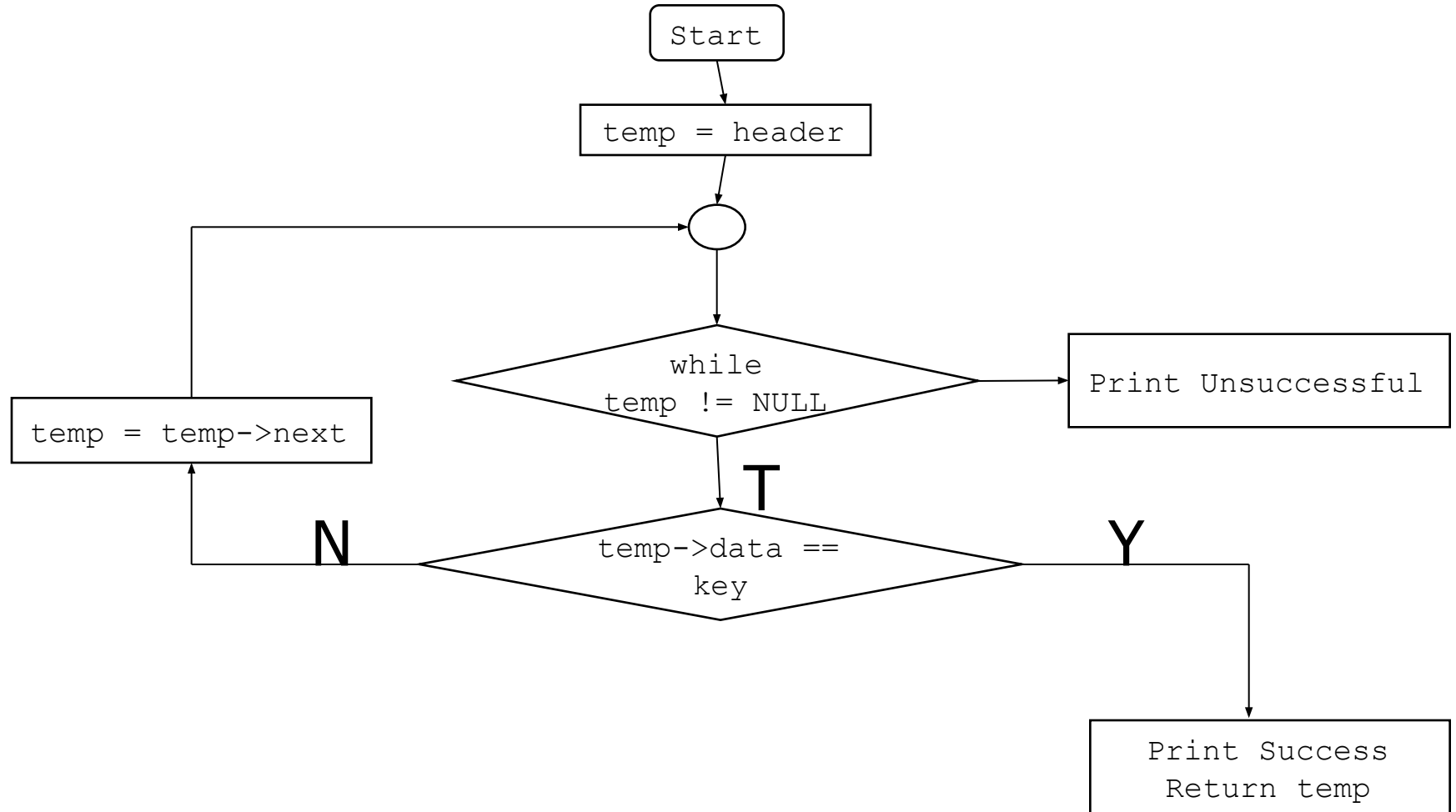


(a) Structure of a node in the linked list



(b) Linear search on a linked list

Flow Chart: Sequential Search with LL



Example: Sequential Search with LL

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

int main()
{
    struct node *header = NULL;
    int K, n;
    printf("Enter the number of nodes: ");
    scanf("%d", &n);
    printf("\nDisplaying the list\n");
    generate(header, num);
    printf("\nEnter key to search: ");
    scanf("%d", &key);
    searchBinary(header, K);
    delete(header);
    return 0;
}
```

Example: Linear Search with LL

```
void generate(struct node *head, int n)
{
    int i;
    struct node *temp;

    for (i = 0; i < num; i++)
    {
        temp = (struct node *)malloc(sizeof(struct node));
        temp->data = rand() % n;
        if (*header == NULL)
        {
            *header = temp;
            temp->next = NULL;
        }
        else
        {
            temp->next = header;
            header = temp;
        }
        printf("%d ", temp->data);
    }
}
```

Example: Linear Search with LL

```
void searchBinary(struct node *temp, int K)
{
    while (temp != NULL)    {
        if (temp->data == K)    {
            printf("key found\n");
            return;
        }
        else temp = temp->next;
    }
    printf("Key not found\n");
}
```

```
void delete(struct node *header) {
    struct node *temp;
    temp = header;
    while (temp != NULL)    {
        temp = temp->next;
        free(header);
        header = temp;
    }
}
```

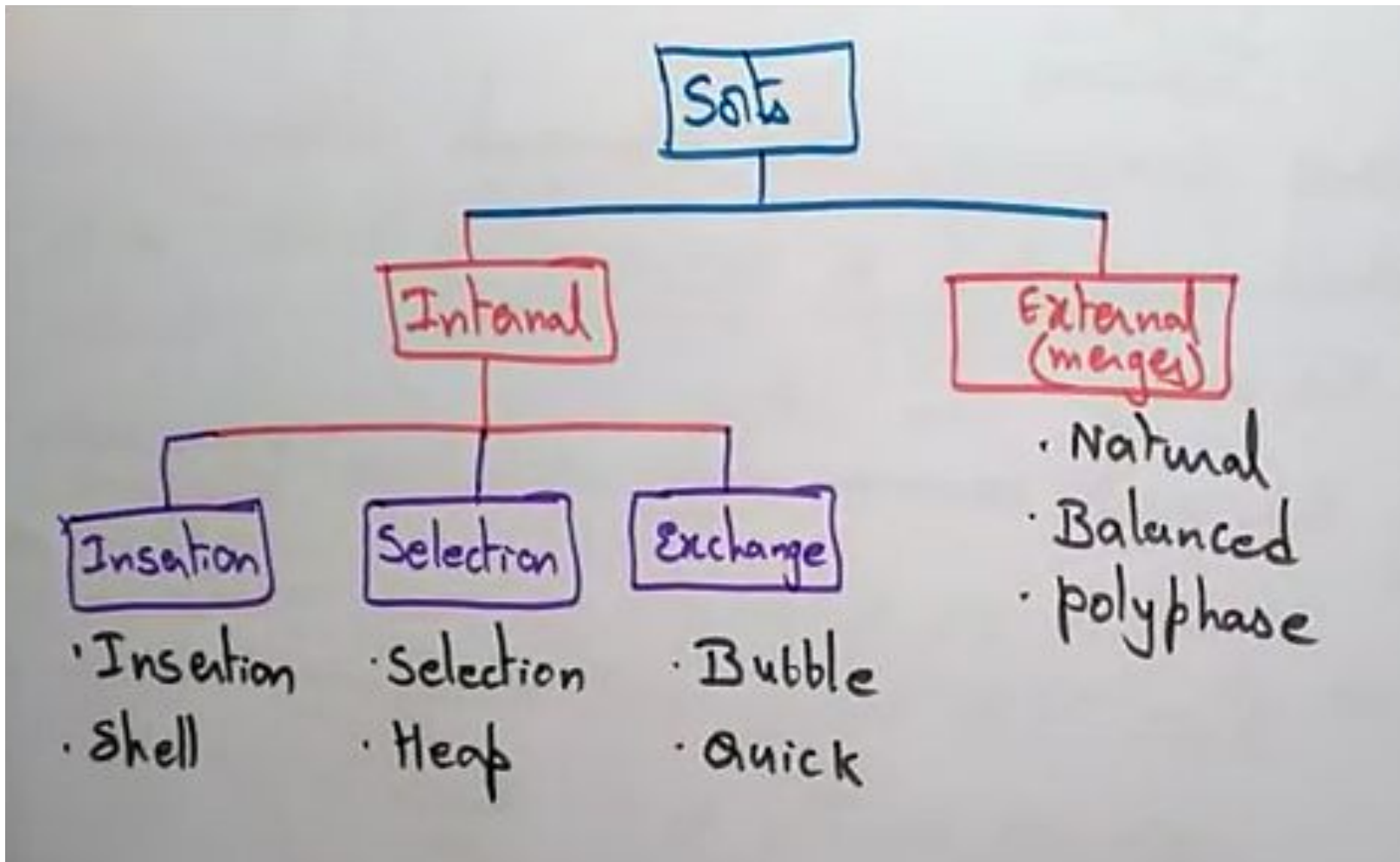
Complexity Analysis

Case	Number of key comparisons	Asymptotic complexity	Remark
Case 1	$T(n) = 1$	$T(n) = O(1)$	Best case
Case 2	$T(n) = \frac{n+1}{2}$	$T(n) = O(n)$	Average case
Case 3	$T(n) = n$	$T(n) = O(n)$	Worst case

Sorting

<https://www.youtube.com/watch?v=TnU8C-OKcZs>

What is Sorting?



What is Sorting?

Sorting: an operation that segregates items into groups according to specified criterion.

$A = \{ 3 \ 1 \ 6 \ 2 \ 1 \ 3 \ 4 \ 5 \ 9 \ 0 \}$

$A = \{ 0 \ 1 \ 1 \ 2 \ 3 \ 3 \ 4 \ 5 \ 6 \ 9 \}$

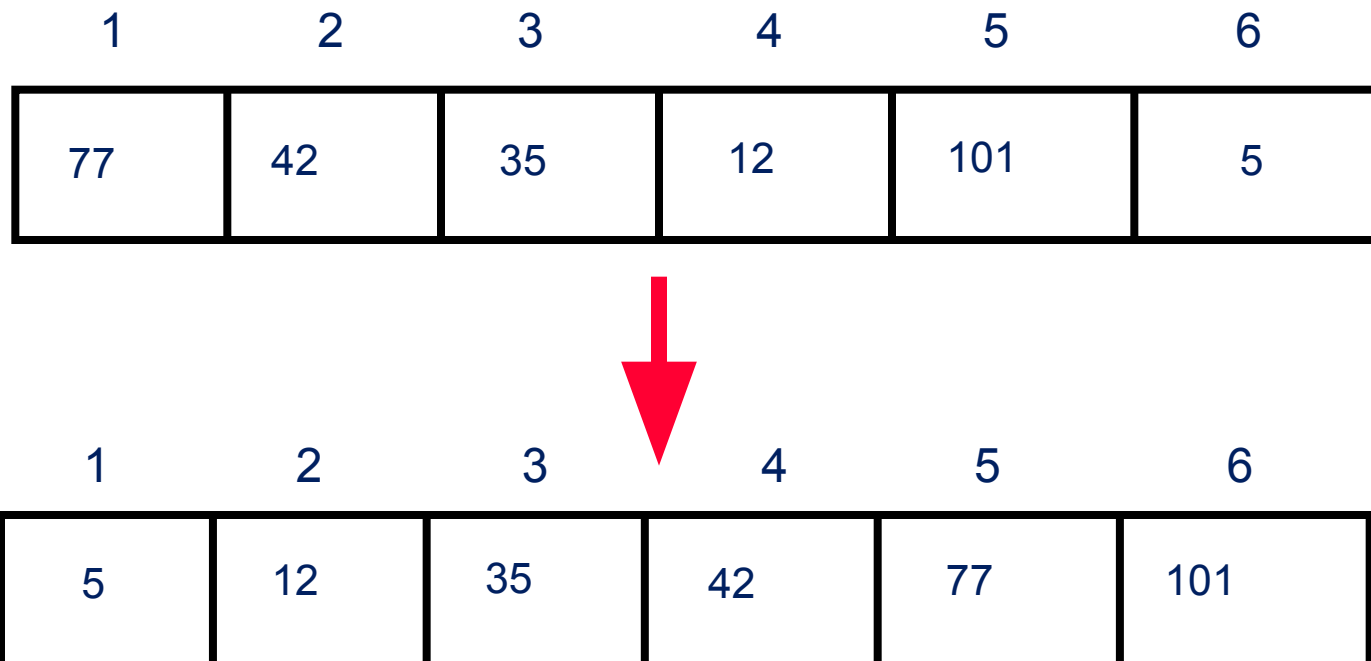
Why Sort and Examples

Consider:

- Sorting Books in Library (Dewey system)
- Sorting Individuals by Height (Feet and Inches)
- Sorting Movies in Blockbuster (Alphabetical)
- Sorting Numbers (Sequential)

Sorting

- Sorting is a process in which records are arranged in ascending or descending order



Types of sorting

There are many, many different types of sorting algorithms, but the primary ones are:

- Selection sort (<https://www.youtube.com/watch?v=EcYWIYfGiAI>)
- Insertion sort (https://www.youtube.com/watch?v=4MU_yI6957E)
- Bubble sort (<https://www.youtube.com/watch?v=VRL2iwrCm8E>)
- Merge sort (https://www.youtube.com/watch?v=eGO_FMca7mk)
- Quick sort (<https://www.youtube.com/watch?v=cYhIWJHRR0I> & <https://www.youtube.com/watch?v=JFNjj7jrvuo>)
- Heap sort
(<https://www.youtube.com/watch?v=yAlOKmgq8do> & <https://www.youtube.com/watch?v=VLLJ6vUQUMY>)
- Radix sort (<https://www.youtube.com/watch?v=XcF1l2pq9ZI>)
- Bucket sort (<https://www.youtube.com/watch?v=geVylsFpxUs>) & <https://www.youtube.com/watch?v=geVylsFpxUs>)

Selection Sort

Selection Sort

- **Selection sort** is a sorting algorithm which works as follows:
 - Find the minimum value in the list
 - Swap it with the value in the first position
 - Repeat the steps above for remainder of the list (starting at the second position)

Example: Selection Sort

- 26 33 43 100 46 88 52 17 53 77
- 17 | 33 43 100 46 88 52 26 53 77
- 17 26 | 43 100 46 88 52 33 53 77
- 17 26 33 | 100 46 88 52 43 53 77
- 17 26 33 43 | 46 88 52 100 53 77
- 17 26 33 43 46 | 88 52 100 53 77
- 17 26 33 43 46 52 | 88 100 53 77
- 17 26 33 43 46 52 53 | 100 88 77
- 17 26 33 43 46 52 53 77 | 88 100
- 17 26 33 43 46 52 53 77 88 | 100

Selection Sort

```
void selectionSort(int numbers[], int array_size) {  
    int i, j, T, min, count;  
    for (i = 0; i < array_size; i++) {  
        min = i;  
        for (j = i + 1; j < array_size; j++) {  
            if (numbers[j] < numbers[min]) {  
                min = j; }  
        }  
        T = numbers[min];  
        numbers[min] = numbers[i];  
        numbers[i] = T;  
    }  
}
```

Time Complexity of Selection Sort

- The total number of comparisons is :-
 $(N-1) + (N-2) + (N-3) + \dots + 1 = N(N-1)/2$
- We can ignore the constant $1/2$
- We can express $N(N-1)$ as $N^2 - N$
- We can ignore N as well since N^2 grows more rapidly than N , making our algorithm $O(N^2)$

Insertion Sort



Insertion Sort

- In insertion sort, each successive element in the array to be sorted is inserted into its proper place with respect to the other, already sorted elements.
- We divide our array in a sorted and an unsorted array
- Initially the sorted portion contains only one element: the first element in the array.
- We take the second element in the array, and put it into its correct place

Insertion Sort

- That is, array[0] and array[1] are in order with respect to each other.
- Then the value in array[2] is put into its proper place, so array [0].... array[2] is sorted and so on.

36
24
10
6
12

36
24
10
6
12

24
36
10
6
12

10
24
36
6
12

6
10
24
36
12

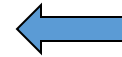
6
10
12
24
36

Insertion Sort

- Our strategy is to search for insertion point from the beginning of the array and shift the element down to make room for new element
- We compare the item at `array[current]` to one before it, and swap if it is less.
- We then compare `array[current-1]` to one before it and swap if necessary.
-

Example: Insertion Sort

- 99 | 55 4 66 28 31 36 52 38 72
- 55 99 | 4 66 28 31 36 52 38 72
- 4 55 99 | 66 28 31 36 52 38 72
- 4 55 66 99 | 28 31 36 52 38 72
- 4 28 55 66 99 | 31 36 52 38 72
- 4 28 31 55 66 99 | 36 52 38 72
- 4 28 31 36 55 66 99 | 52 38 72
- 4 28 31 36 52 55 66 99 | 38 72
- 4 28 31 36 38 52 55 66 99 | 72
- 4 28 31 36 38 52 55 66 72 99 |



Insertion Sort Algorithm

```
void insertionSort(int array[], int length)
{
    int i, j, value;
    for(i = 1; i < length; i++)
    {
        value = a[i];
        for (j = i - 1; j >= 0 && a[ j ] > value; j--)
        {
            a[j + 1] = a[ j ];
        }
        a[j + 1] = value;
    }
}
```


Bubble Sort

Bubble Sort

- Bubble sort is similar to selection sort in the sense that it repeatedly finds the largest/smallest value in the unprocessed portion of the array and puts it back.
- However, finding the largest value is not done by selection this time.
- We "bubble" up the largest value instead.

Bubble Sort

- Compares adjacent items and exchanges them if they are out of order.
- Comprises of several passes.
- In one pass, the largest value has been “bubbled” to its proper position.
- In second pass, the last value does not need to be compared.

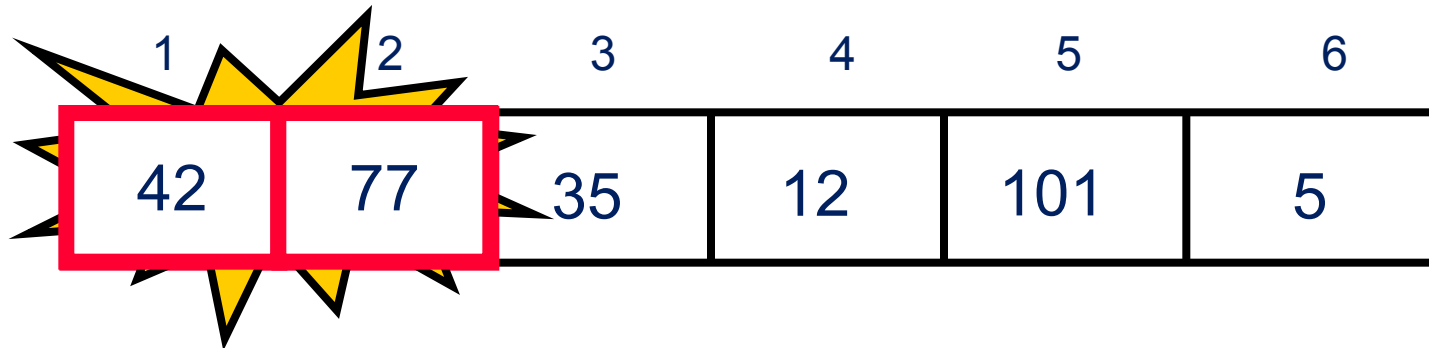
Bubble Sort

- Traverse a collection of elements
 - Move from the front to the end
 - “Bubble” the largest value to the end using pair-wise comparisons and swapping

1	2	3	4	5	6
77	42	35	12	101	5

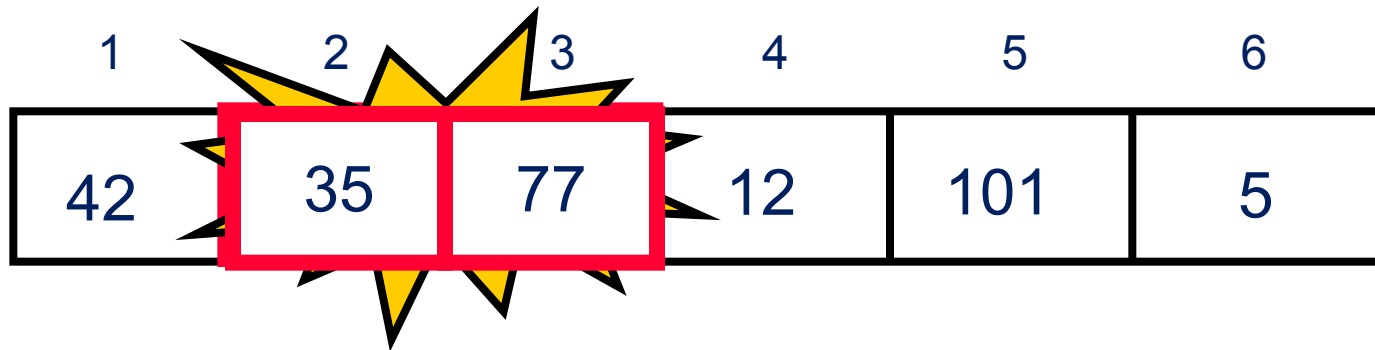
Bubble Sort

- Traverse a collection of elements
 - Move from the front to the end
 - “Bubble” the largest value to the end using pair-wise comparisons and swapping



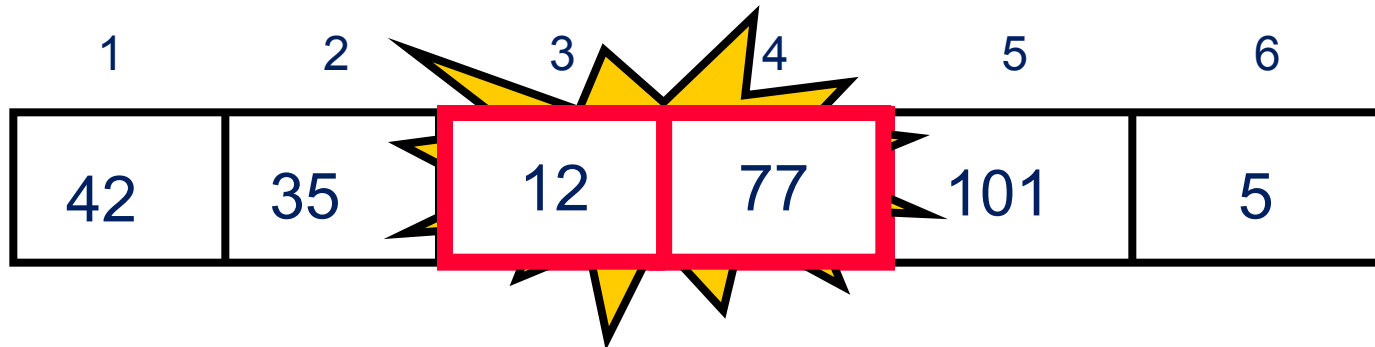
Bubble Sort

- Traverse a collection of elements
 - Move from the front to the end
 - “Bubble” the largest value to the end using pair-wise comparisons and swapping



Bubble Sort

- Traverse a collection of elements
 - Move from the front to the end
 - “Bubble” the largest value to the end using pair-wise comparisons and swapping



Bubble Sort

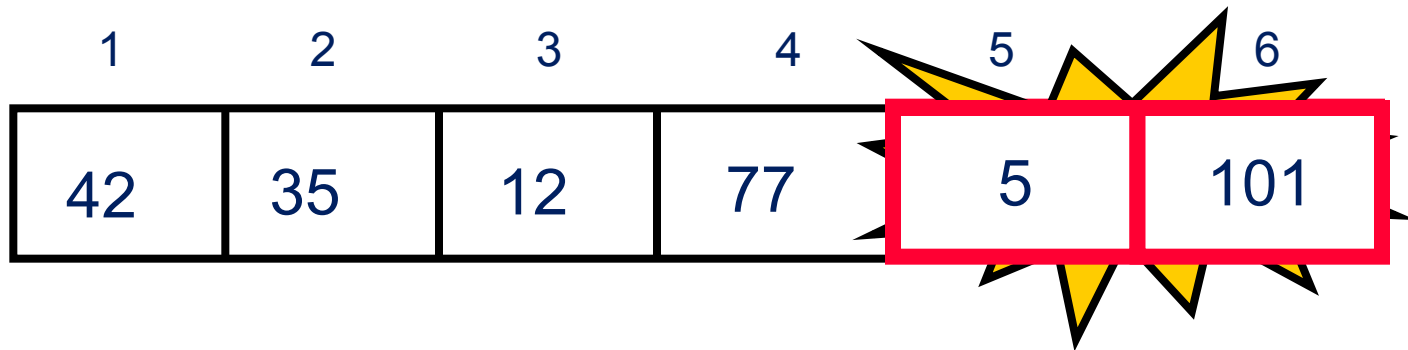
- Traverse a collection of elements
 - Move from the front to the end
 - “Bubble” the largest value to the end using pair-wise comparisons and swapping

1	2	3	4	5	6
42	35	12	77	101	5

No need to swap

Bubble Sort

- Traverse a collection of elements
 - Move from the front to the end
 - “Bubble” the largest value to the end using pair-wise comparisons and swapping



Bubble Sort

- Traverse a collection of elements
 - Move from the front to the end
 - “Bubble” the largest value to the end using pair-wise comparisons and swapping

1	2	3	4	5	6
42	35	12	77	5	101

Largest value correctly placed

Items of Interest

- Notice that only the largest value is correctly placed
- All other values are still out of order
- So we need to repeat this process

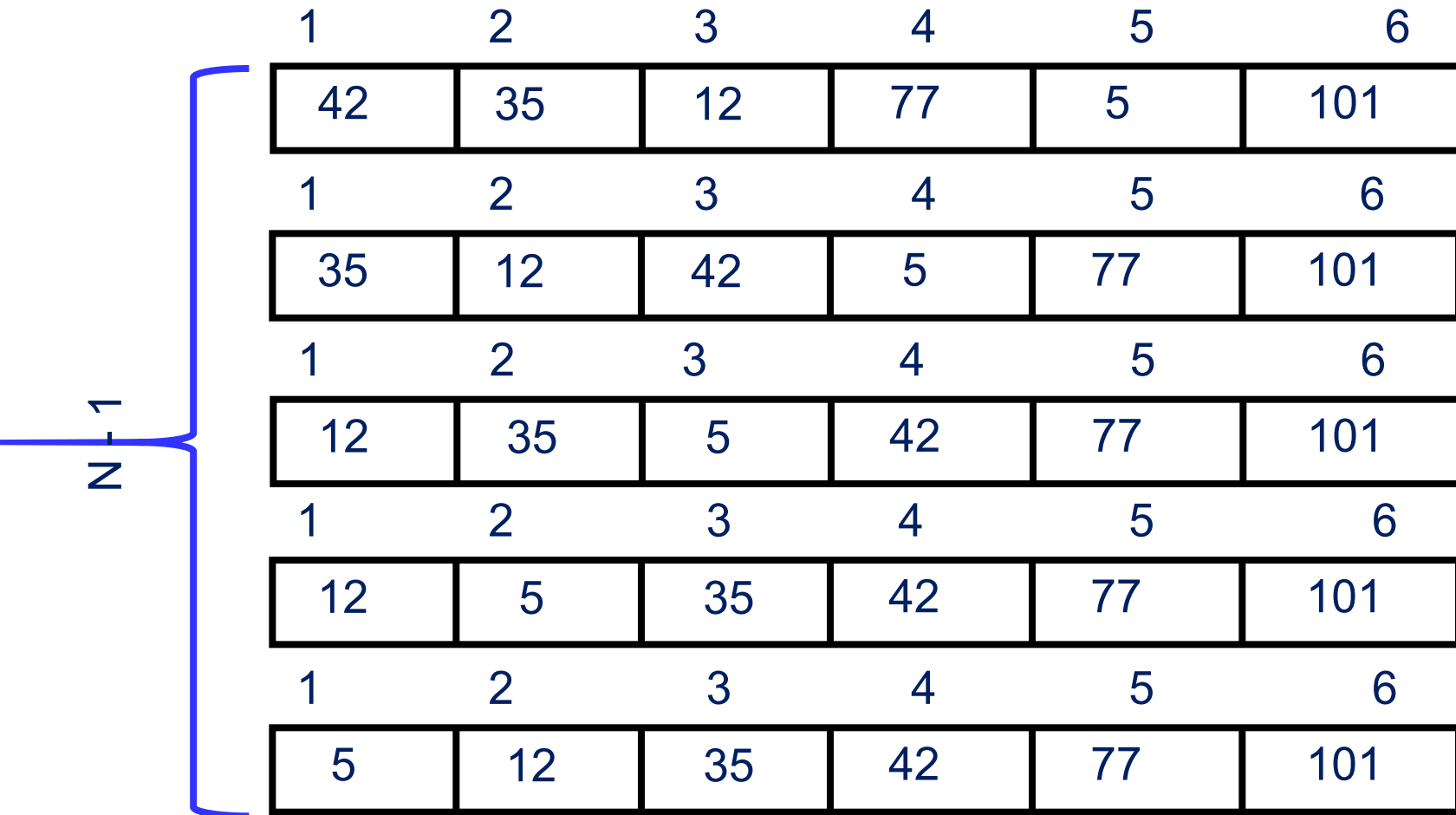
1	2	3	4	5	6
42	35	12	77	5	101

Largest value correctly placed

Repeat “Bubble Up” How Many Times?

- If we have N elements...
- And if each time we bubble an element, we place it in its correct location...
- Then we repeat the “bubble up” process $N - 1$ times.
- This guarantees we’ll correctly place all N elements.

“Bubbling” All the Elements



Reducing the Number of Comparisons

1	2	3	4	5	6
77	42	35	12	101	5
1	2	3	4	5	6
42	35	12	77	5	101
1	2	3	4	5	6
35	12	42	5	77	101
1	2	3	4	5	6
12	35	5	42	77	101
1	2	3	4	5	6
12	5	35	42	77	101

Already Sorted Collections?

- What if the collection was already sorted?
- What if only a few elements were out of place and after a couple of “bubble ups,” the collection was sorted?
- We want to be able to detect this and “stop early”!

1	2	3	4	5	6
5	12	35	42	77	101

Using a Boolean “Flag”

- We can use a boolean variable to determine if any swapping occurred during the “bubble up.”
- If no swapping occurred, then we know that the collection is already sorted!
- This boolean “flag” needs to be reset after each “bubble up.”

Bubble Sort Algorithm

```
void bubbleSort (int S[ ], int length) {  
    bool isSorted = false;  
    while(!isSorted)  
    {  
        isSorted = true;  
        for(int i = 0; i<length; i++)  
        {  
            if(S[i] > S[i+1])  
            {  
                int temp = S[i];  
                S[i] = S[i+1];  
                S[i+1] = temp;  
                isSorted = false;  
            }  
        }  
        length--;  
    }  
}
```

Summary

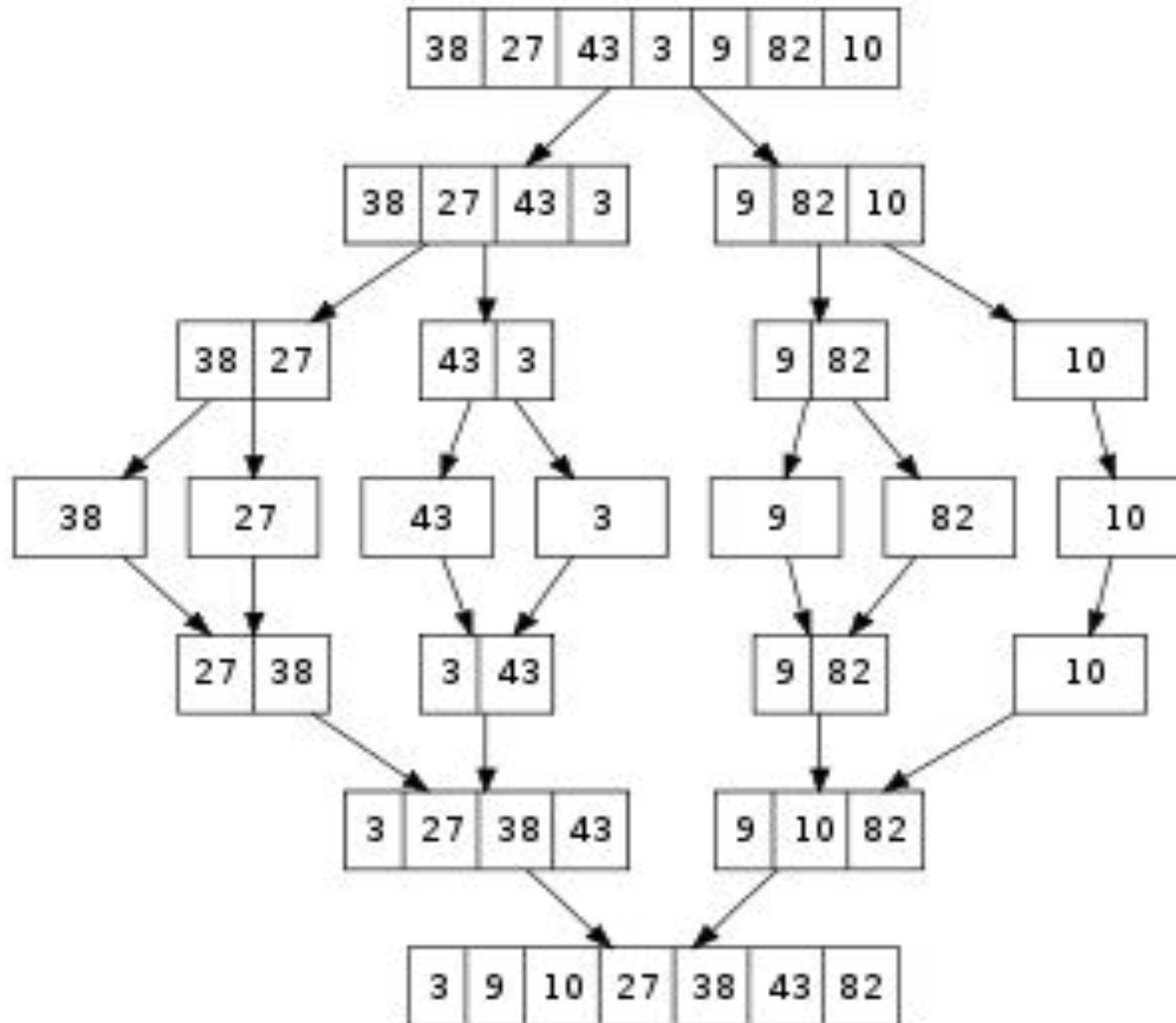
- The insertion sort is a good middle-of-the-road choice for sorting lists of a few thousand items or less.
- The insertion sort is over twice as fast as the bubble sort and almost 40% faster than the selection sort. The insertion sort shouldn't be used for sorting lists larger than a couple thousand items or repetitive sorting of lists larger than a couple hundred items.

Mergesort

Divide and Conquer

- Divide and Conquer cuts the problem in half each time, but uses the result of both halves:
 - cut the problem in half until the problem is trivial
 - solve for both halves
 - combine the solutions

Merge Sort

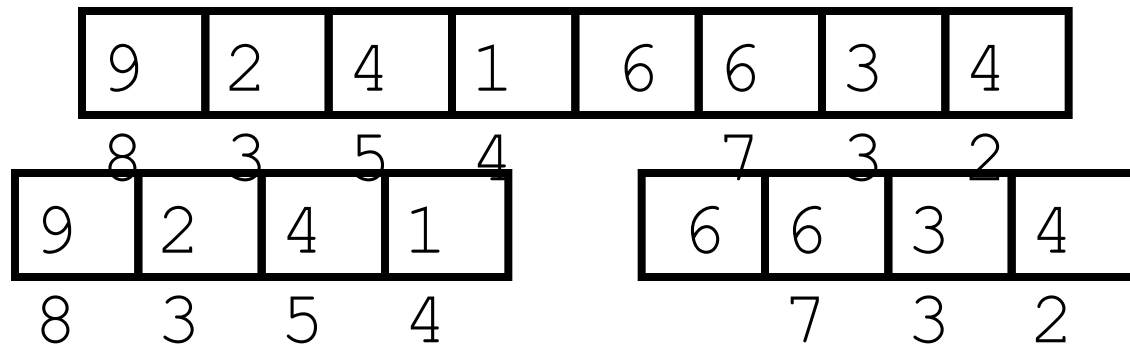


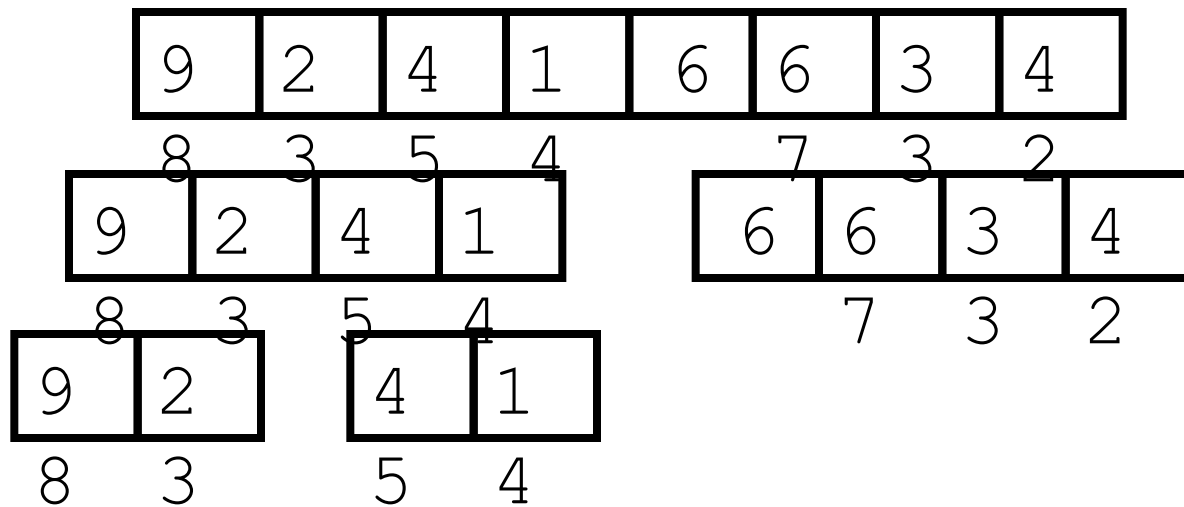
Mergesort

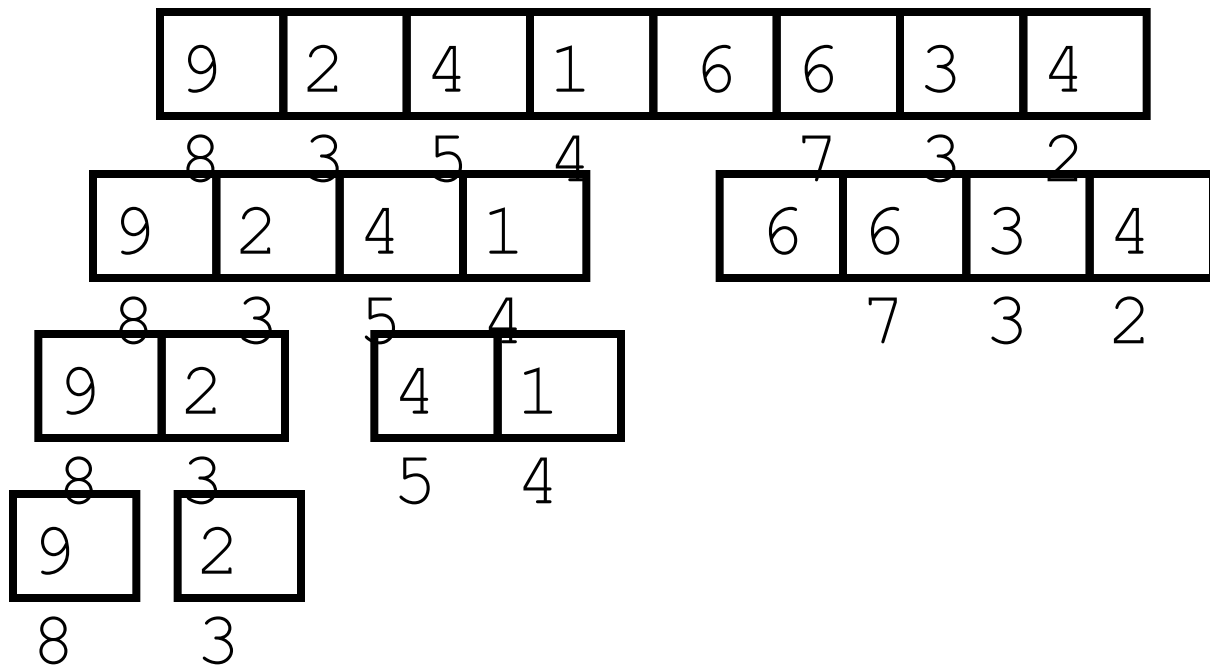
- A divide-and-conquer algorithm:
- Divide the unsorted array into 2 halves until the sub-arrays only contain one element
- Merge the sub-problem solutions together:
 - Compare the sub-array's first elements
 - Remove the smallest element and put it into the result array
 - Continue the process until all elements have been put into the result array

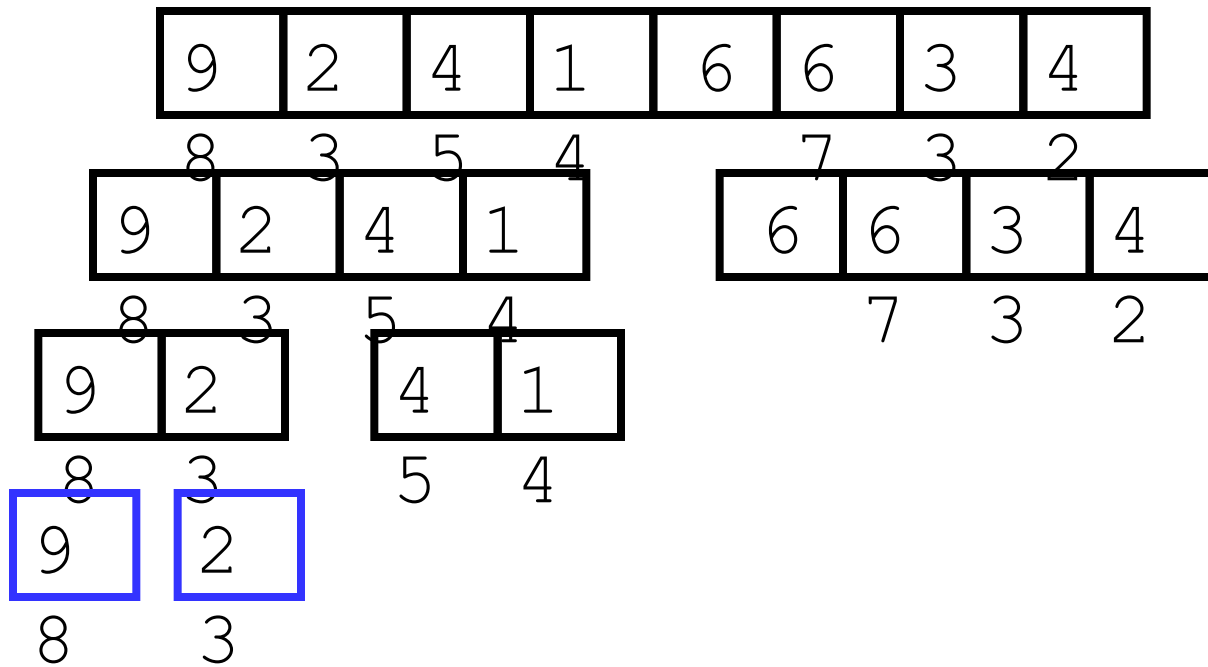
37	23	6	89	15	12	2	19
----	----	---	----	----	----	---	----

9	2	4	1	6	6	3	4
8	3	5	4		7	3	2

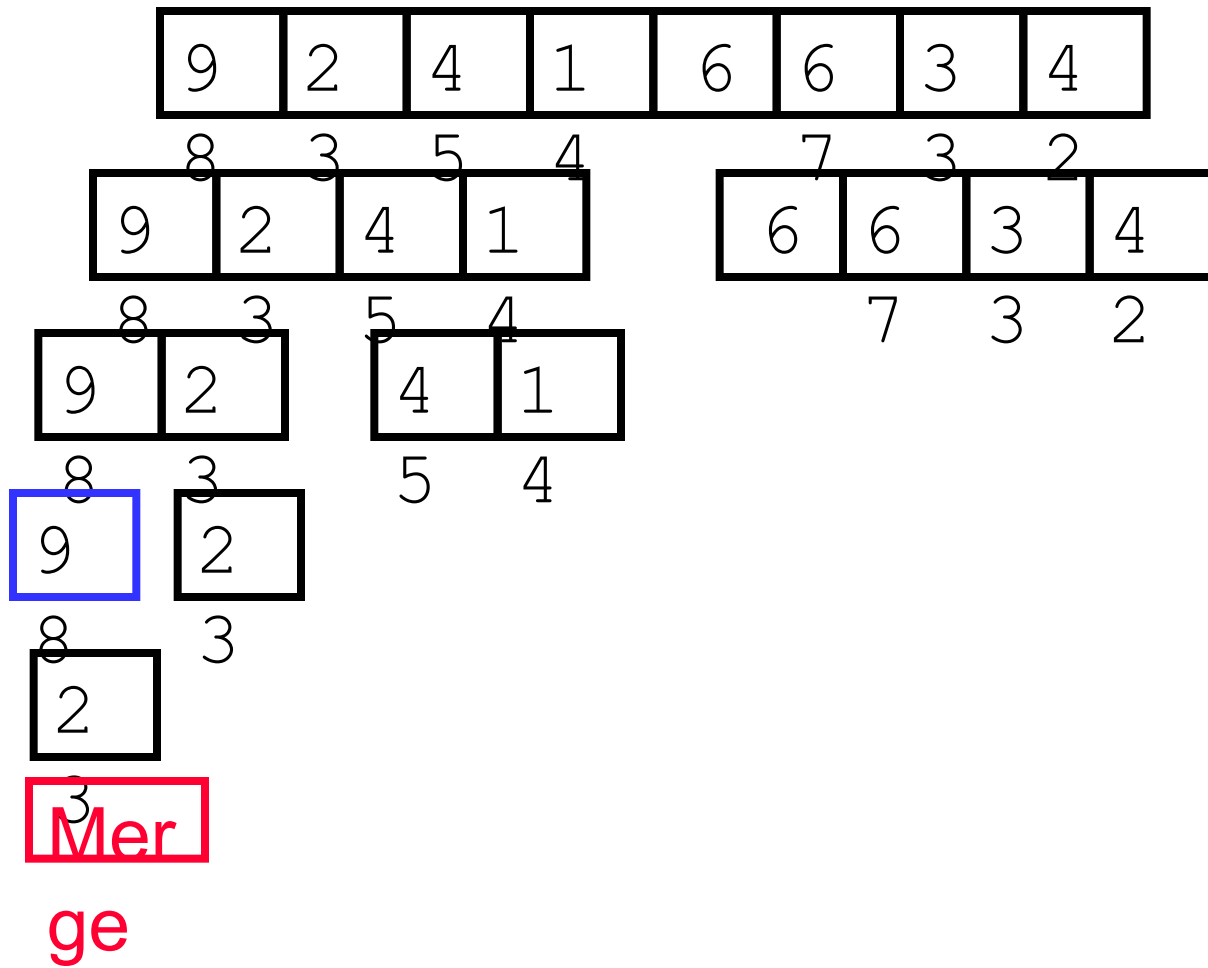


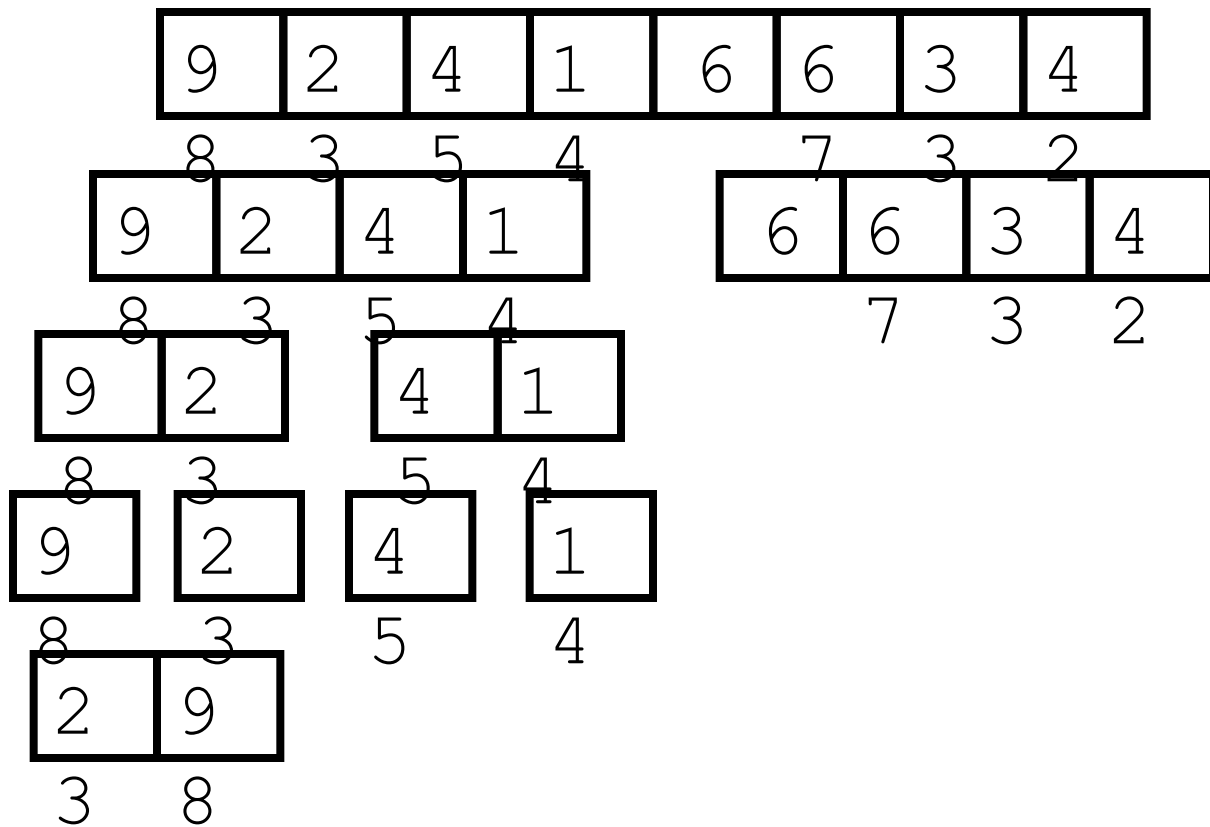


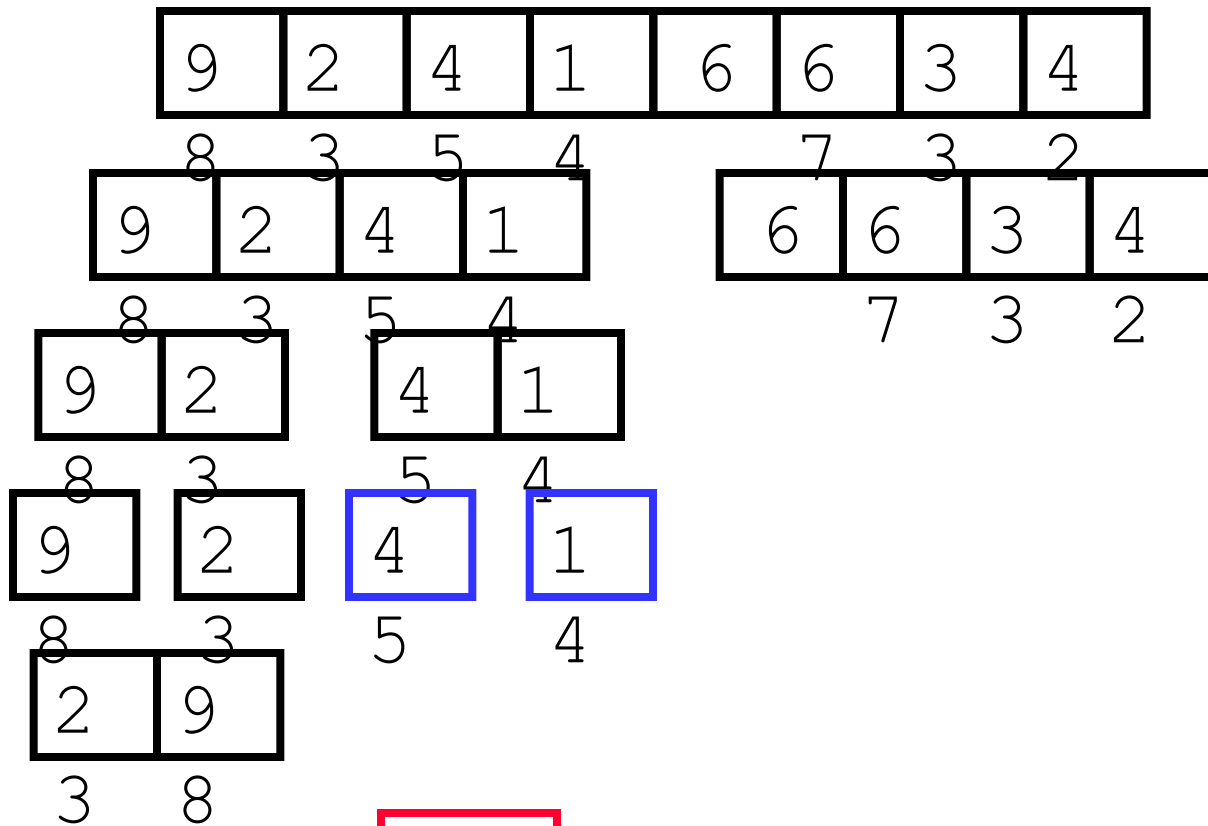




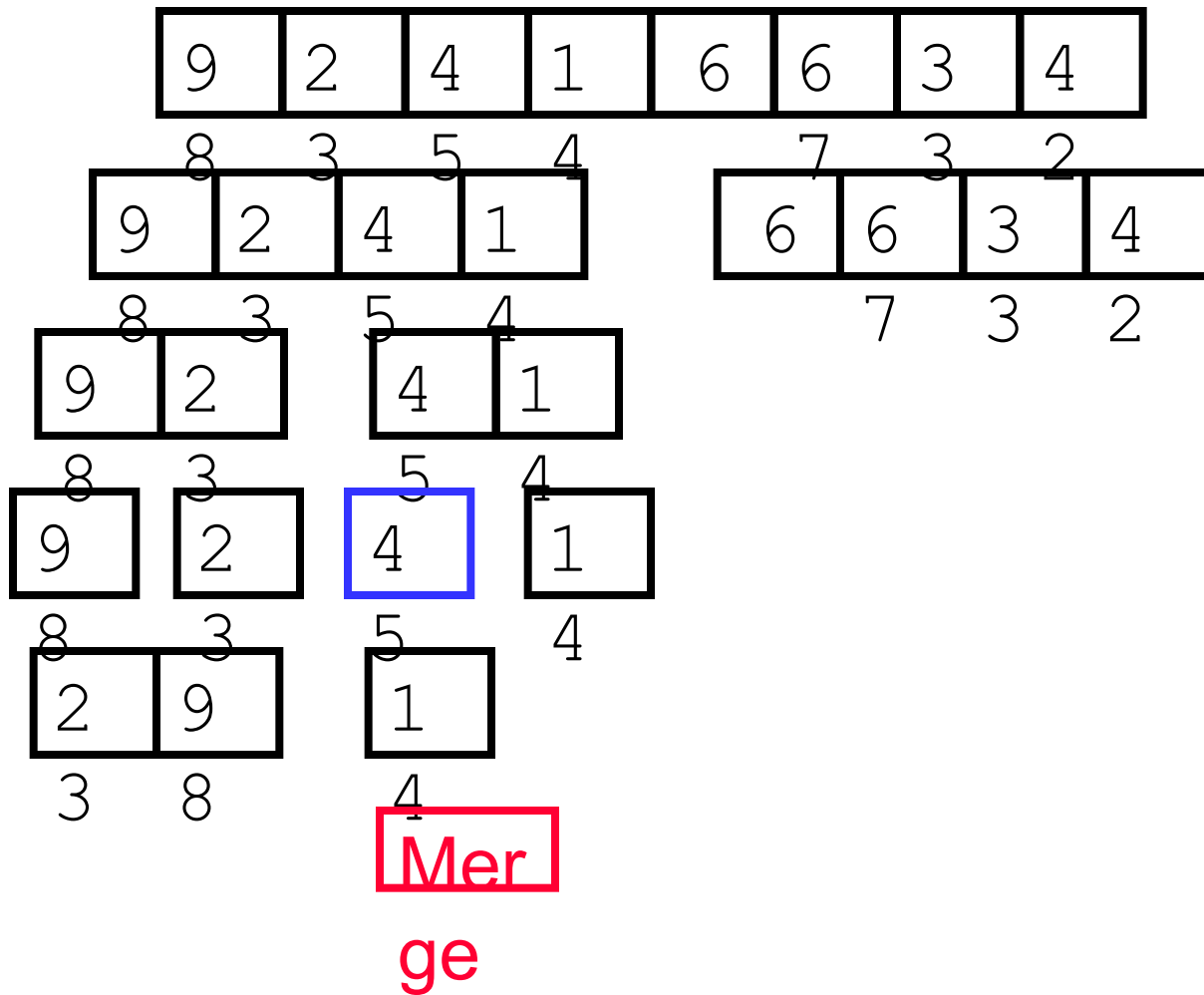
Mer
ge

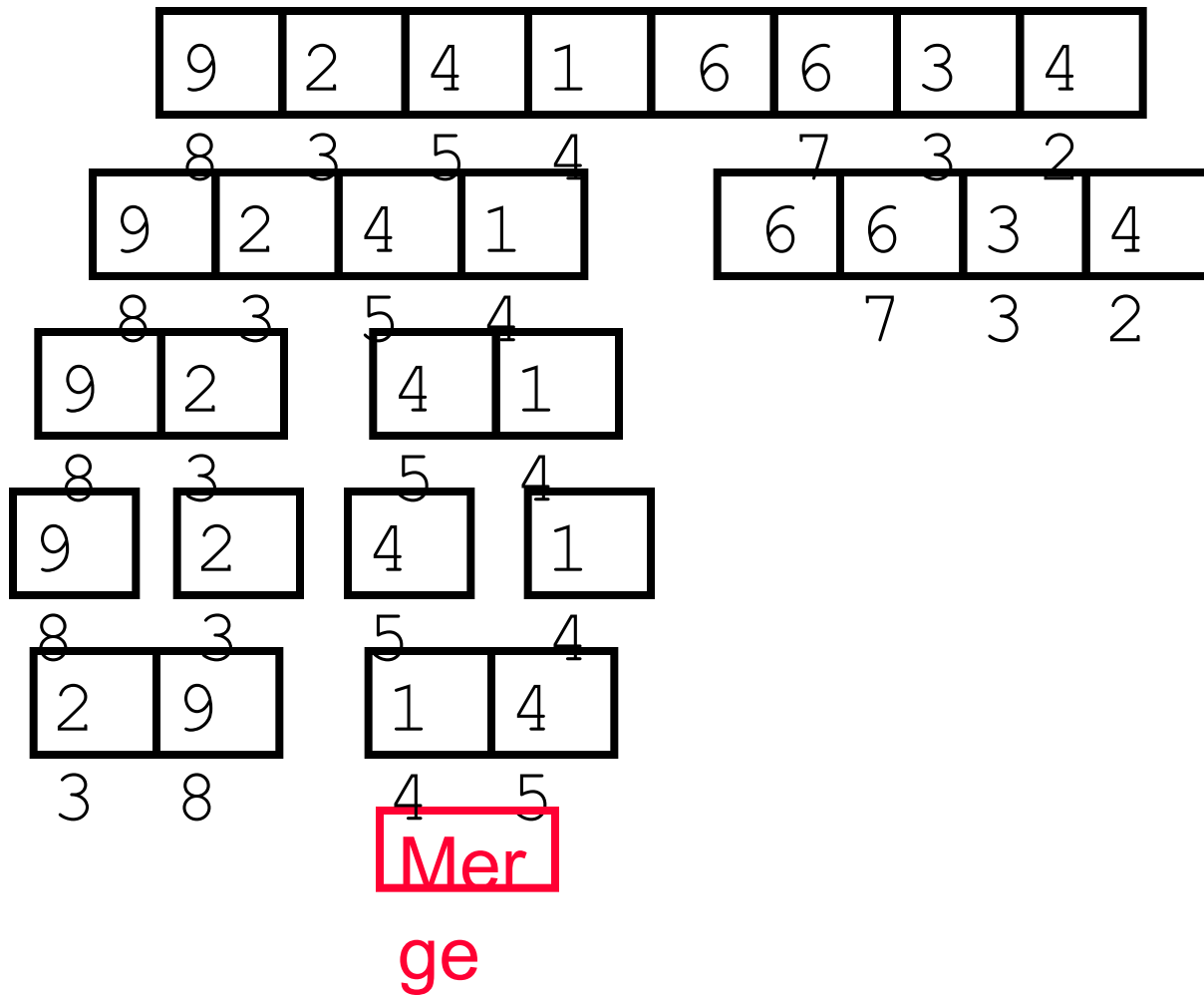


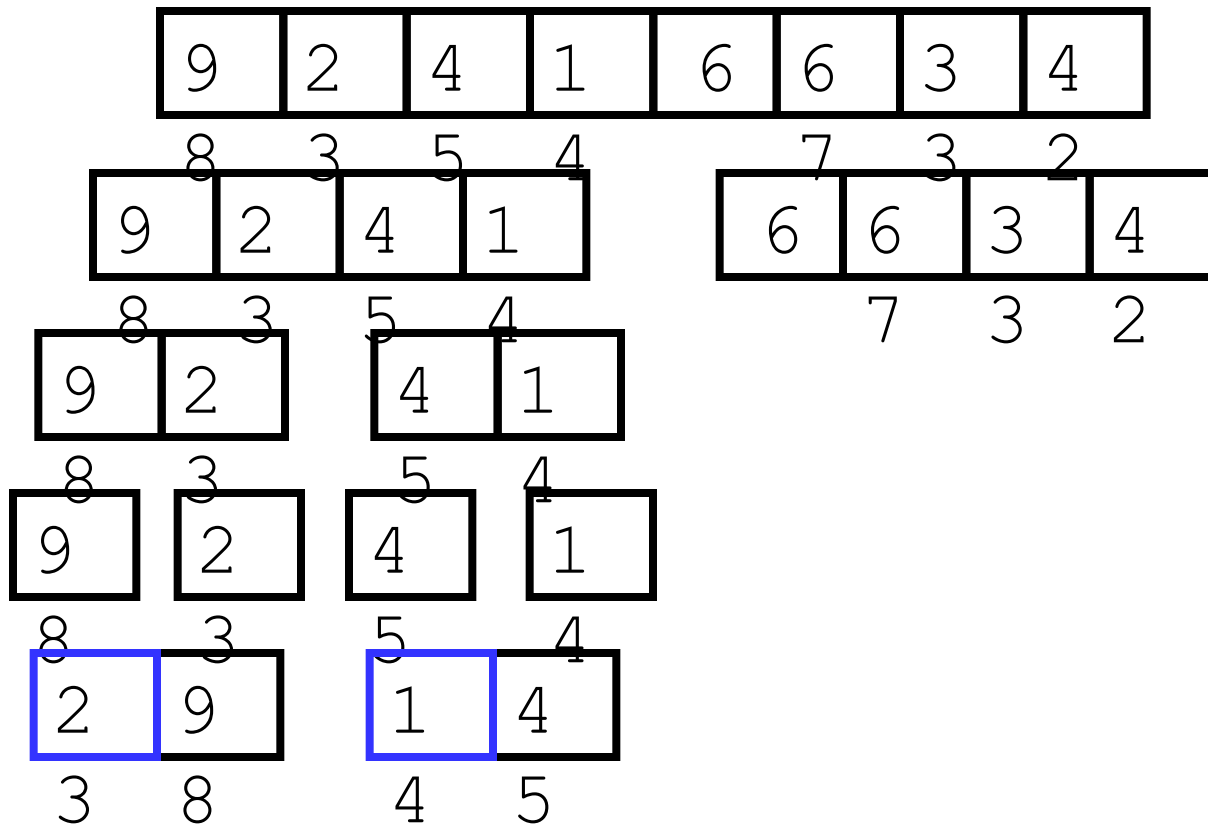


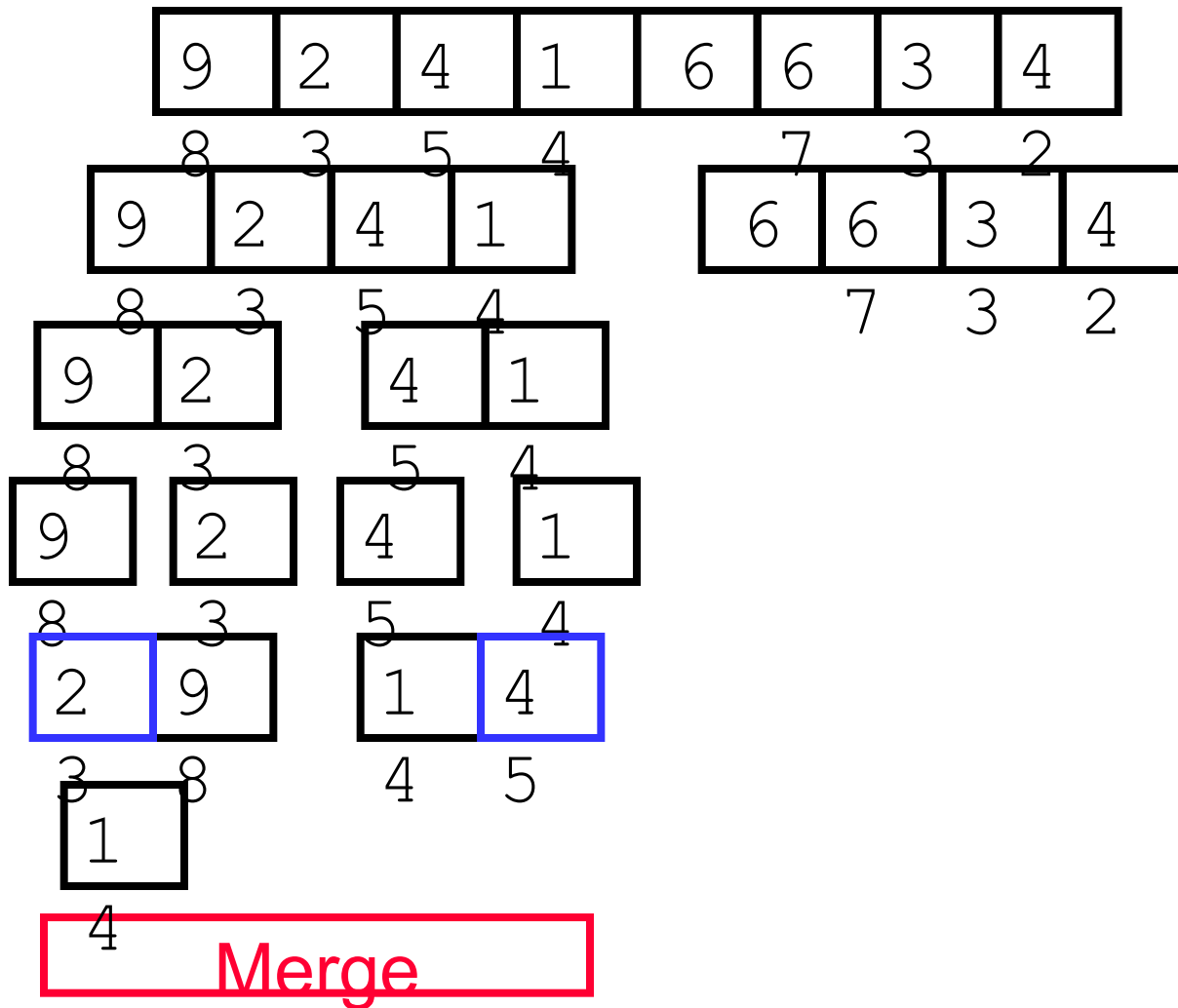


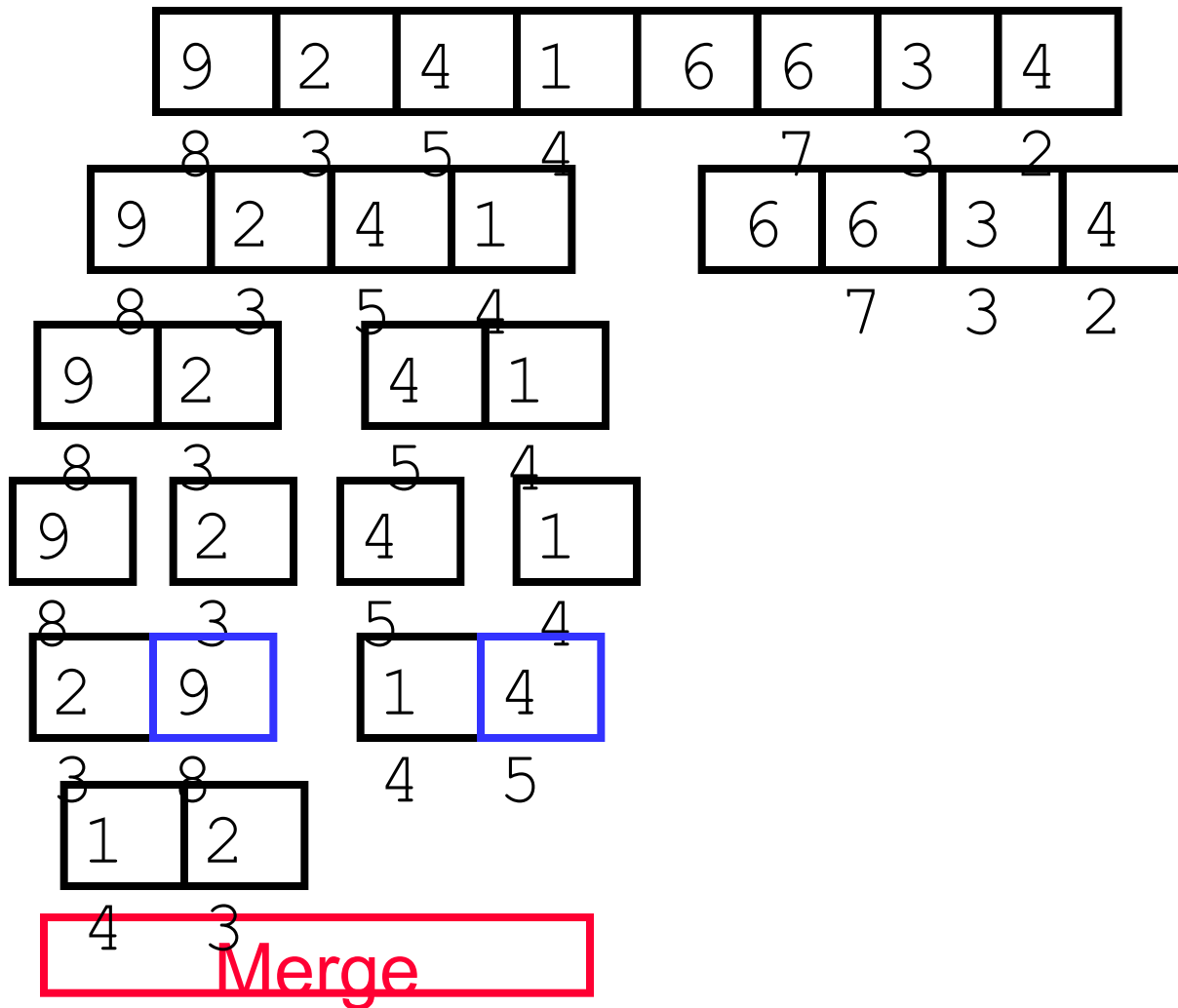
Mer
ge

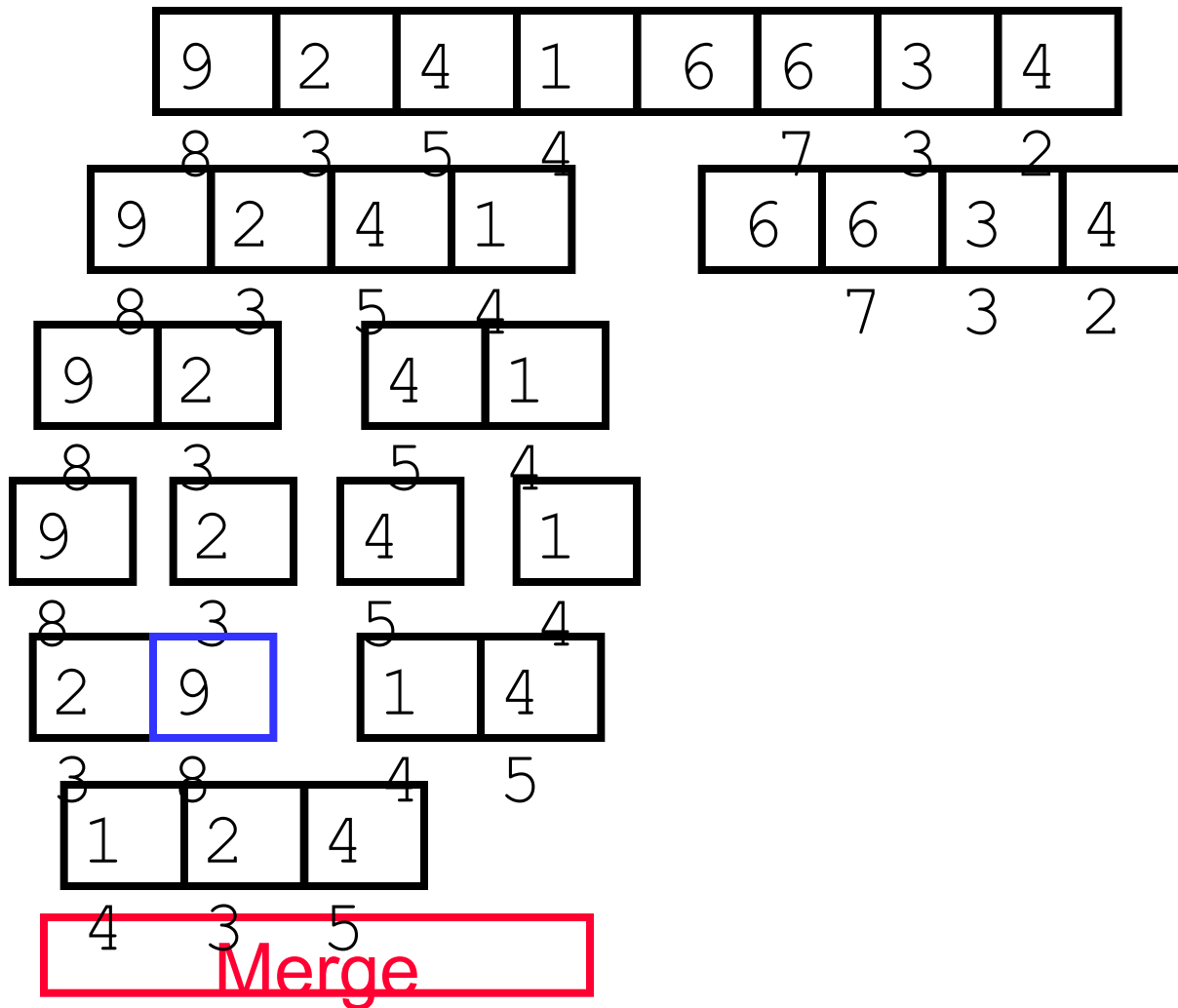


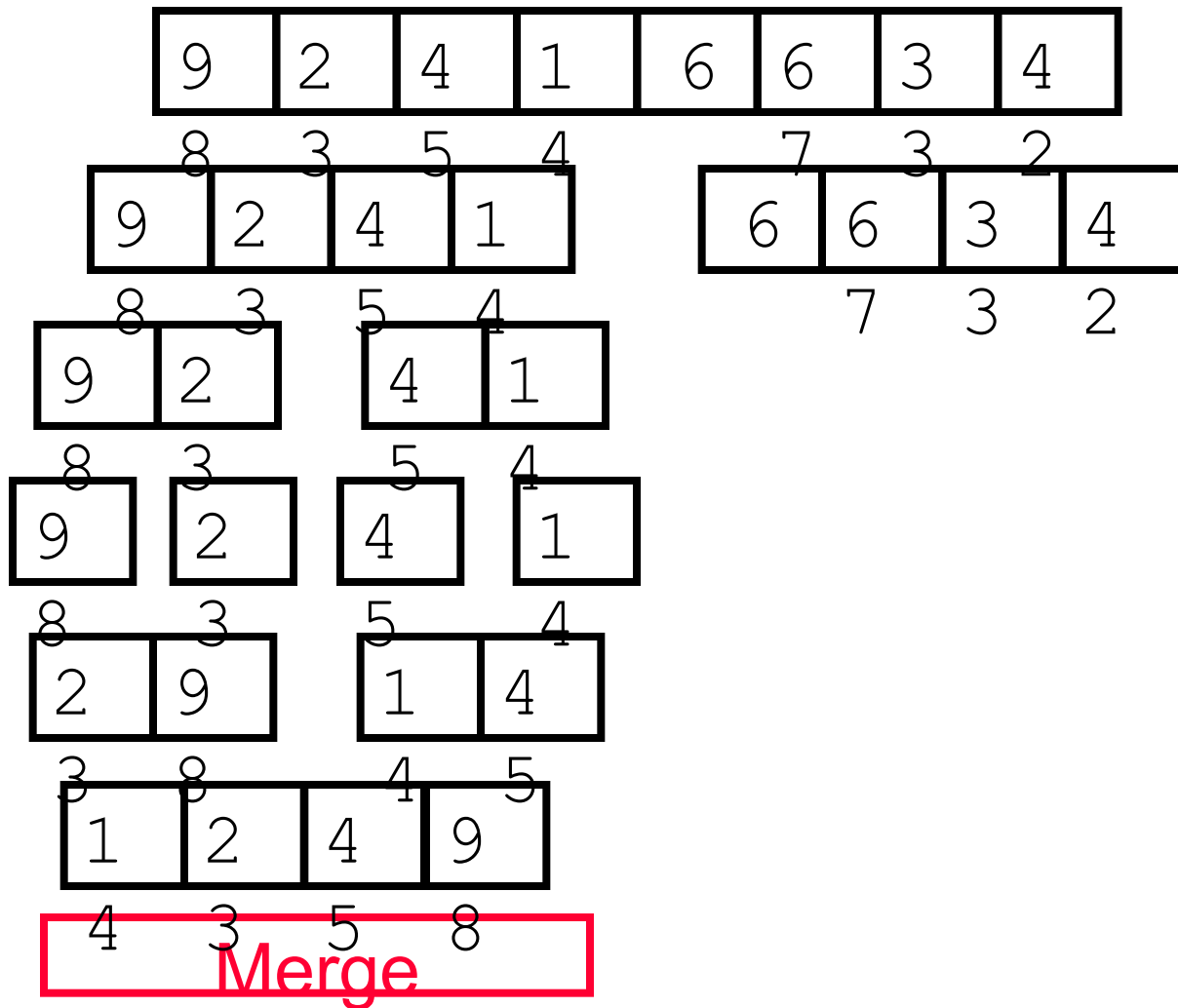


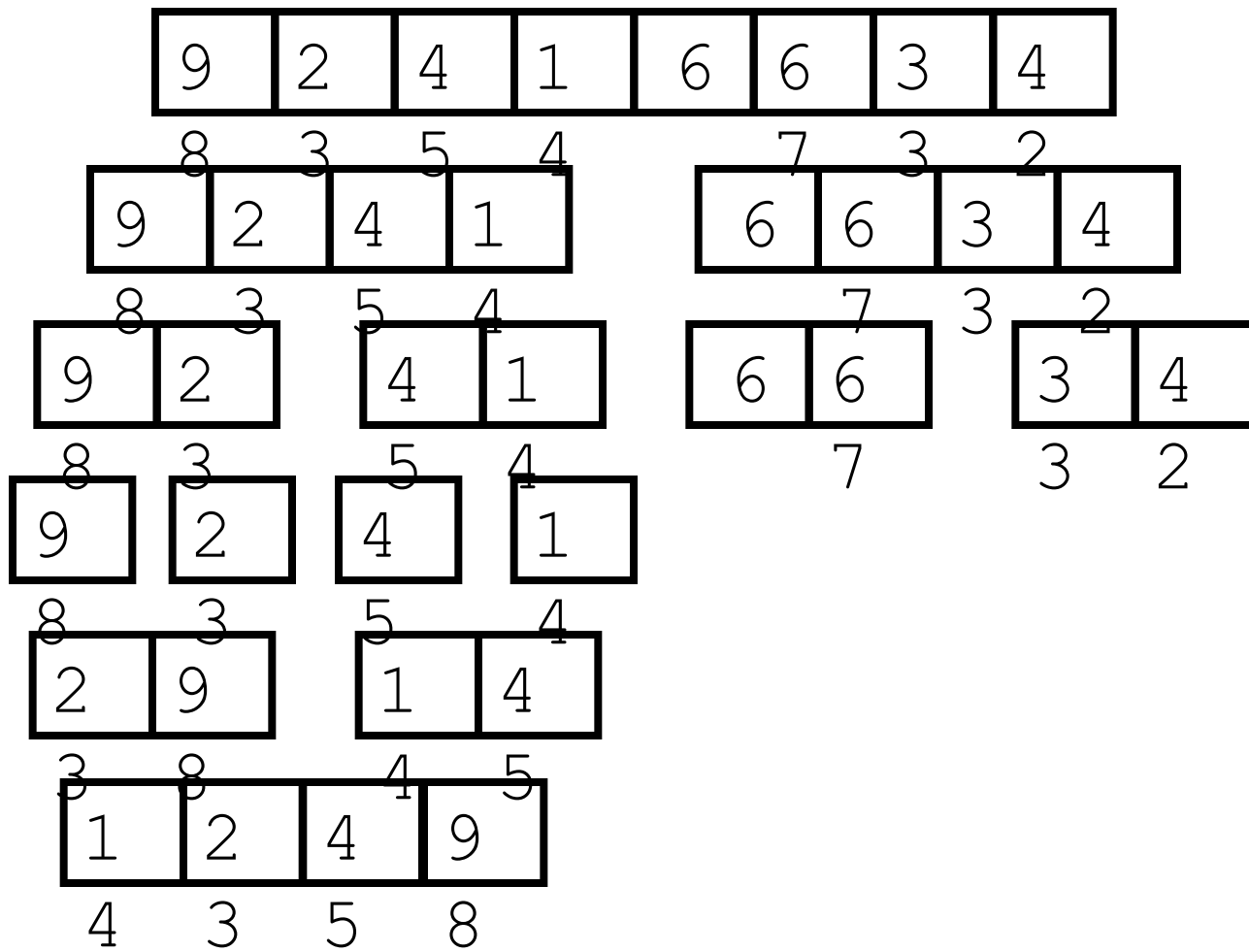


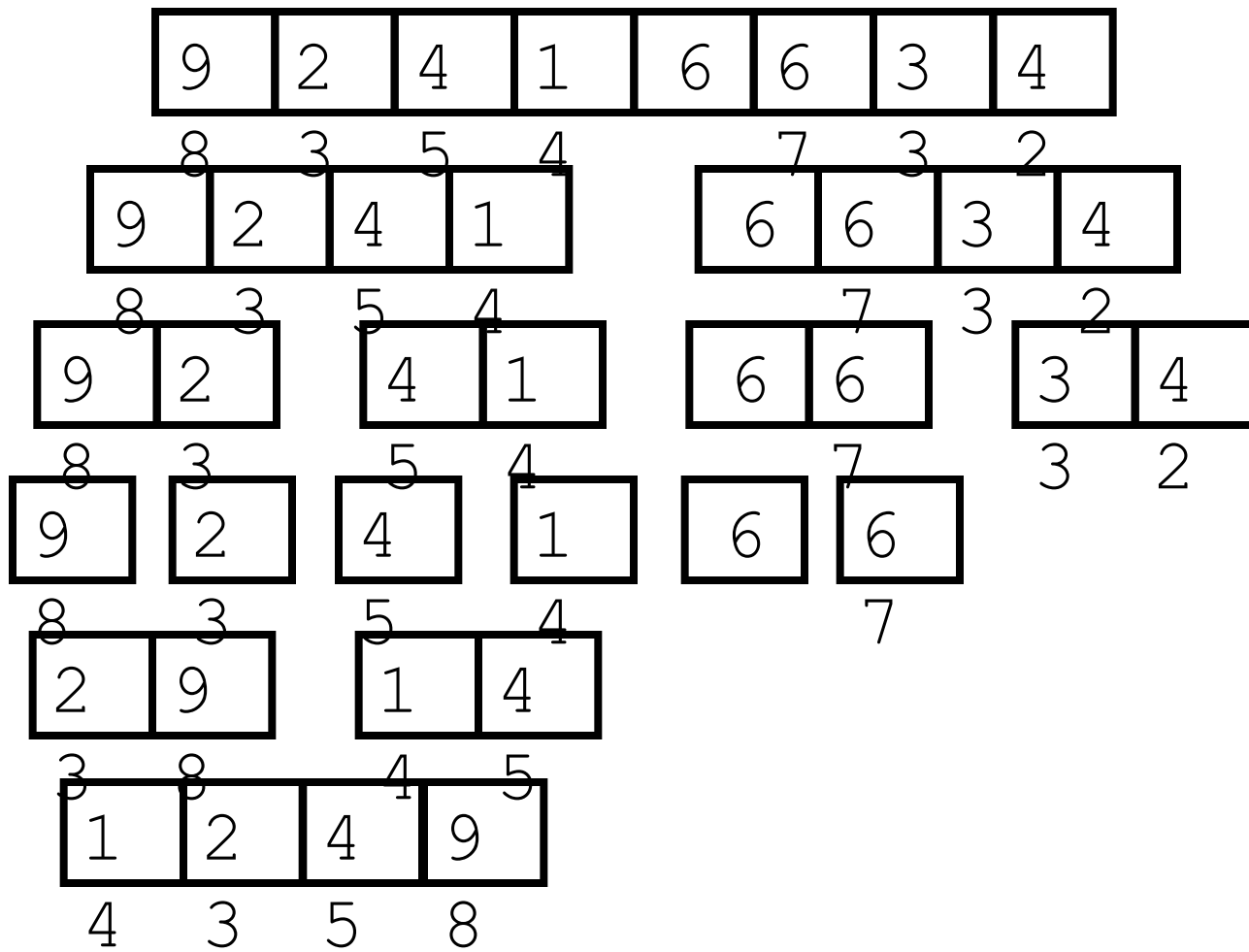


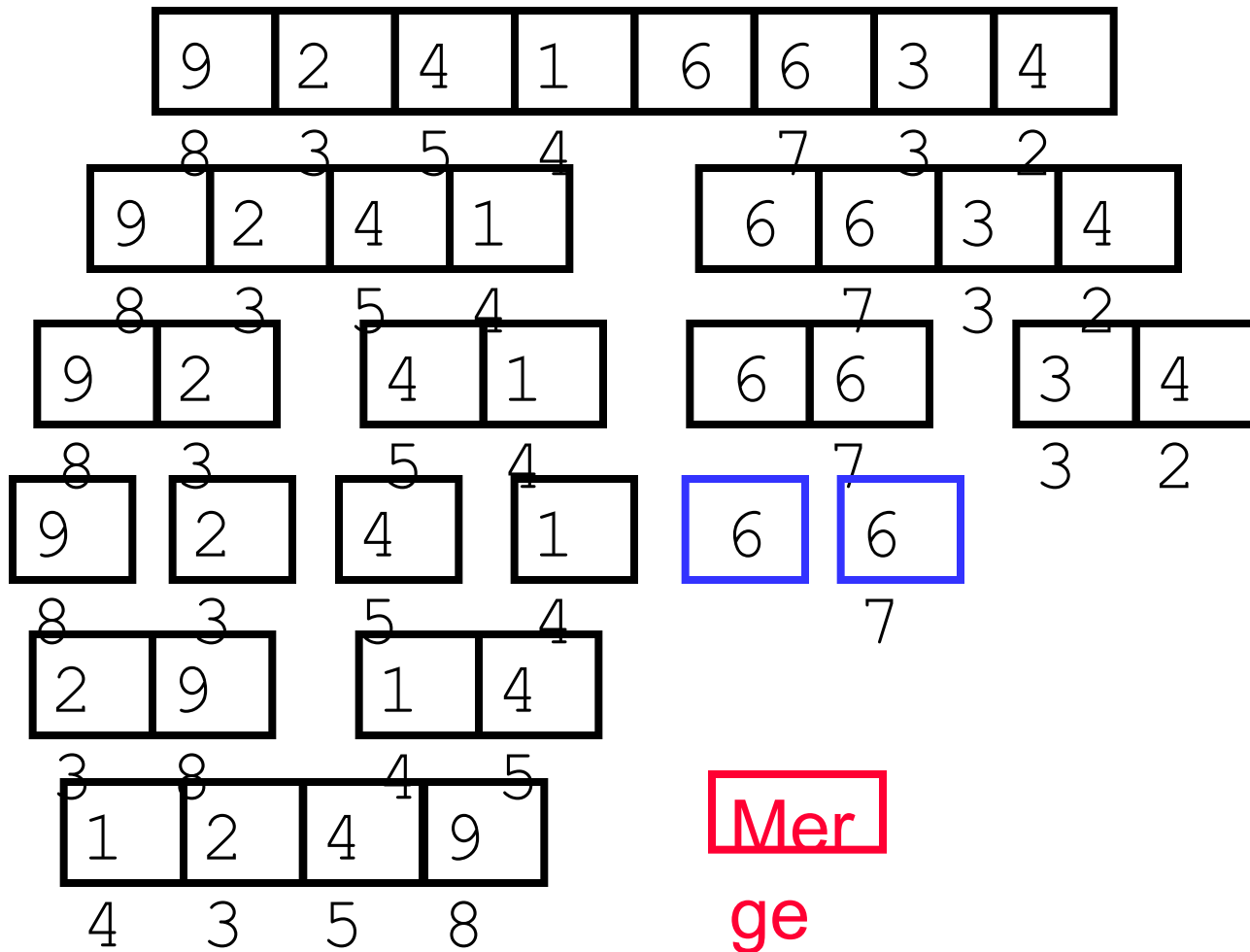


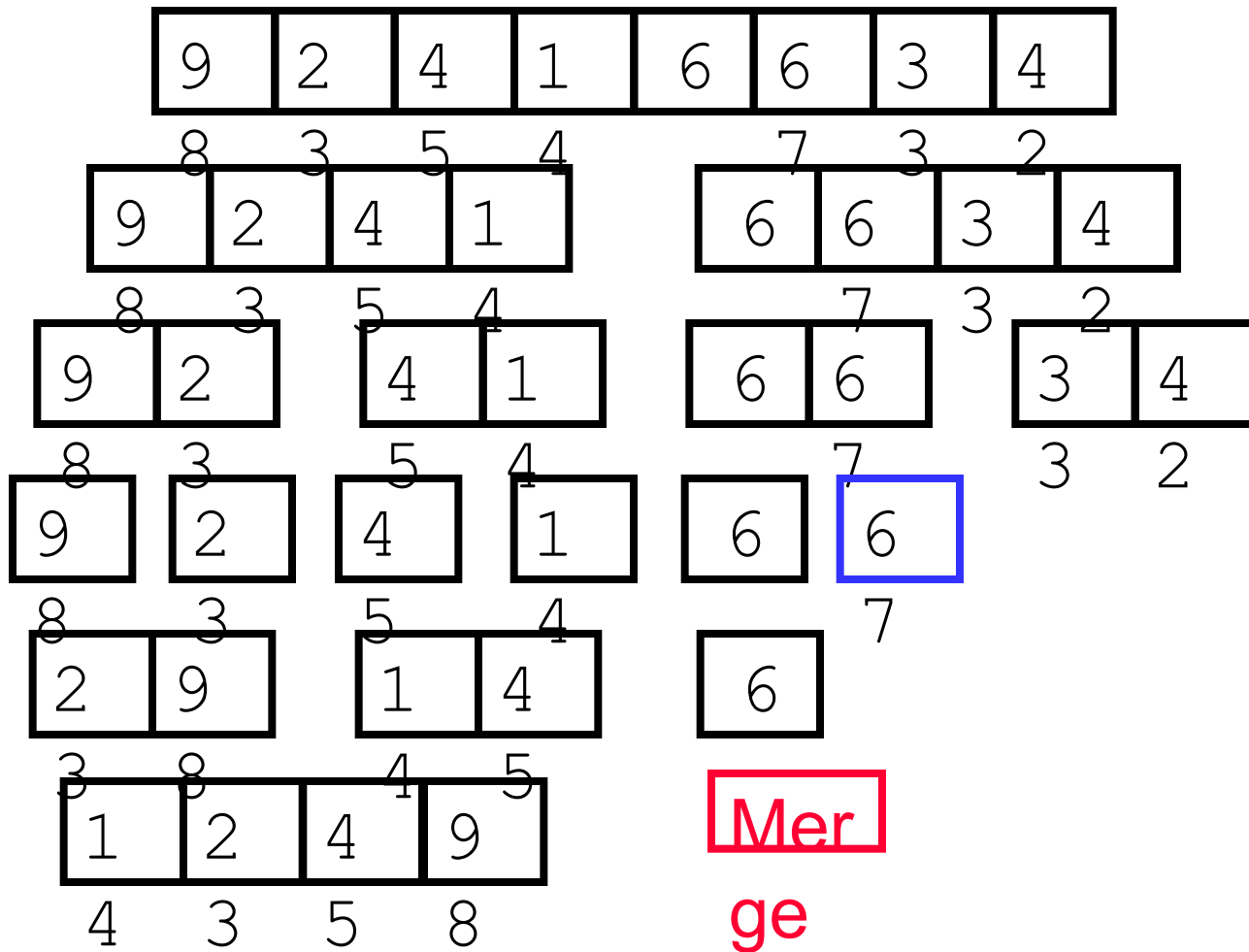


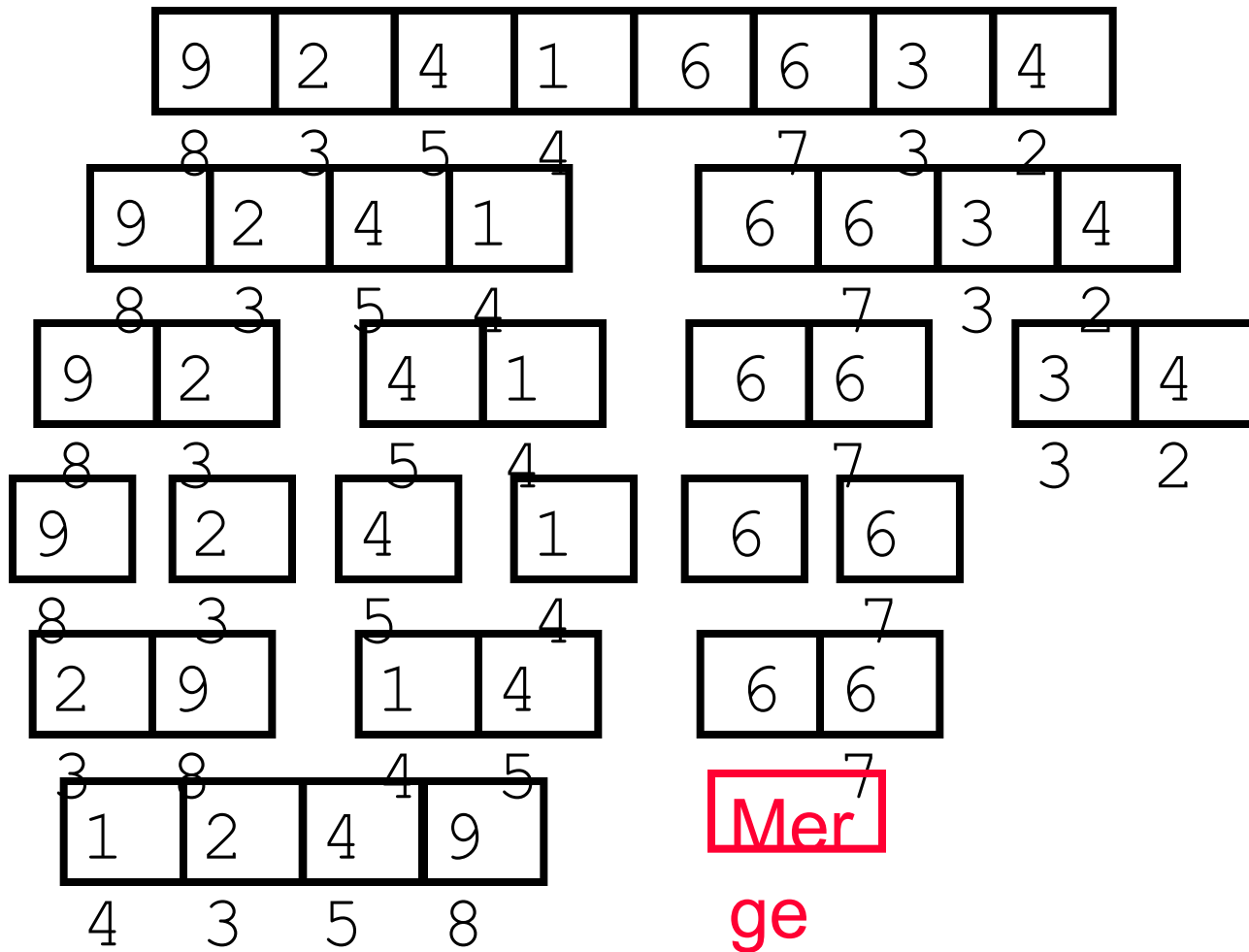


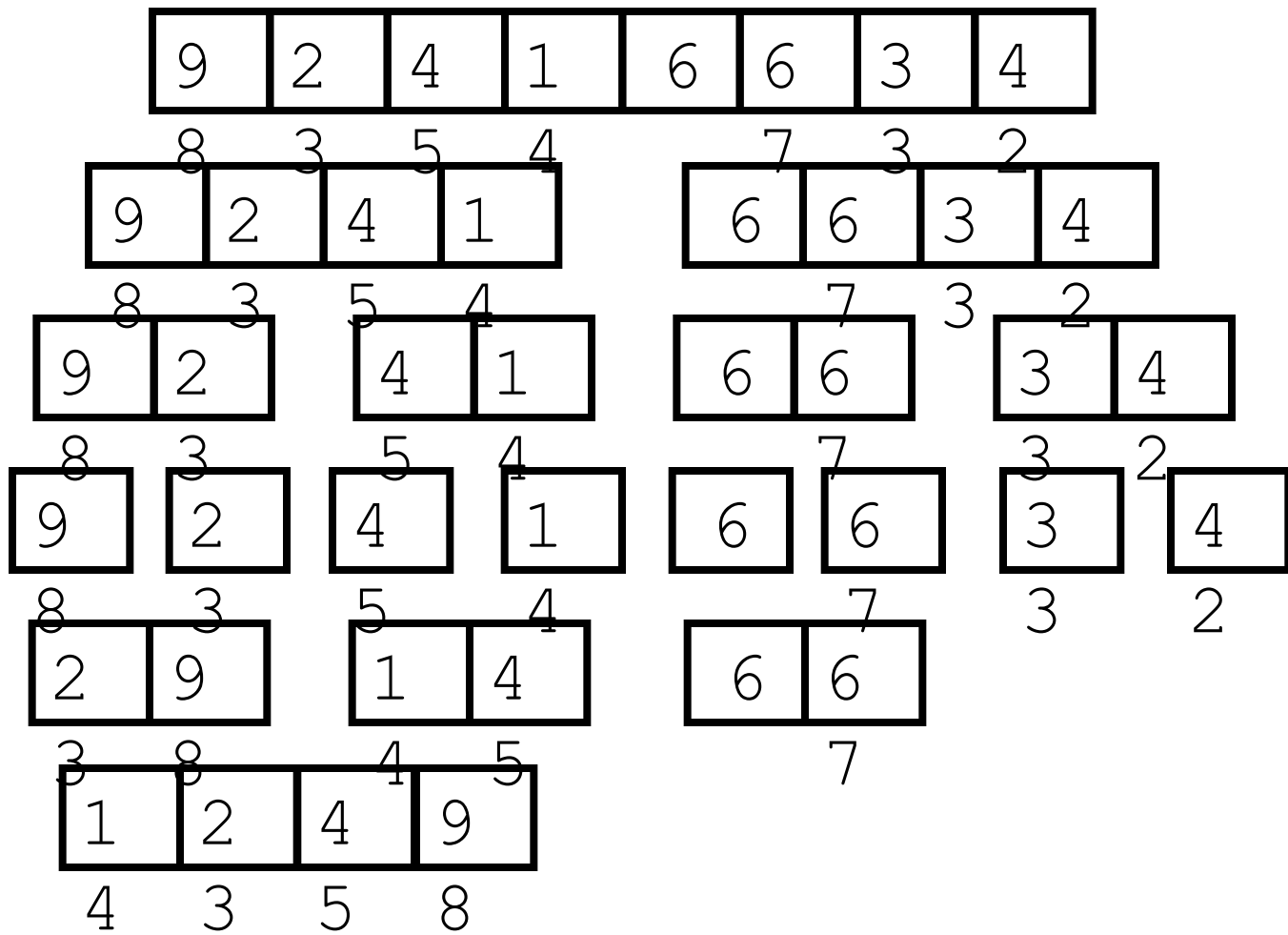


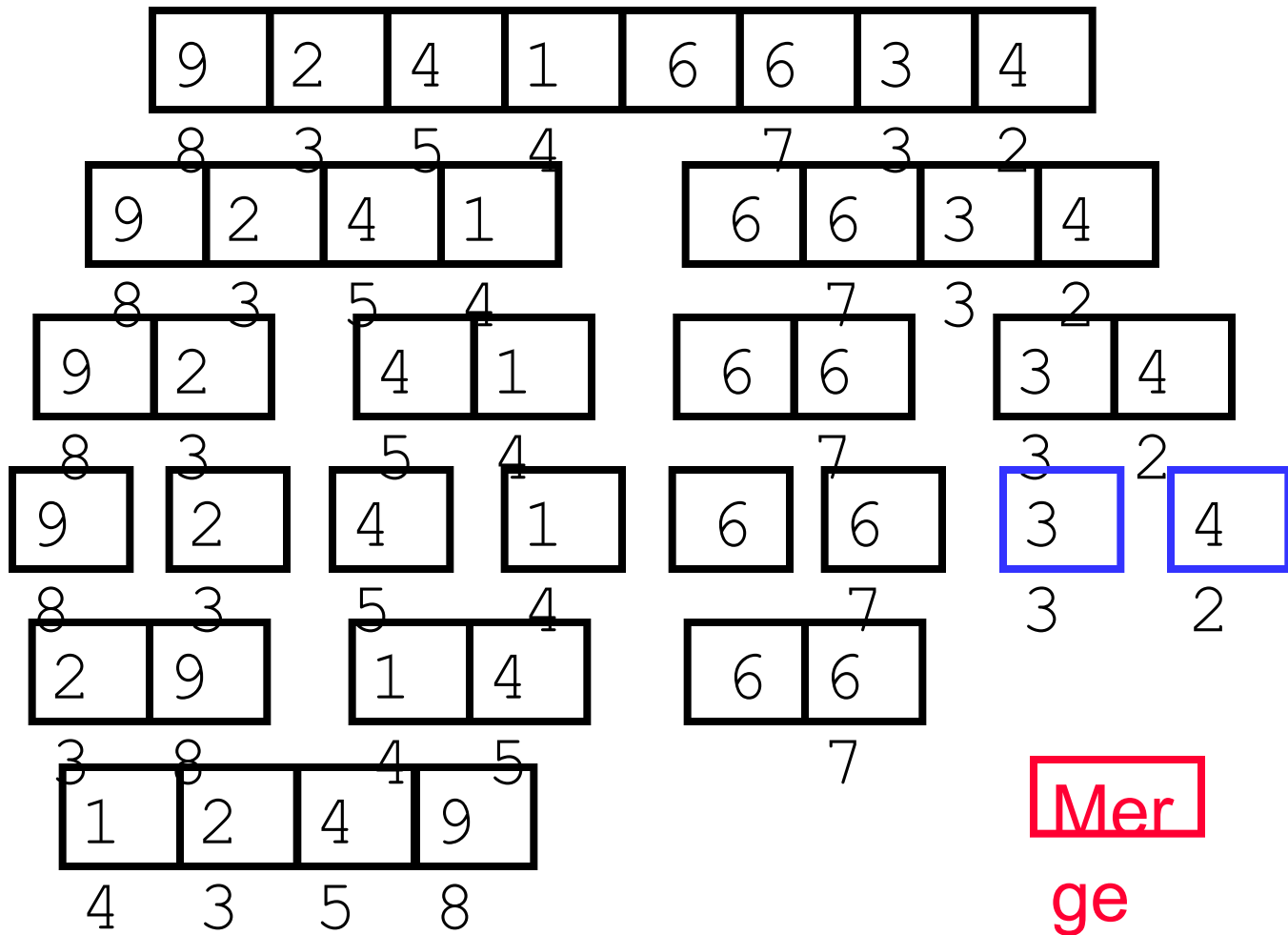


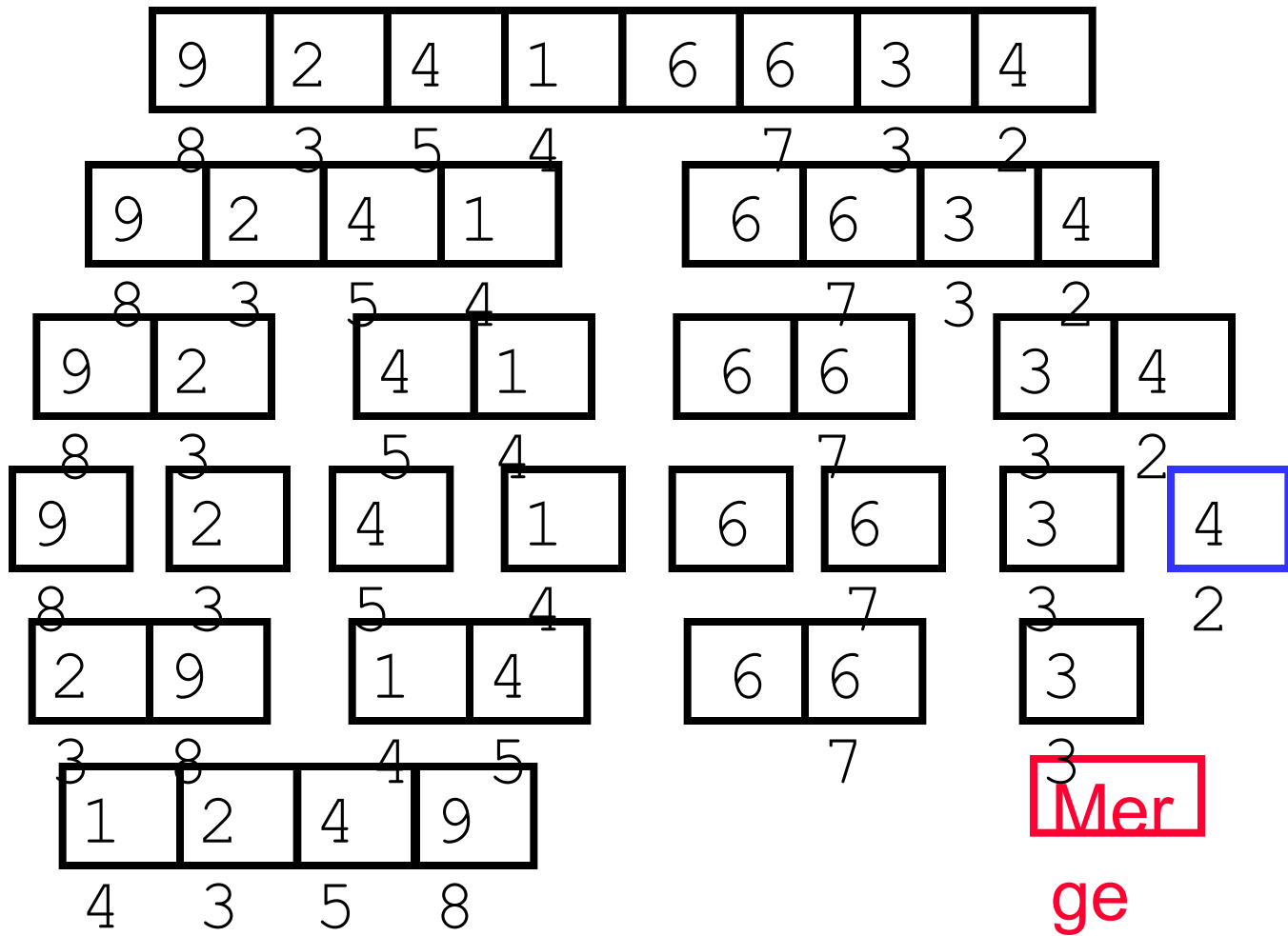


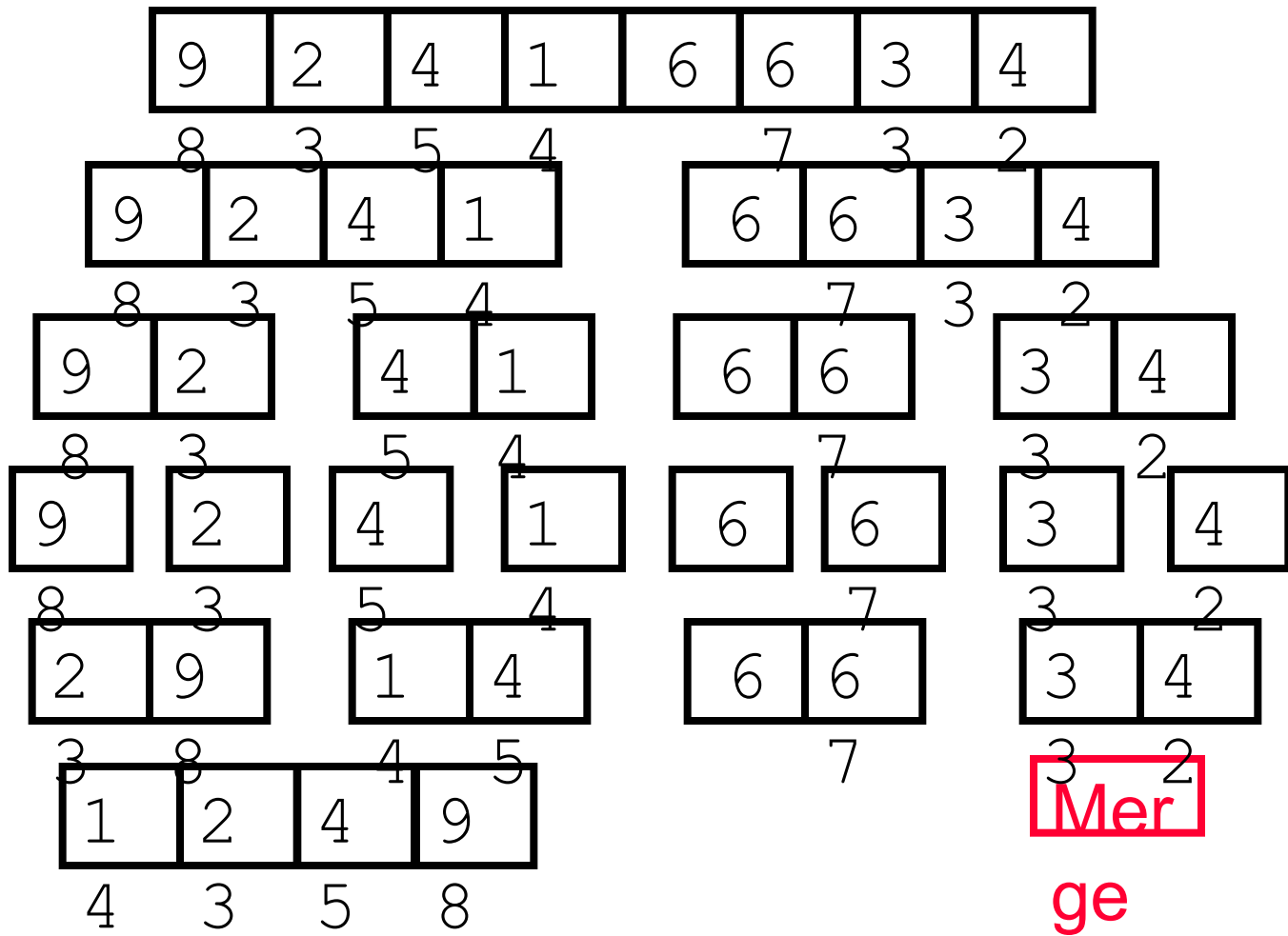


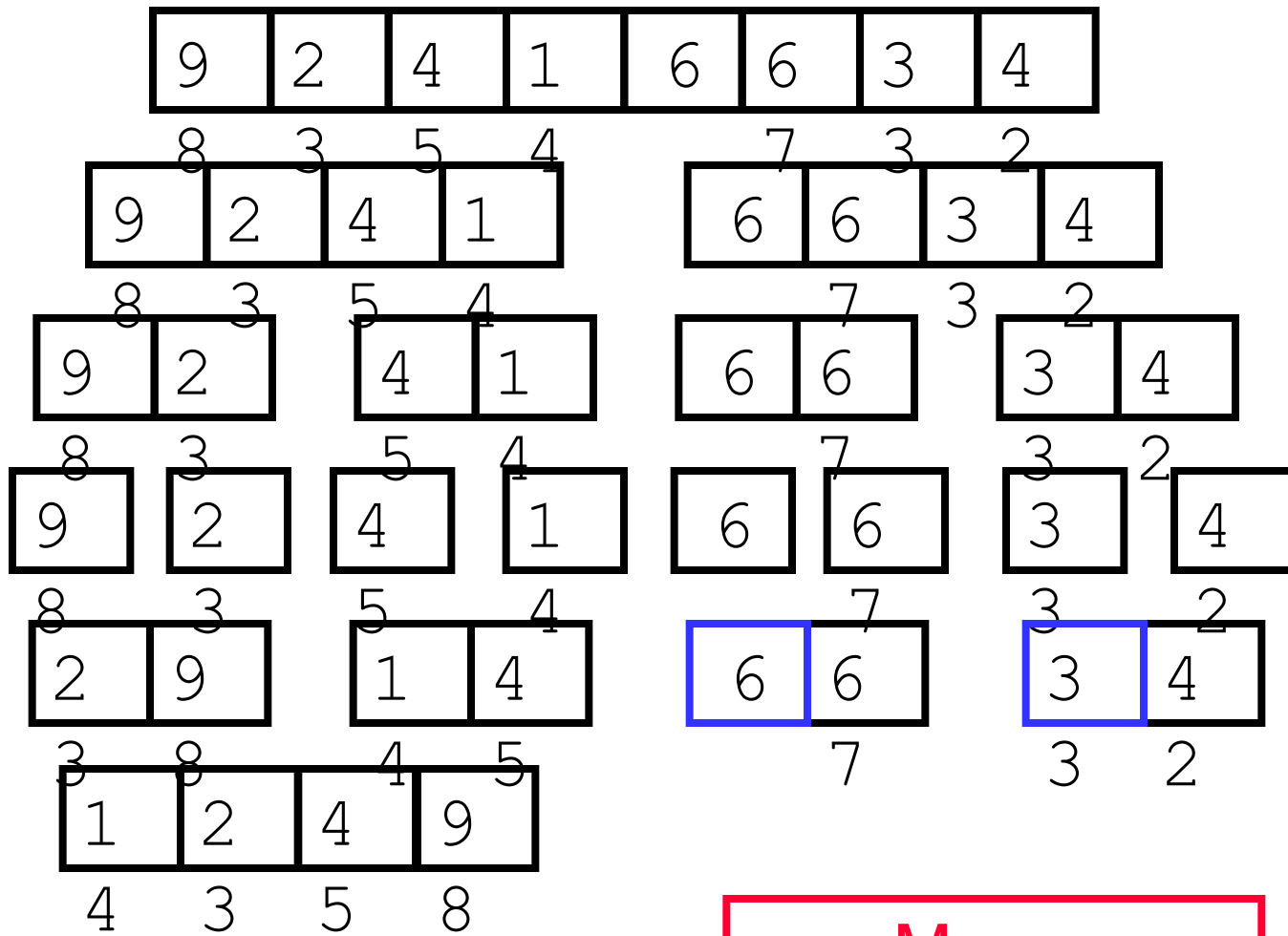




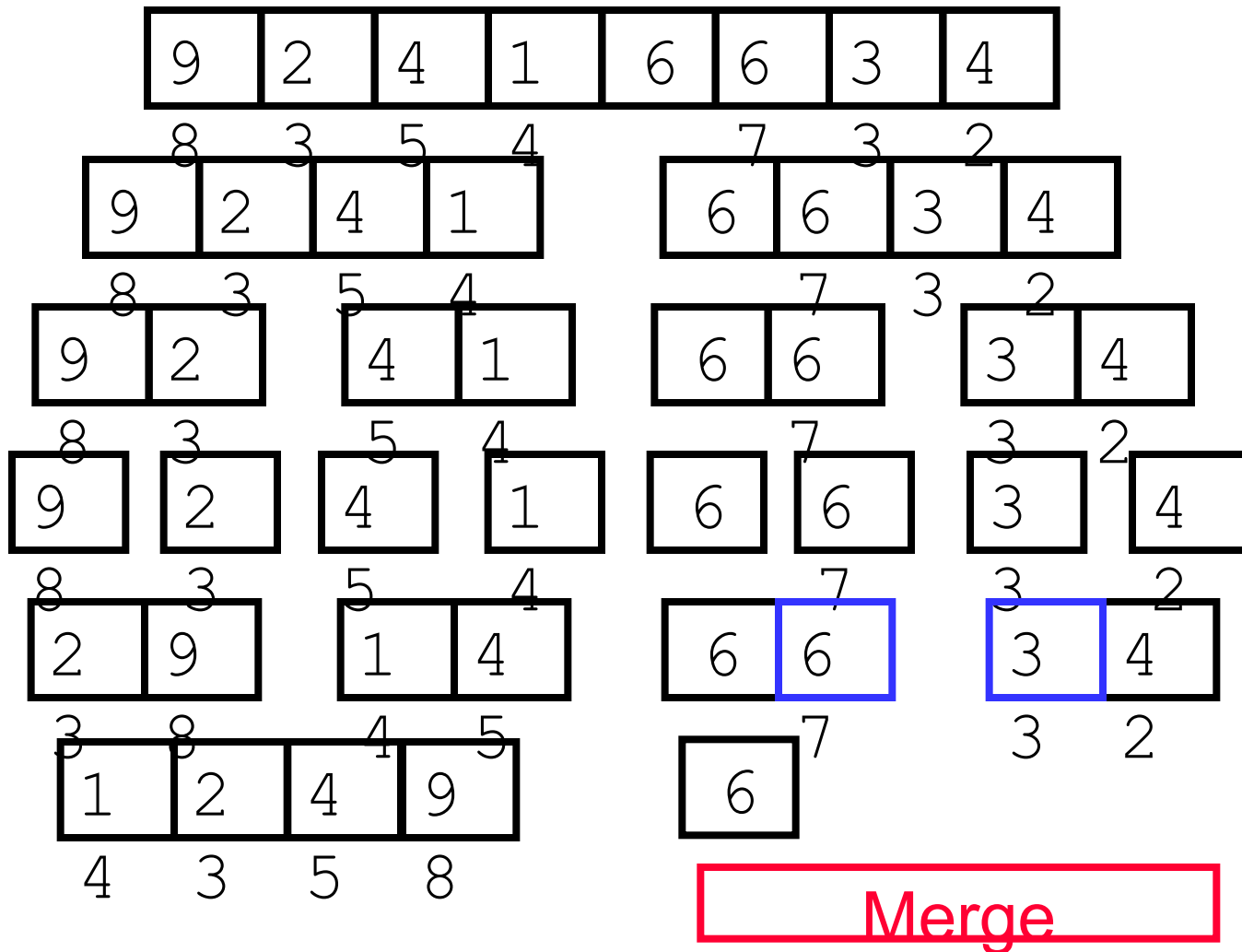


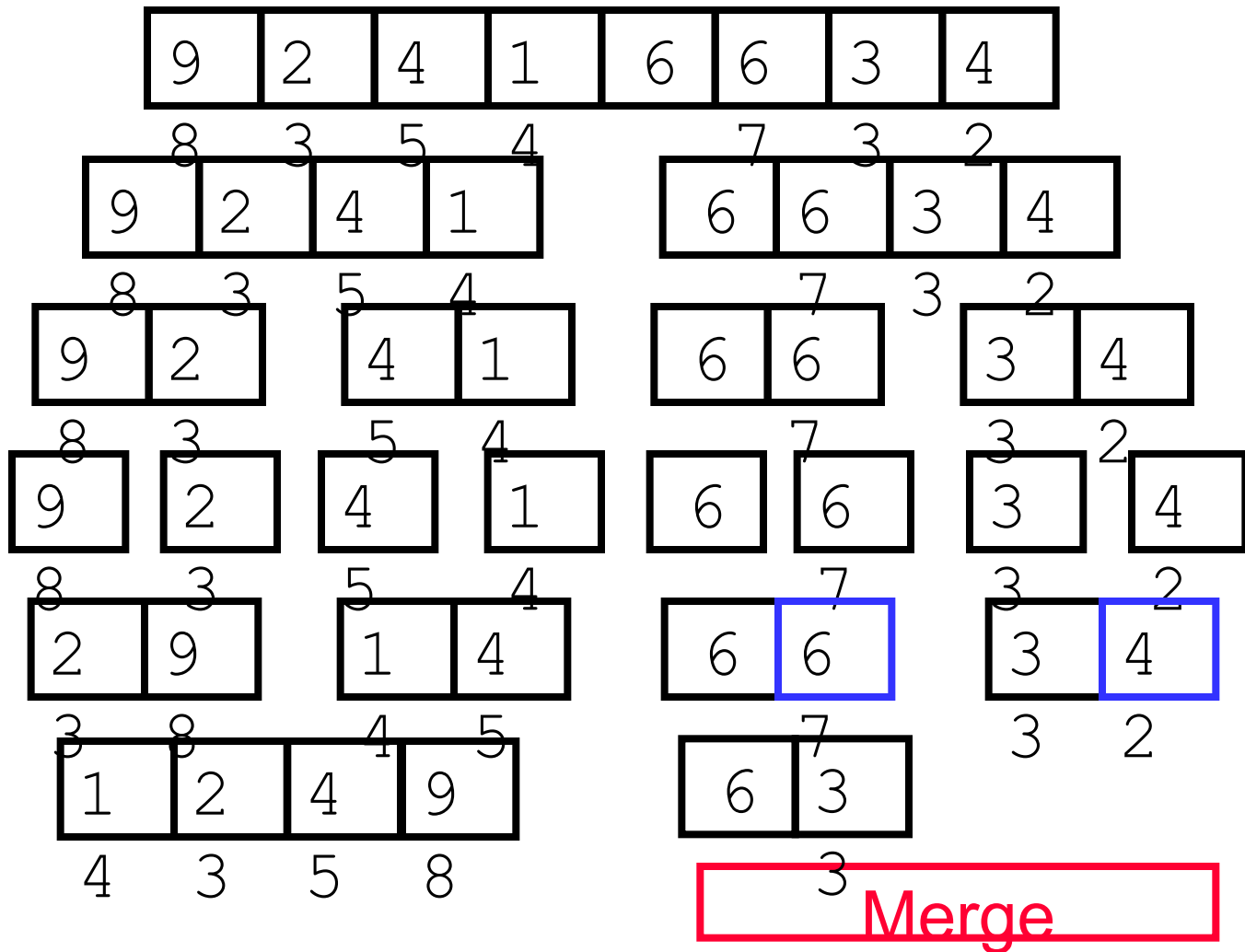


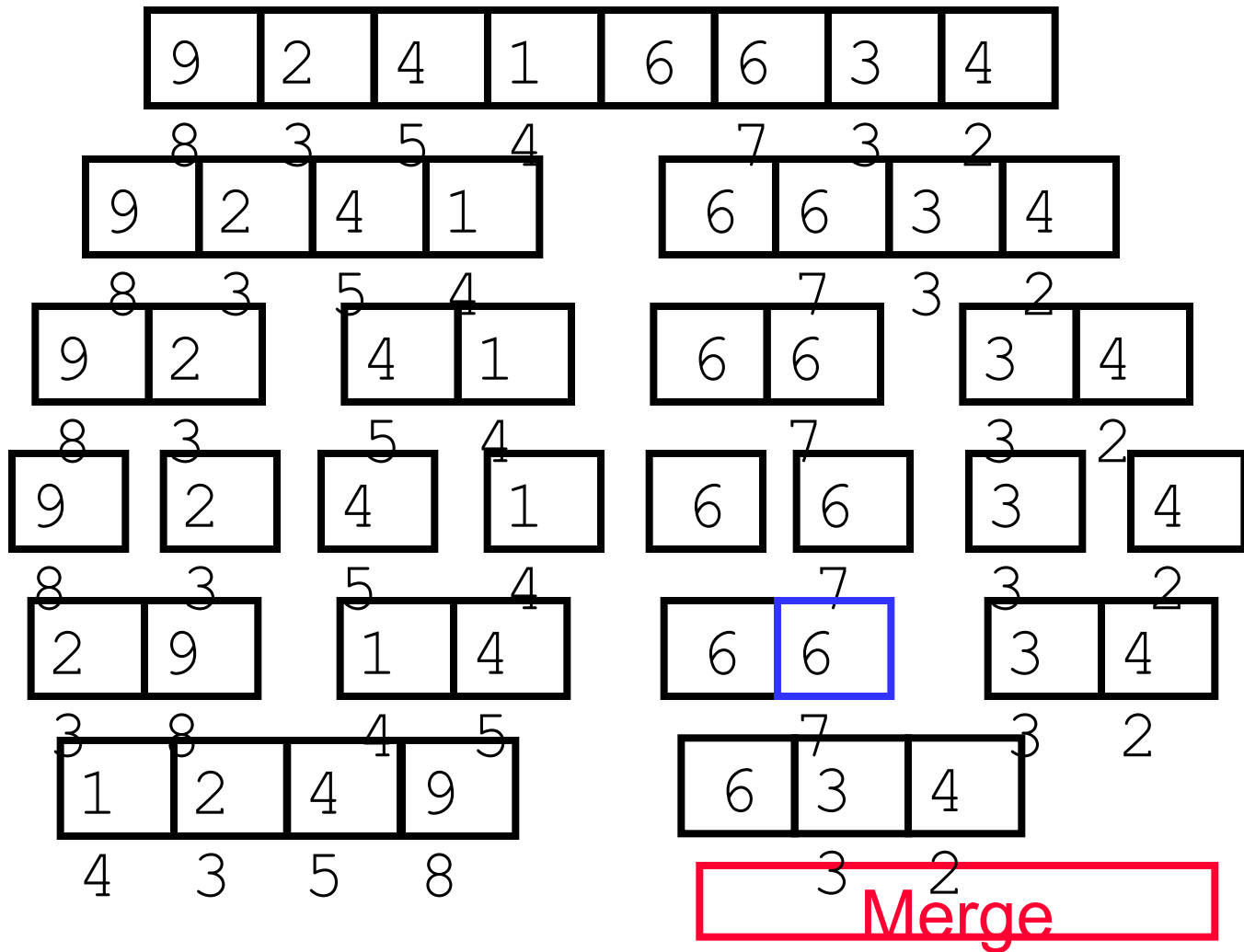


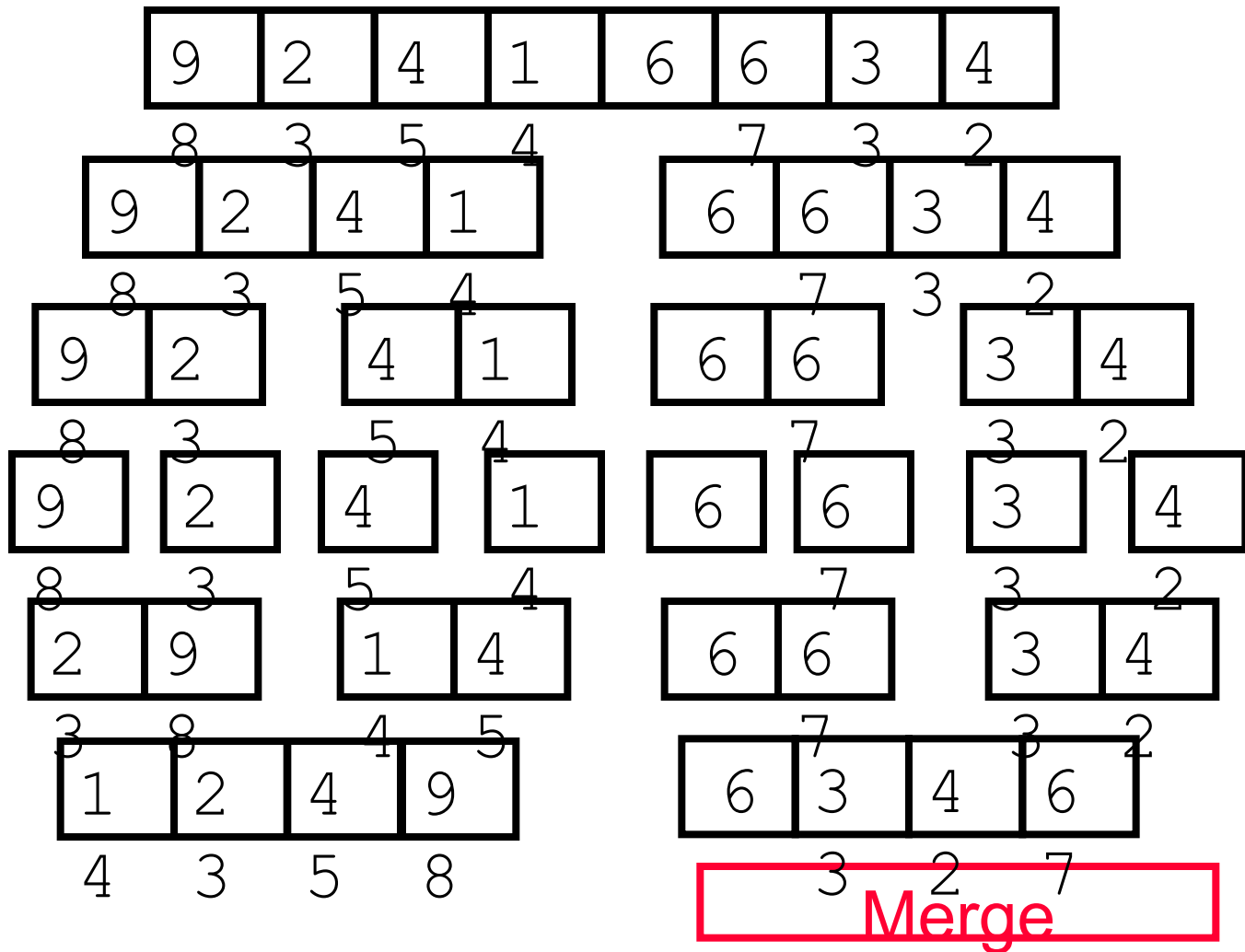


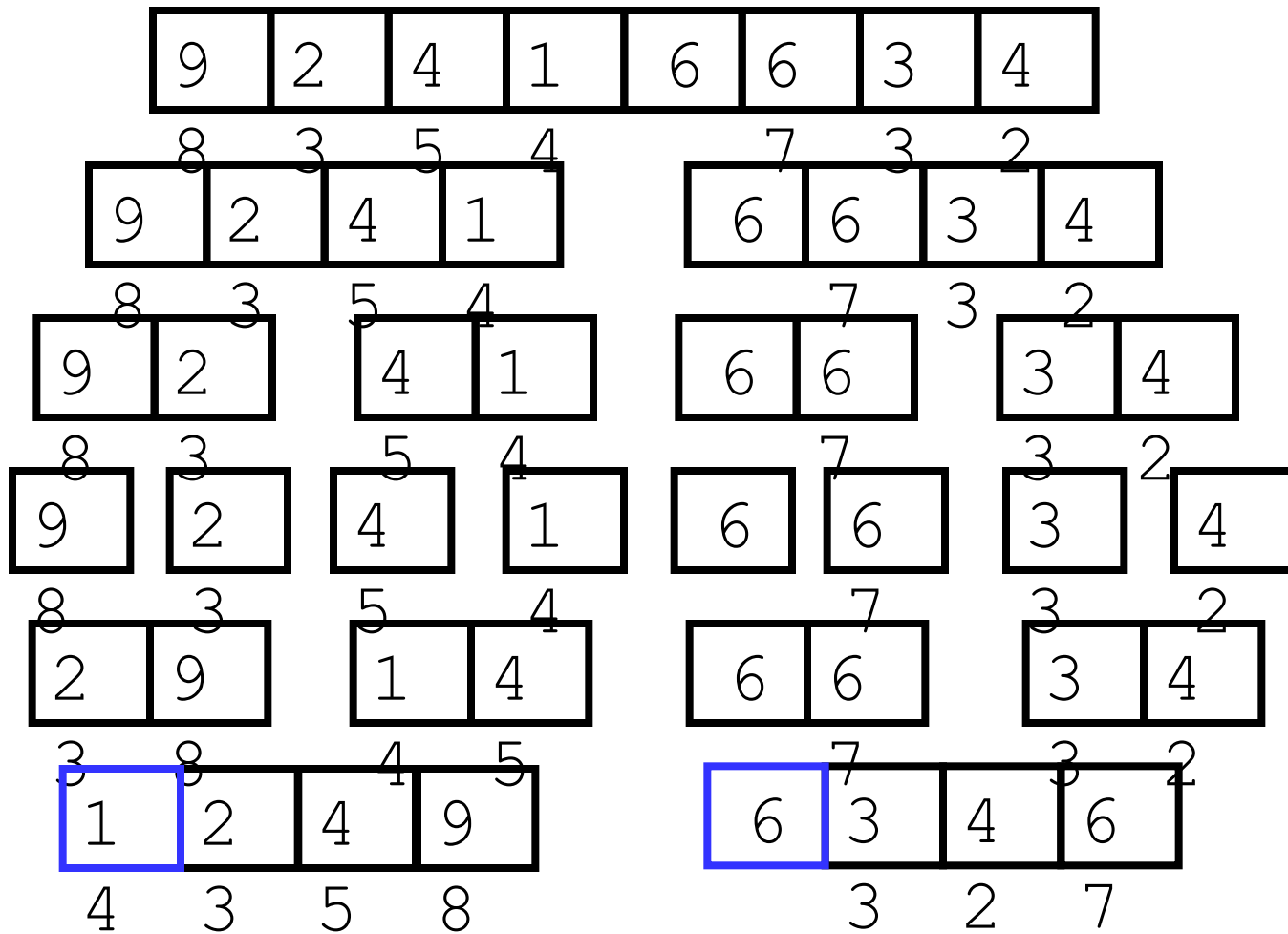
Merge



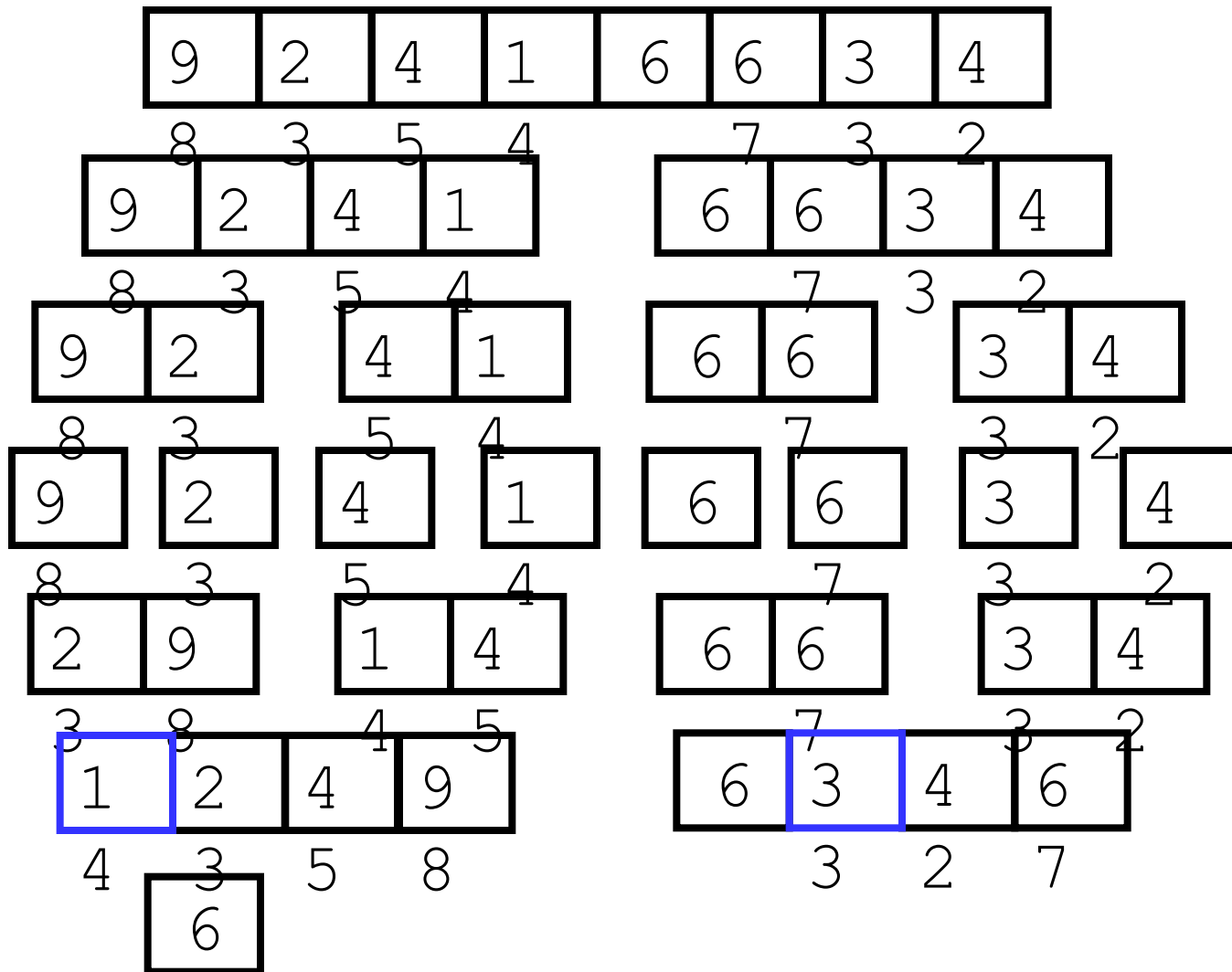




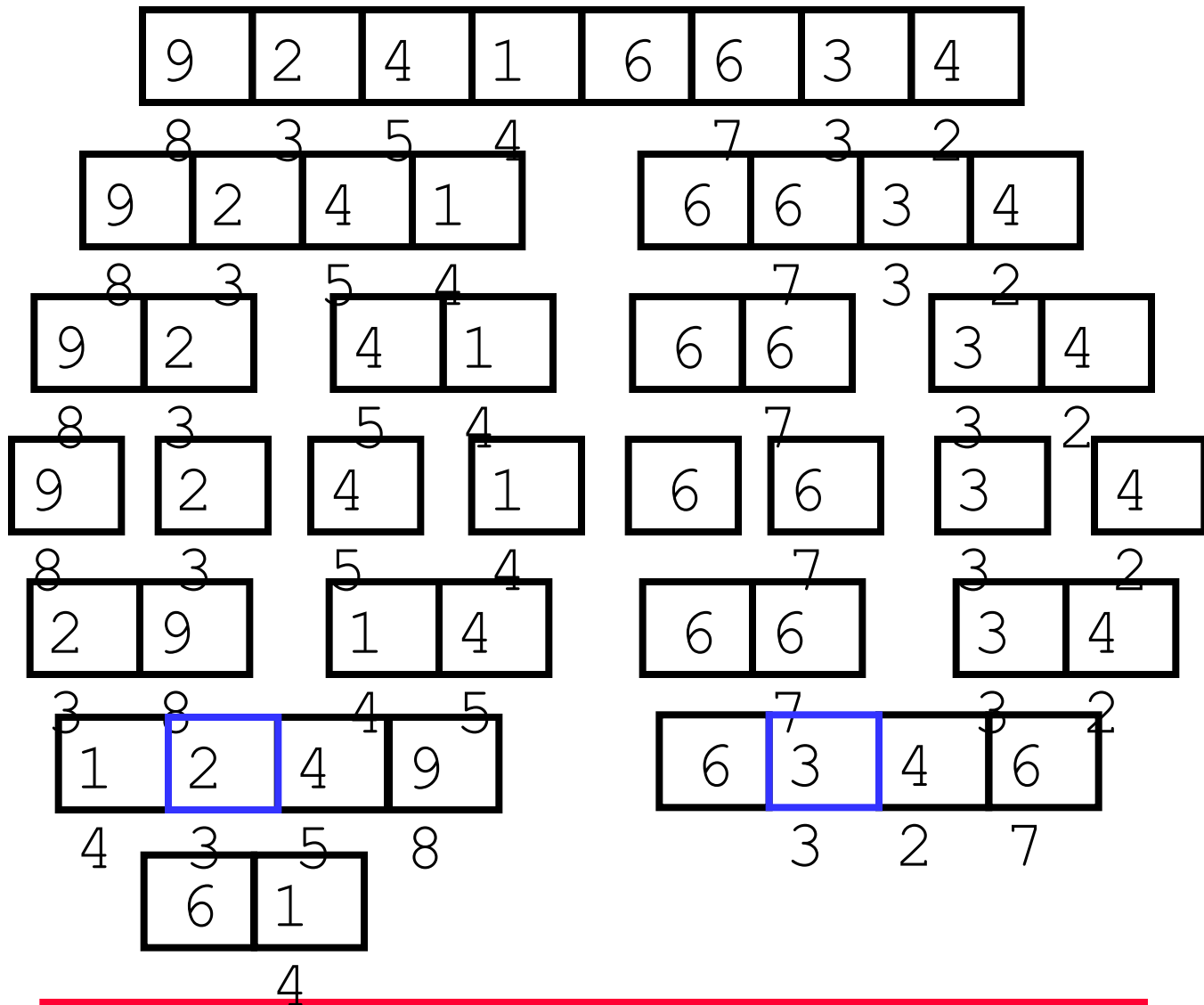


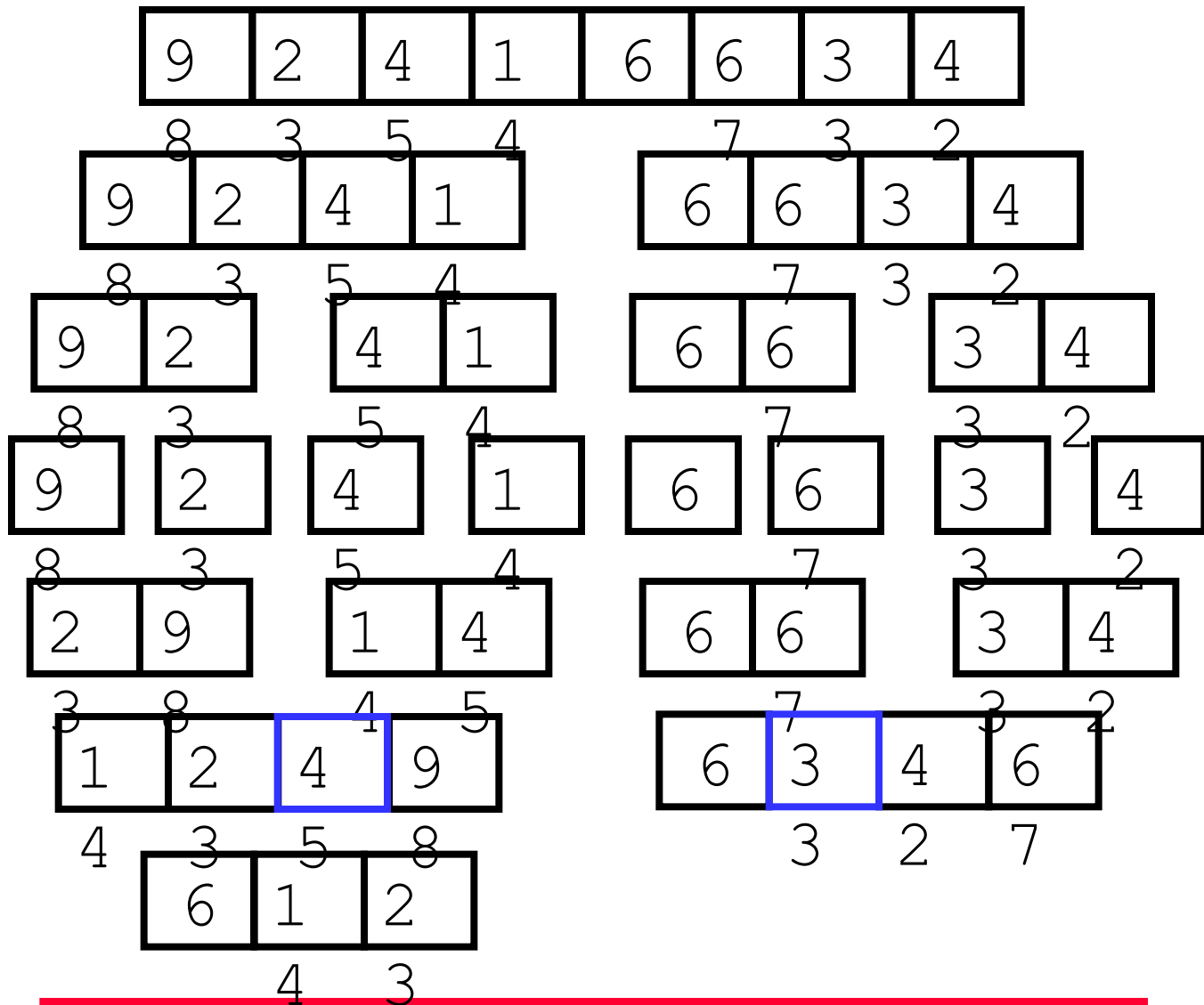


Merge

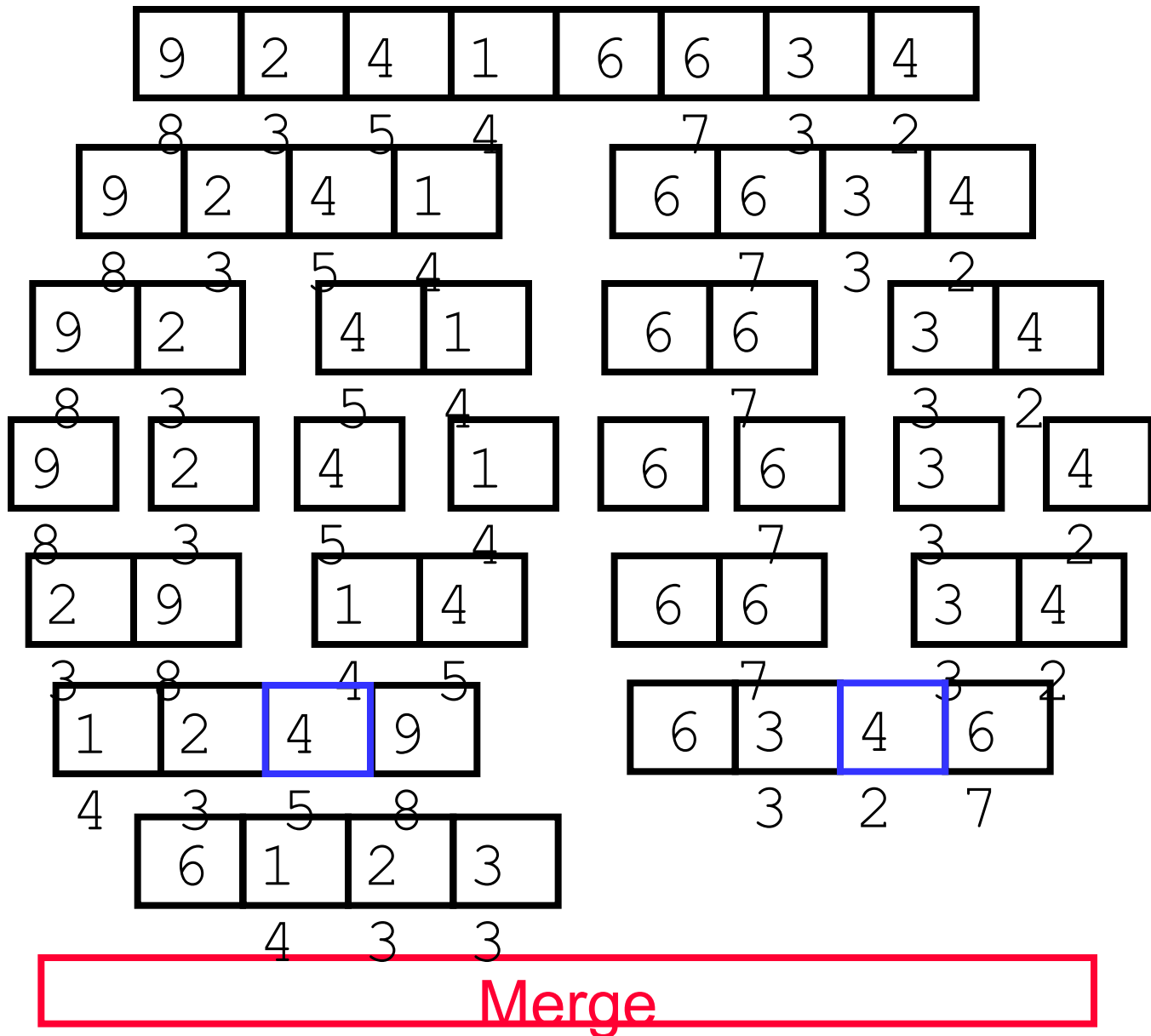


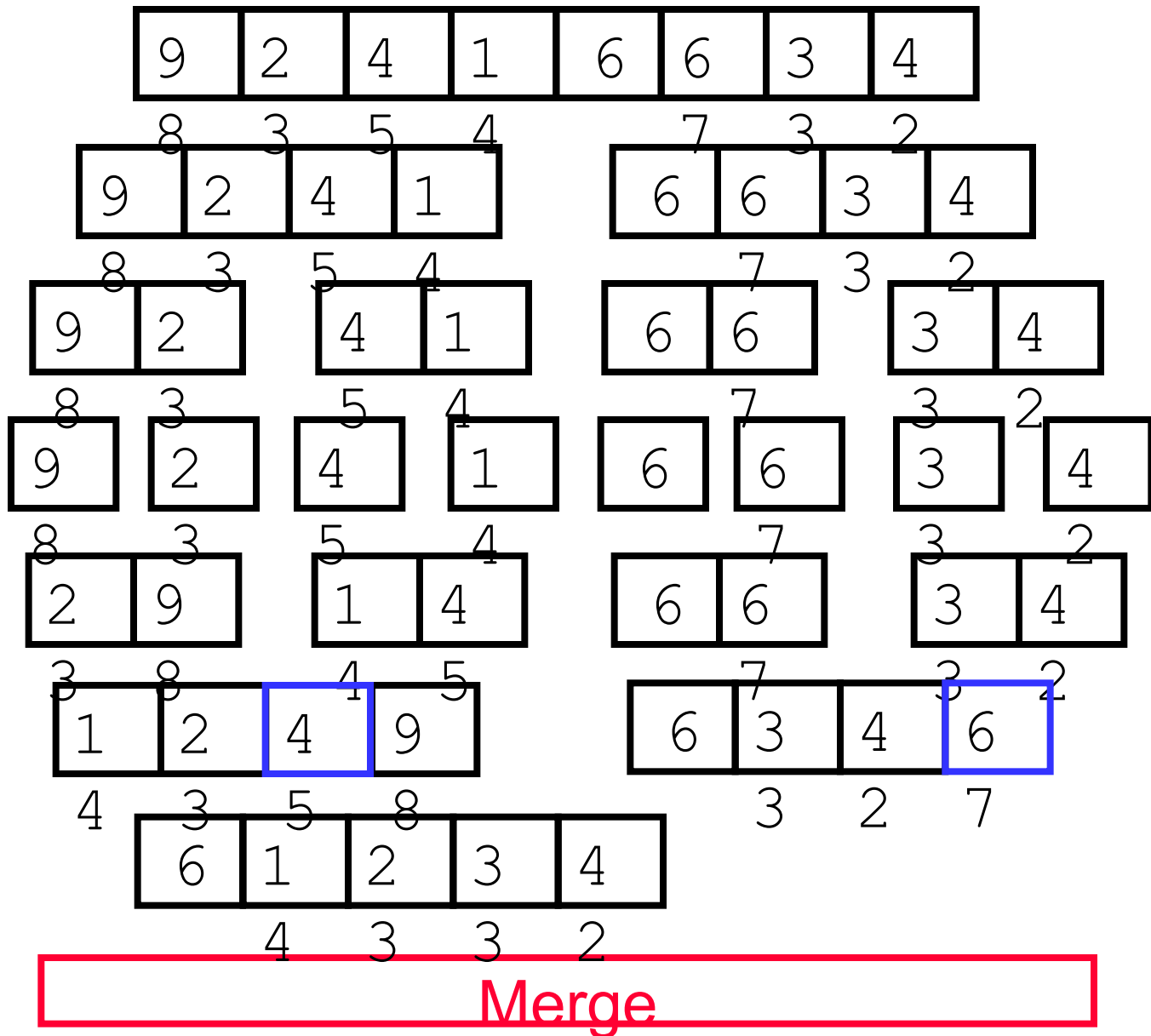
Merge

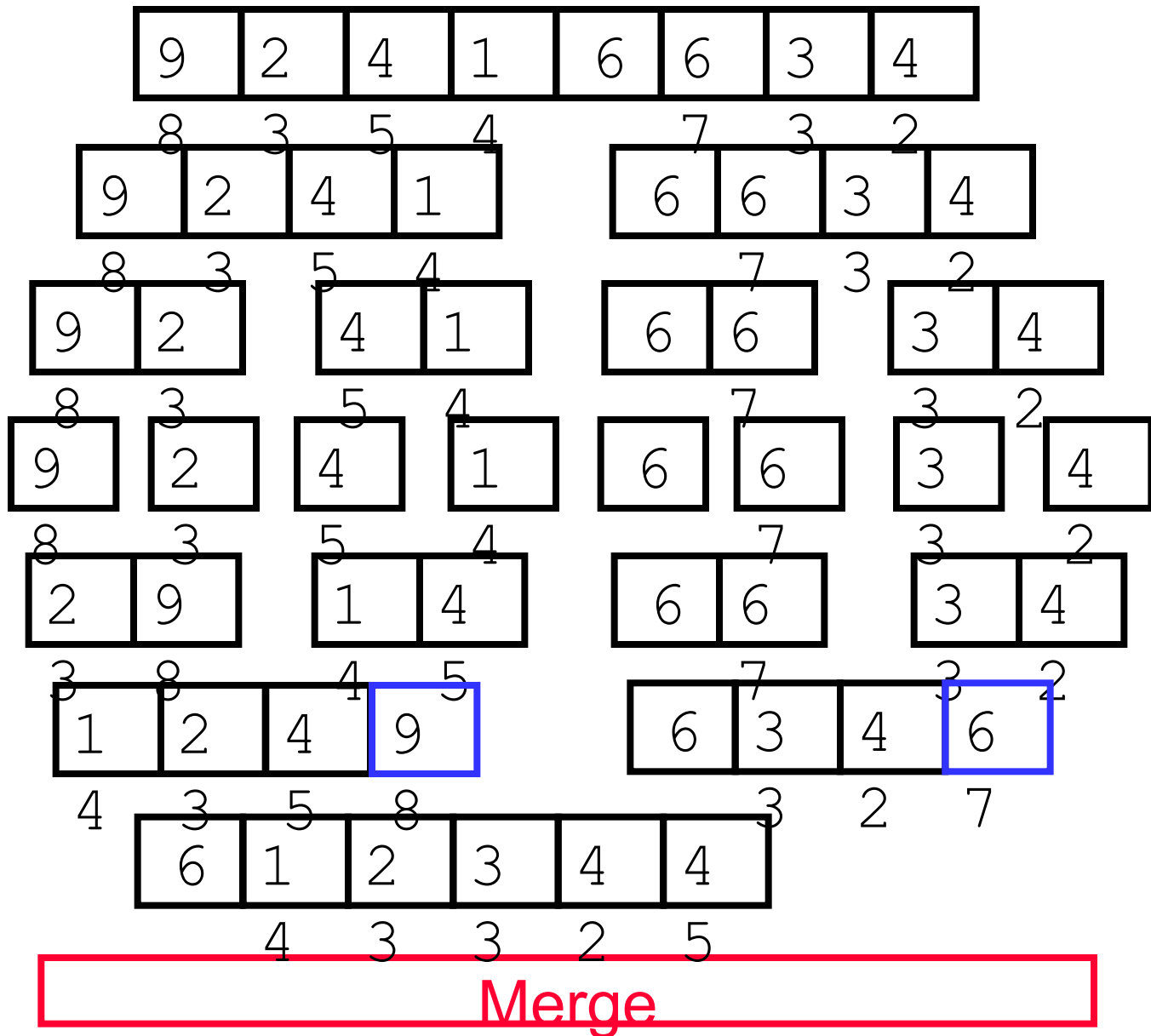


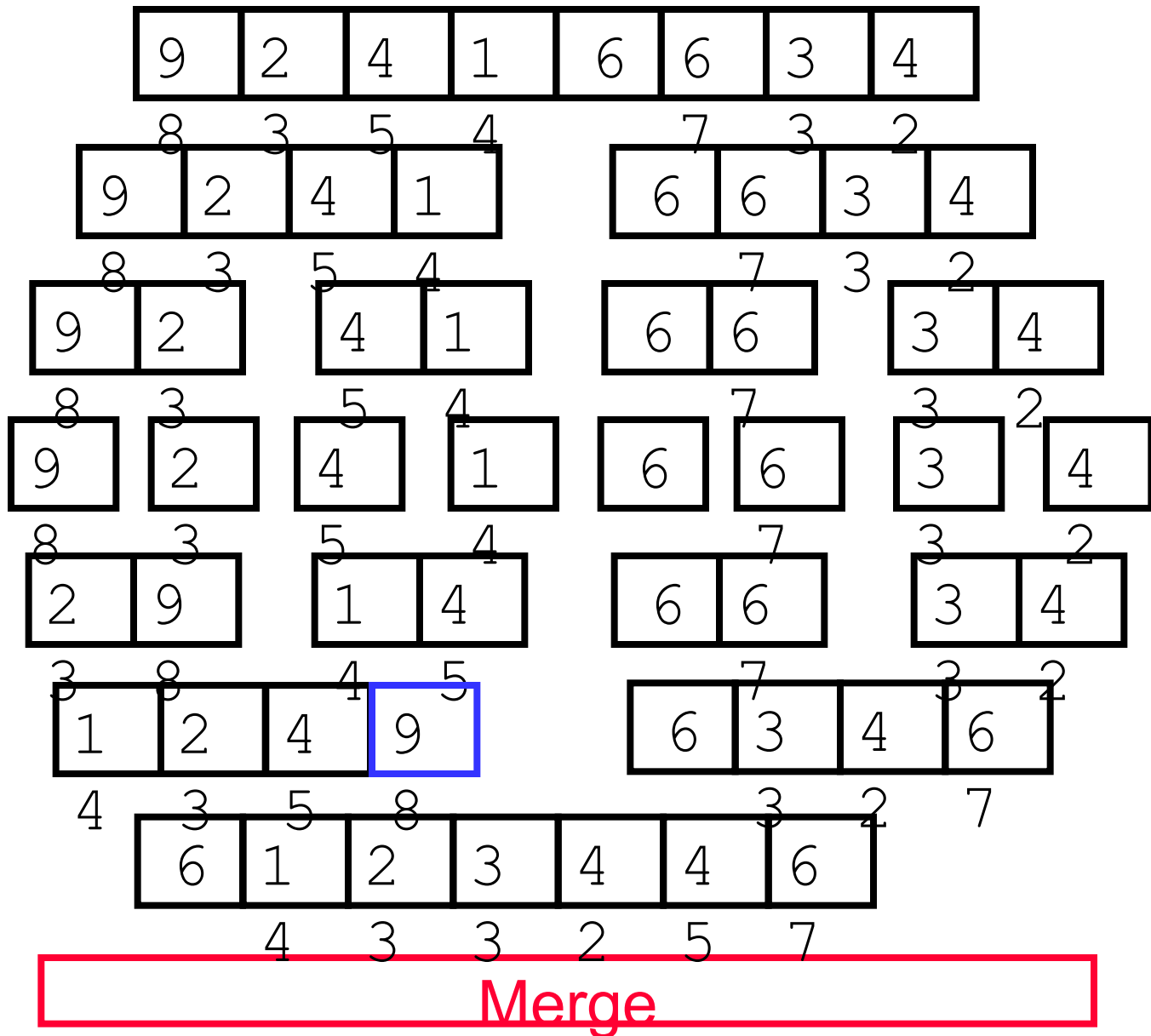


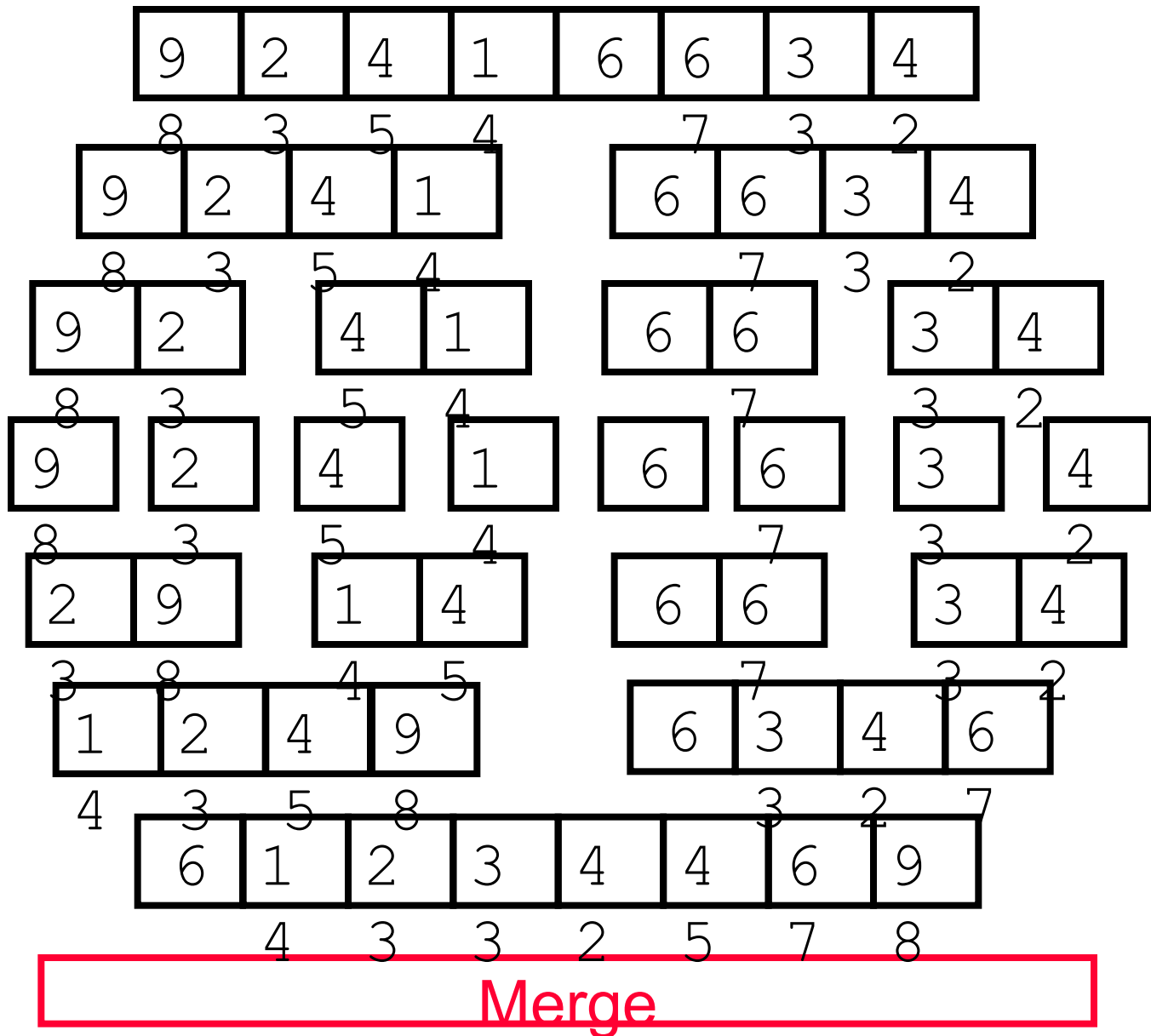
Merge











9	2	4	1	6	6	3	4
8	3	5	4		7	3	2



6	1	2	3	4	4	6	9
	4	3	3	2	5	7	8

Algorithm

Mergesort(Passed an array)

- if array size > 1

 - Divide array in half

 - Call Mergesort on first half.

 - Call Mergesort on second half.

 - Merge two halves.

Merge(Passed two arrays)

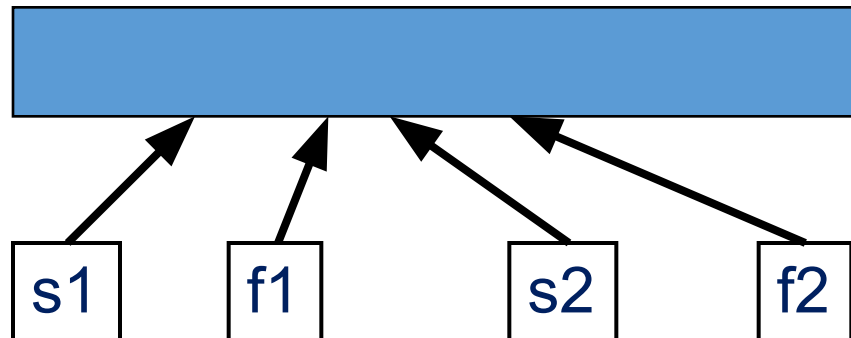
- Compare leading element in each array

- Select lower and place in new array.

 - (If one input array is empty then place remainder of other array in output array)

Merge Sort

- We don't really pass in two arrays!
- We pass in one array with indicator variables which tell us where one set of data starts and finishes and where the other set of data starts and finishes.



Merge Sort

```
void merge(int*,int*,int,int,int);  
void mergesort(int *a, int*b, int low, int high) {  
    int pivot;  
    if(low<high) {  
        pivot=(low+high)/2;  
        mergesort(a,b,low,pivot);  
        mergesort(a,b,pivot+1,high);  
        merge(a,b,low,pivot,high);  
    }  
}  
  
int main() {  
    int a[] = {12,10,43,23,78,45,56,98,41,90,24};  
    int num; num = sizeof(a)/sizeof(int);  
    int b[num];  
    mergesort(a,b,0,num-1);  
    for(int i=0; i<num; i++) cout<<a[i]<<" "; cout<<endl; }
```

```

void merge(int *a, int *b, int low, int pivot, int high) {
    int h,i,j,k; h=low; i=low; j=pivot+1;
    while((h<=pivot)&&(j<=high)) {
        if(a[h]<=a[j]) { b[i]=a[h]; h++; }
        else { b[i]=a[j]; j++; }
        i++; }
    if(h>pivot) {
        for(k=j; k<=high; k++) {
            b[i]=a[k]; i++; } }
    else {
        for(k=h; k<=pivot; k++) {
            b[i]=a[k]; i++; } }
    for(k=low; k<=high; k++) a[k]=b[k]; }

```

Quicksort / partition-exchange sort

Quick sort is a divide and conquer algorithm.

Quick sort first divides a large list into two smaller sub-lists: the low elements and the high elements. Quick sort can then recursively sort the sub-lists.

The steps are:

1. Pick an element, called a pivot, from the list.
2. Reorder the list so that all elements with values less than the pivot come before the pivot, while all elements with values greater than the pivot come after it (equal values can go either way). After this partitioning, the pivot is in its final position. This is called the partition operation.
3. Recursively apply the above steps to the sub-list of elements with smaller values and separately the sub-list of elements with greater values.

The base case of the recursion are lists of size zero or one, which never need to be sorted.

Quick Sort

5 4 9 7 1 8 3 2 6

4 1 3 2 5 9 7 8 6

1 3 2 4 5 7 8 6 9

1 3 2 4 5 6 7 8 9

1 2 3 4 5 6 7 8 9

1 2 3 4 5 6 7 8 9

 $\leq$  $<$ 

Quicksort

24 5 3 35 14 23 19 43 2

2 5 3 14 19 23 35 43 24

2 3 5 14 19 23 35 43 24

2 3 5 14 19 23 24 35 43

Quick Sort

```
void quickSort(int a[], int first, int last);
int pivot(int a[], int first, int last);
void swap(int& a, int& b);
Void main()
{
    int test[] = { 7, -13, 1, 3, 10, 5, 2, 4 };
    int N = sizeof(test)/sizeof(int);
    quickSort(test, 0, N-1);
}
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}
```

```

void quickSort( int a[], int first, int last) {
    int pivotElement;
    if(first < last) {
        pivotElement = pivot(a, first, last);
        quickSort(a, first, pivotElement-1);
        quickSort(a, pivotElement+1, last); }
    }

int pivot(int a[], int first, int last){
    int p = first;
    int pivotElement = a[first];
    for(int i = first+1 ; i <= last ; i++){
        if(a[i] <= pivotElement){
            p++;
            swap(a[i], a[p]);
        }
    }
    swap(a[p], a[first]);
    return p;
}

```

```
int pivot(int a[], int first, int last){  
    int p = first;  
    int pivotElement = a[first];  
    for(int i = first+1 ; i <= last ; i++){  
        if(a[i] <= pivotElement){  
            p++;  
            swap(a[i], a[p]);  
        }  
    }  
    swap(a[p], a[first]);  
    return p;  
}
```


Heap Sort

Heap Sort

→ Heap sort

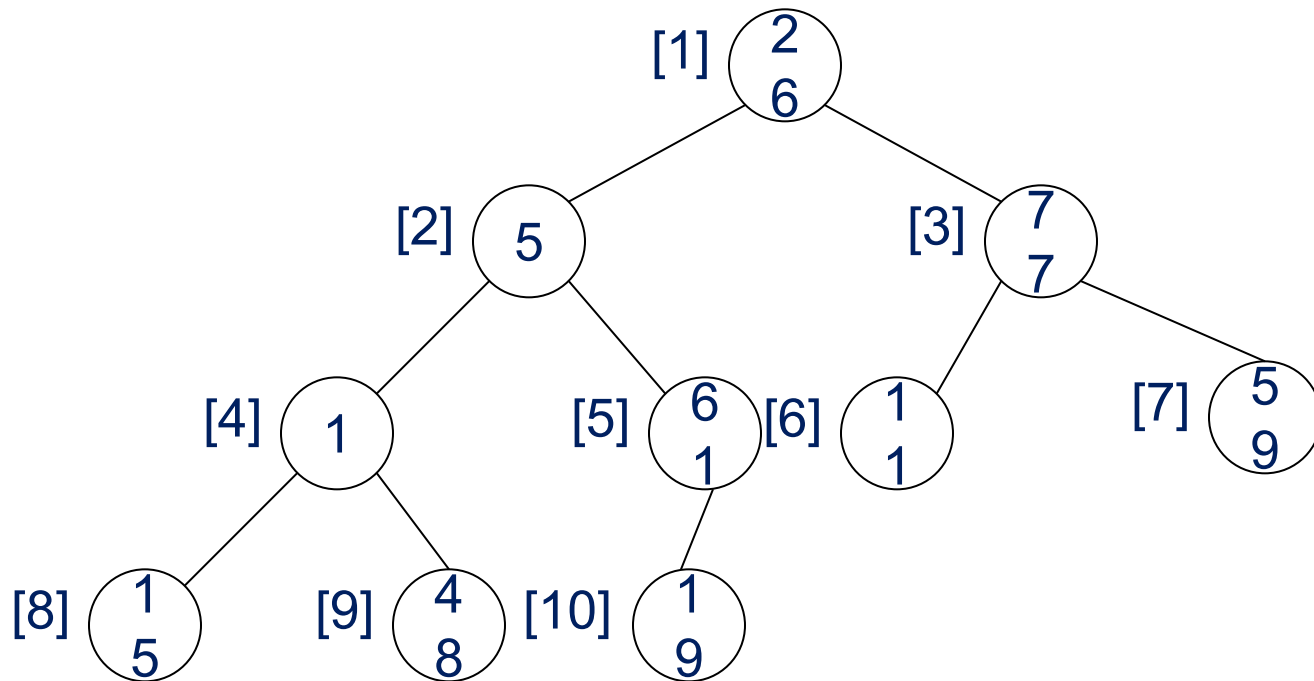
⇒ The heap sort is an improved version of the selection sort in which the target element (the root) is selected and exchanged with the last element in the unsorted list.

Heap Sort

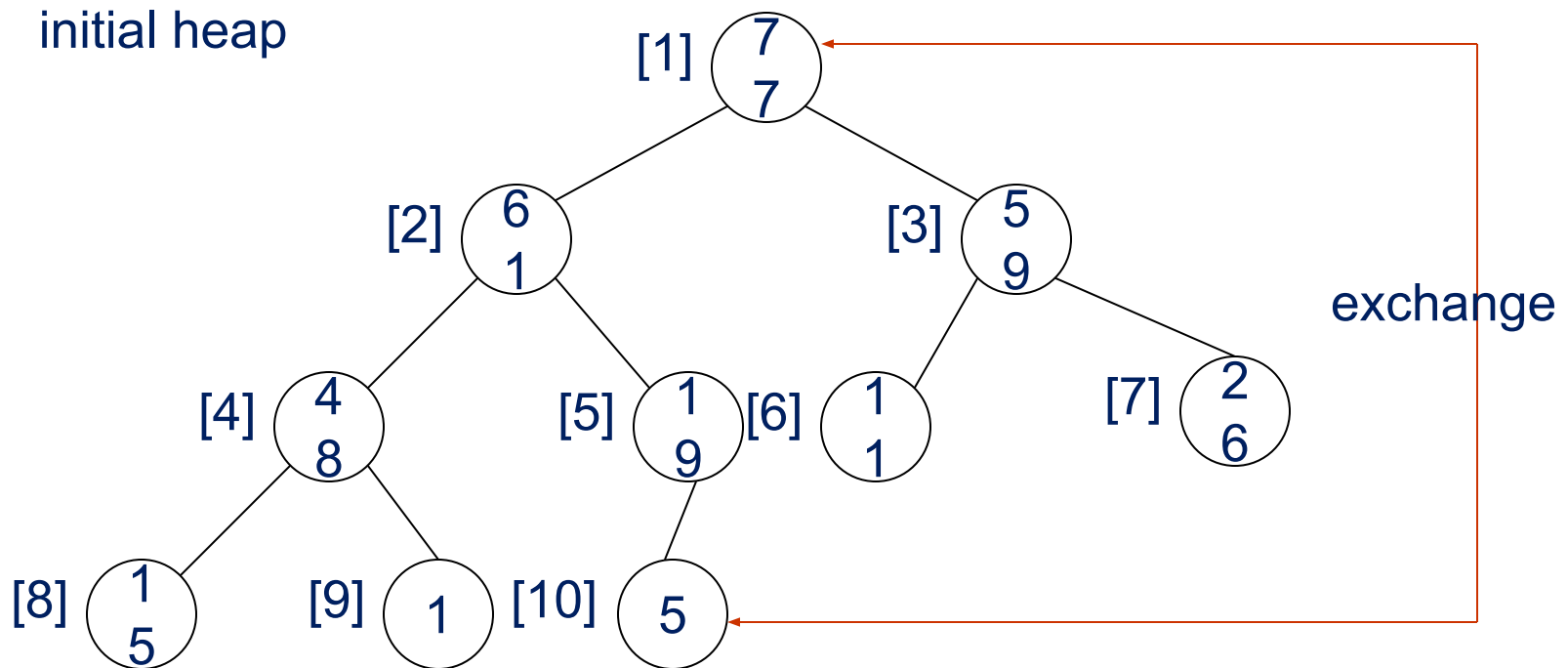
Array interpreted as a binary tree

1	2	3	4	5	6	7	8	9	10
26	5	77	1	61	11	59	15	48	19

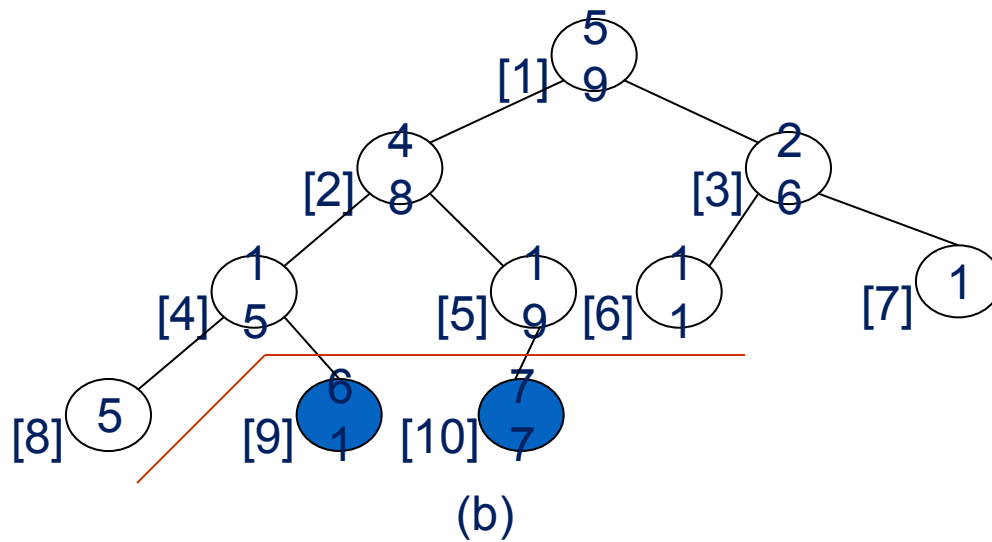
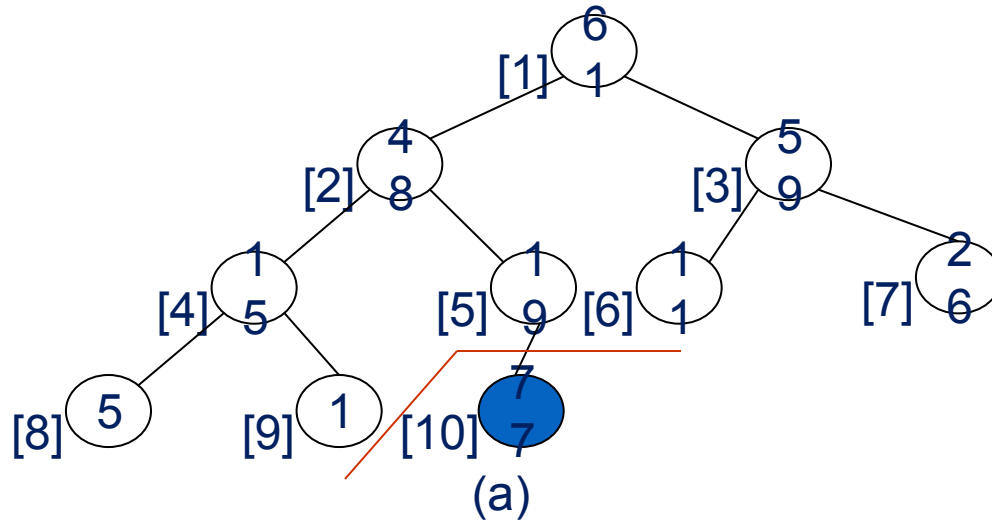
input file



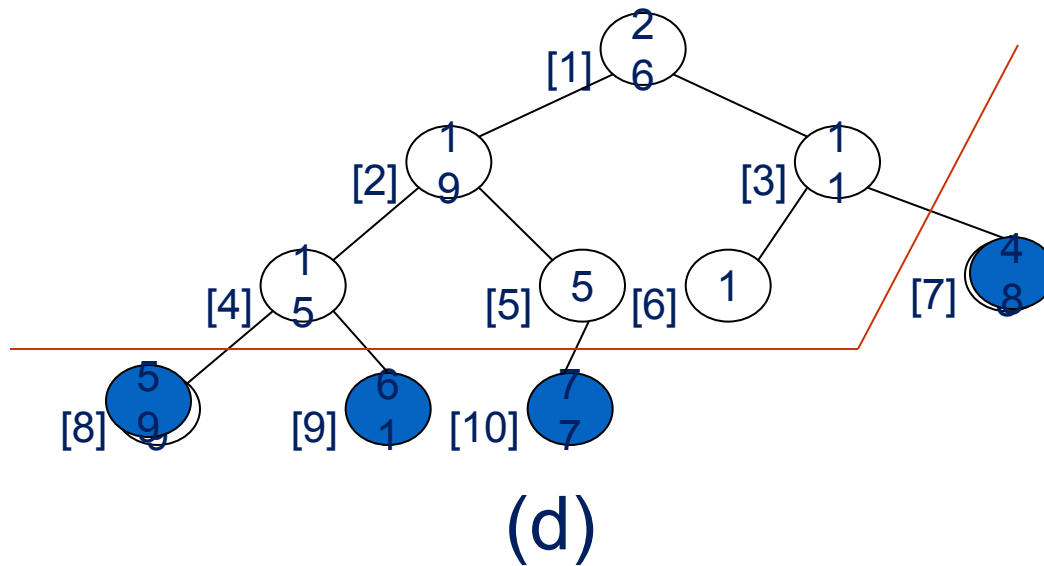
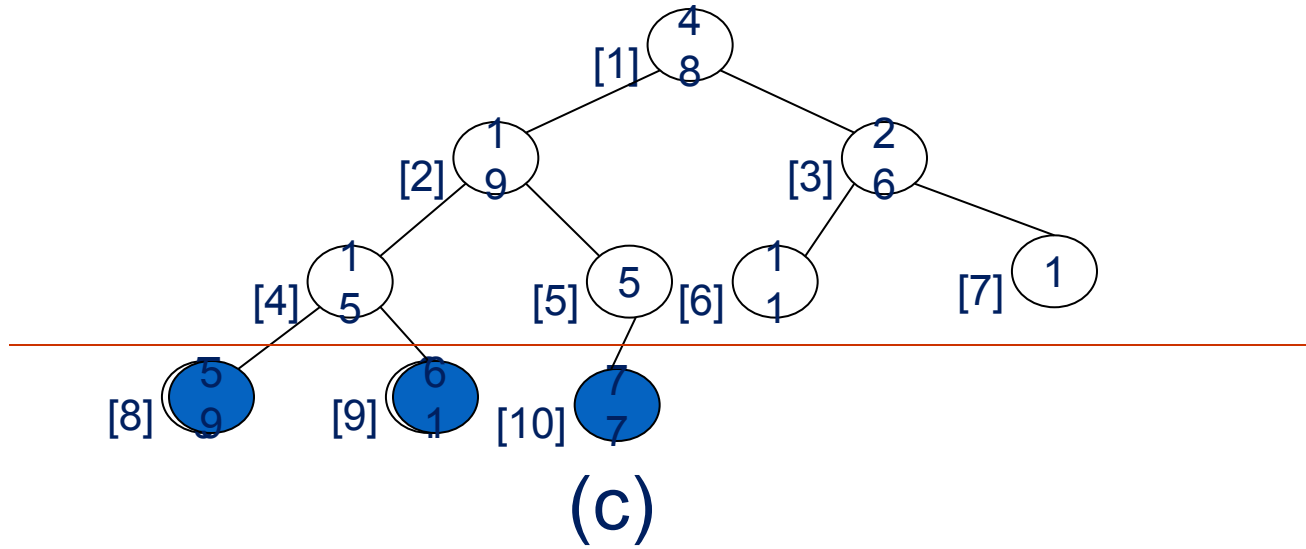
Max heap following first **for** loop of *heapsort*



Heap sort example

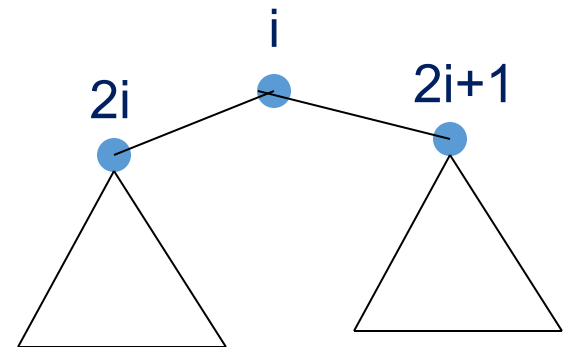


Heap sort example



Heap Sort

```
void adjust(element list[], int root, int n)
{
    int child, rootkey;  element temp;
    temp=list[root];    rootkey=list[root].key;
    child=2*root;
    while (child <= n) {
        if ((child < n) &&
            (list[child].key < list[child+1].key))
            child++;
        if (rootkey > list[child].key) break;
        else {
            list[child/2] = list[child];
            child *= 2;
        }
    }
    list[child/2] = temp;
}
```



Heap Sort

```
void heapsort(element list[], int n)
{
    int i, j;
    element temp;
    for (i=n/2; i>0; i--) adjust(list, i, n);
    for (i=n-1; i>0; i--) {
        SWAP(list[1], list[i+1], temp);
        adjust(list, 1, i);
    }
}
```

ascending order (max heap)

bottom-up

n-1 cycles

top-down

Radix Sort

Radix Sort

Radix Sort Learn Easily

Do following for each digit i where i varies from least significant digit to most significant digit.

Sort input array using Counting Sort according to i th digit.

Eq 15 10 17 25 49 16 128 76 540

$$\begin{array}{c} 128 \\ \swarrow \downarrow \searrow 13 \\ 100s \quad 10s \end{array}$$

$19 \rightarrow 128, 140, 160$
 $13 \rightarrow 140, 160, 128$
 $103 \rightarrow 128, 140, 160$
 $1013 \rightarrow 128, 140, 160$

1's digit: 15 19 17 25 49 16 128 76 540

10, 540, 15, 25, 16, 76, 17, 128, 49

10's digit

10, 540, 15, 25, 16, 76, 17, 128, 49

10, 15, 16, 17, 25, 128, 540, 49, 76

100's digit

010, 015, 016, 017, 025, 128, 540, 049, 076

010, 015, 016, 017, 025, 049, 076, 128, 540

Radix Sort

Radix Sort

Lexicographic Sort

(2016, 10) (2016, 8) (2015, 8) (2015, 9)

Sort in asc. order of year,
within each year, sort in
asc. order of marks.

O/p: 2015 8 Broad: year
 2015 9 Narrow: marks.
 2016 8
 2016 10

Sorting wrt marks

(2016, 8) (2015, 8) (2015, 9) (2016, 10)

Sorting wrt year

(2015, 8) (2015, 9) (2016, 8) (2016, 10)

RadixSort (S, N)
 $\uparrow$ seq $\uparrow$ Range

```
for (i = d  $\rightarrow$  i = 1)
{
    S = BucketSort(S, N, i)
}
return S;
```

Running Time

Bucket Sort is $O(N + n)$
Radix Sort is $O(d(N + n))$

<https://www.youtube.com/watch?v=XcF1l2pq9ZI>

Radix Sort



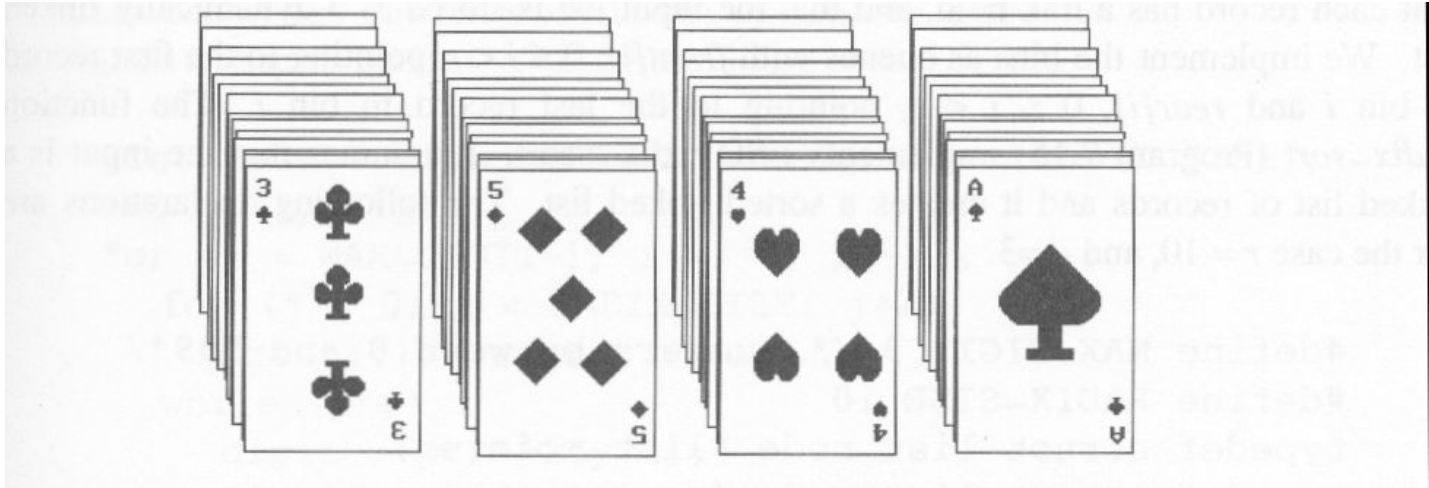
$R_0, R_1, \dots, R_{n-1}$ are said to be sorted w.r.t. $K_0, K_1, \dots, K_{r-1}$ iff

$$(k_i^0, k_i^1, \dots, k_i^{r-1}) \leq (k_{i+1}^0, k_{i+1}^1, \dots, k_{i+1}^{r-1}) \quad 0 \leq i < n-1$$

Most significant digit first: sort on K^0 , then K^1 , ...

Least significant digit first: sort on K^{r-1} , then K^{r-2} , ...

Arrangement of cards after first pass of an MSD sort

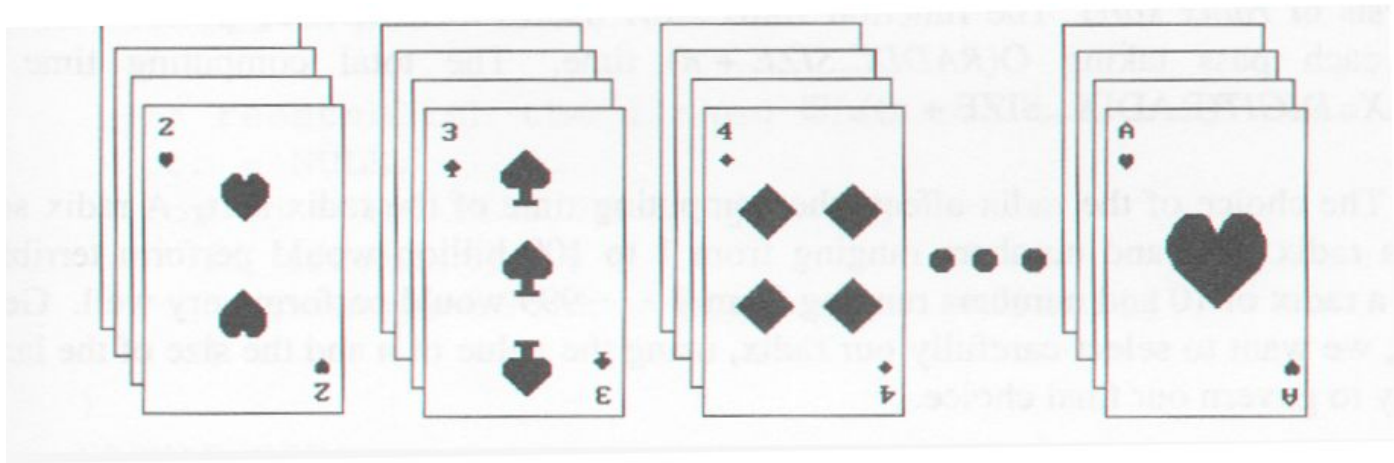


Suits: ♣ < ♦ < ♥ < ♠

Face values: 2 < 3 < 4 < ... < J < Q < K < A

- (1) MSD sort first, e.g., bin sort, four bins ♣ ♦ ♥ ♠
LSD sort second, e.g., insertion sort
- (2) LSD sort first, e.g., bin sort, 13 bins
2, 3, 4, ..., 10, J, Q, K, A
MSD sort, e.g., bin sort four bins ♣ ♦ ♥ ♠

Arrangement of cards after first pass of LSD sort



Radix Sort

$$0 \leq K \leq 999$$

$(K^0,$	$K^1,$	$K^2)$
MSD		LSD
0-9	0-9	0-9

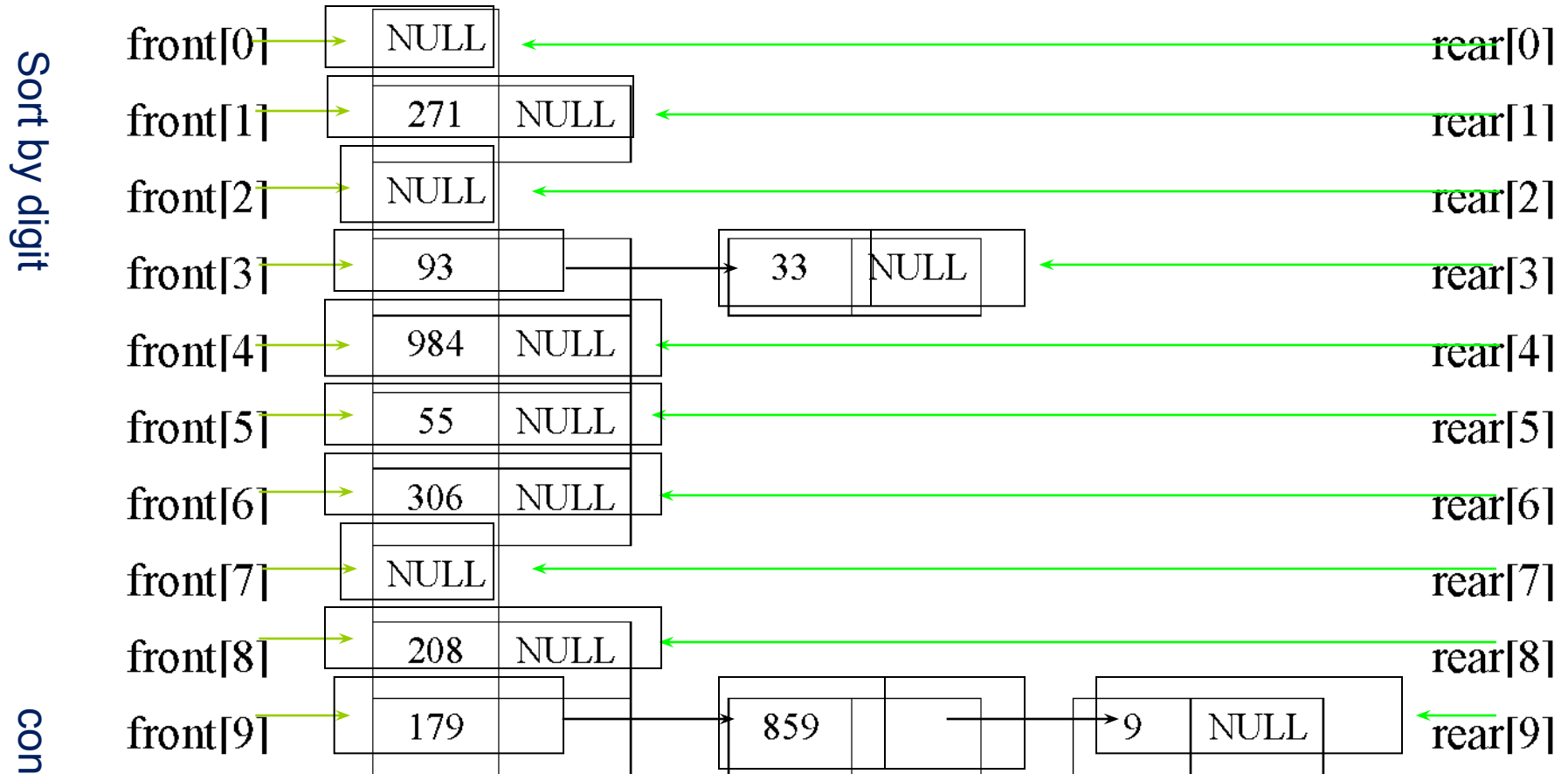
radix 10 sort

radix 2 sort

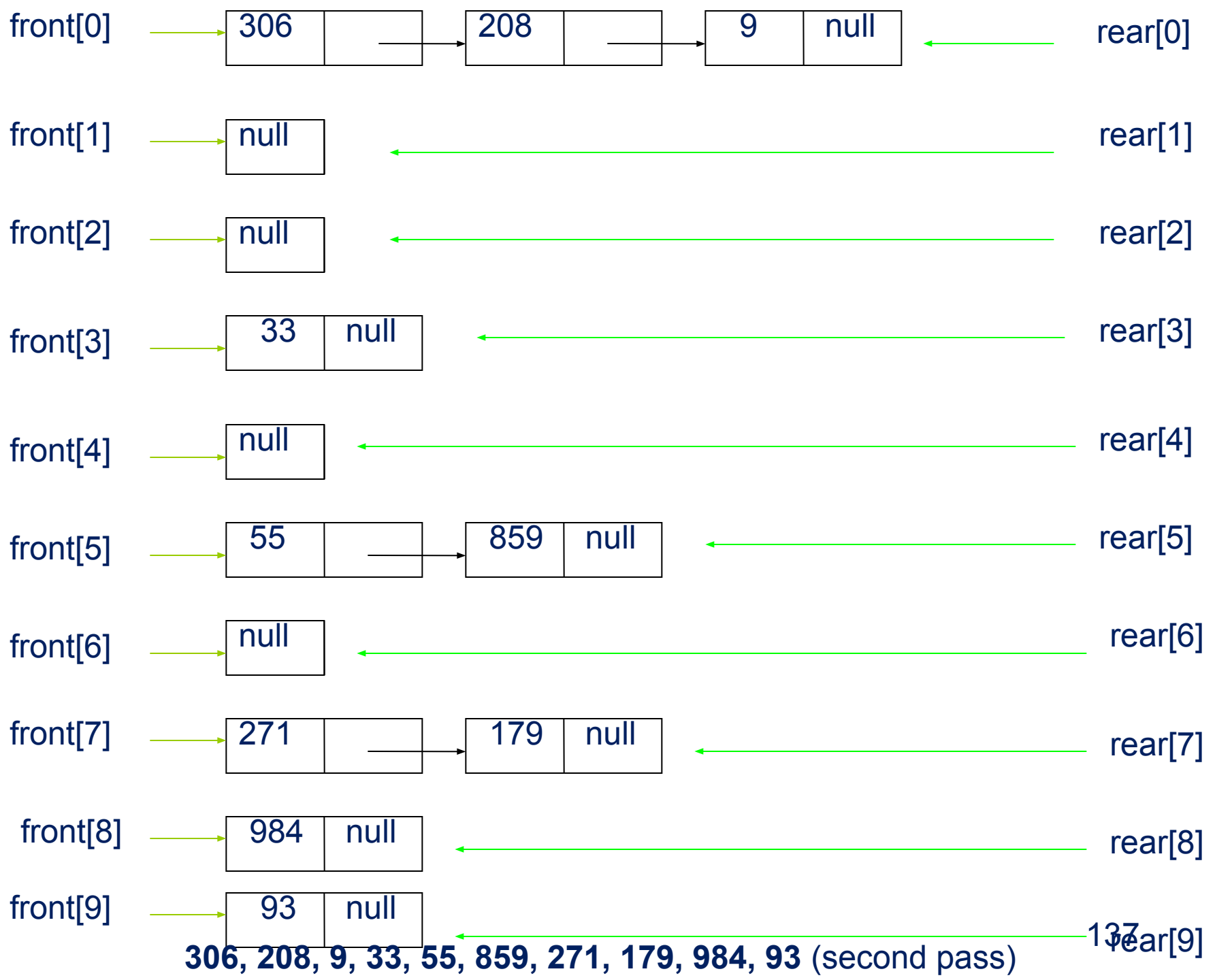
Example for LSD Radix Sort

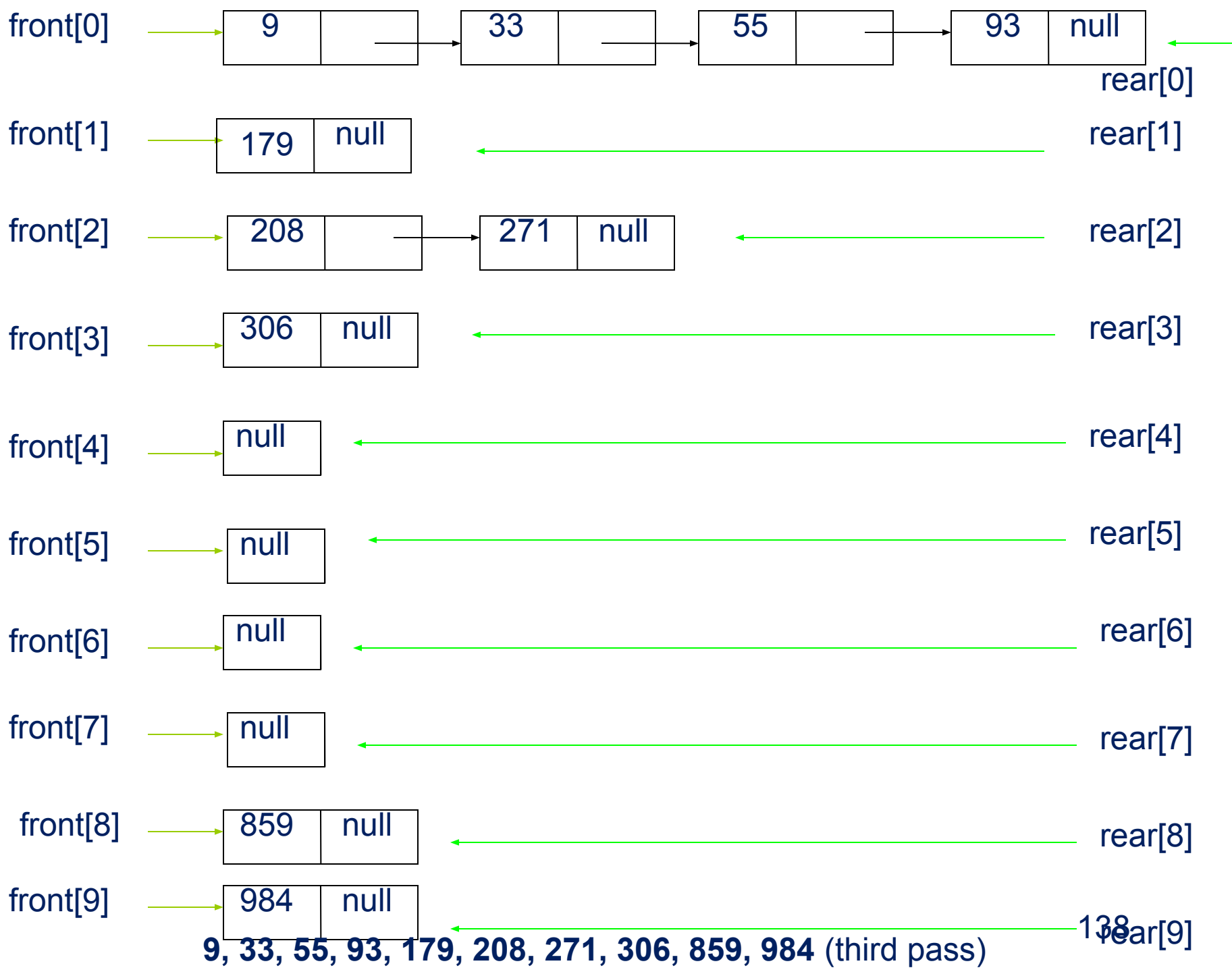
d (digit) = 3, r (radix) = 10 ascending order

179, 208, 306, 93, 859, 984, 55, 9, 271, 33



271, 93, 33, 984, 55, 306, 208, 179, 859, 9 After the first pass





Data Structures for LSD Radix Sort

An LSD radix r sort,

$R_0, R_1, \dots, R_{n-1}$ have the keys that are d -tuples
($x_0, x_1, \dots, x_{d-1}$)

```
#define MAX_DIGIT 3
```

```
#define RADIX_SIZE 10
```

```
typedef struct list_node *list_pointer;
```

```
typedef struct list_node {
```

```
    int key[MAX_DIGIT];
```

```
    list_pointer link;
```

```
}
```

LSD Radix Sort

```
list_pointer radix_sort(list_pointer ptr)
{
    list_pointer front[RADIX_SIZE],
                rear[RADIX_SIZE];
    int i, j, digit;
    for (i=MAX_DIGIT-1; i>=0; i--) {
        for (j=0; j<RADIX_SIZE; j++)
            front[j]=rear[j]=NULL;
        while (ptr) {
            digit=ptr->key[i];
            if (!front[digit]) front[digit]=ptr;
            else rear[digit]->link=ptr;
```

Initialize bins to be empty queue.

Put records into queues.

		rear[digit]=ptr;	
		ptr=ptr->link;	Get next
	O(n	}	record.
	)	/* reestablish the linked list for the	
		next pass */	
O(d(n+r		ptr= NULL;	
))		for (j=RADIX_SIZE-1; j>=0; j++)	
		if (front[j]) {	
	O(r	rear[j]->link=ptr;	
	)	ptr=front[j];	
		}	
		}	
		return ptr;	
		}	

Bucket sort

Bucket sort

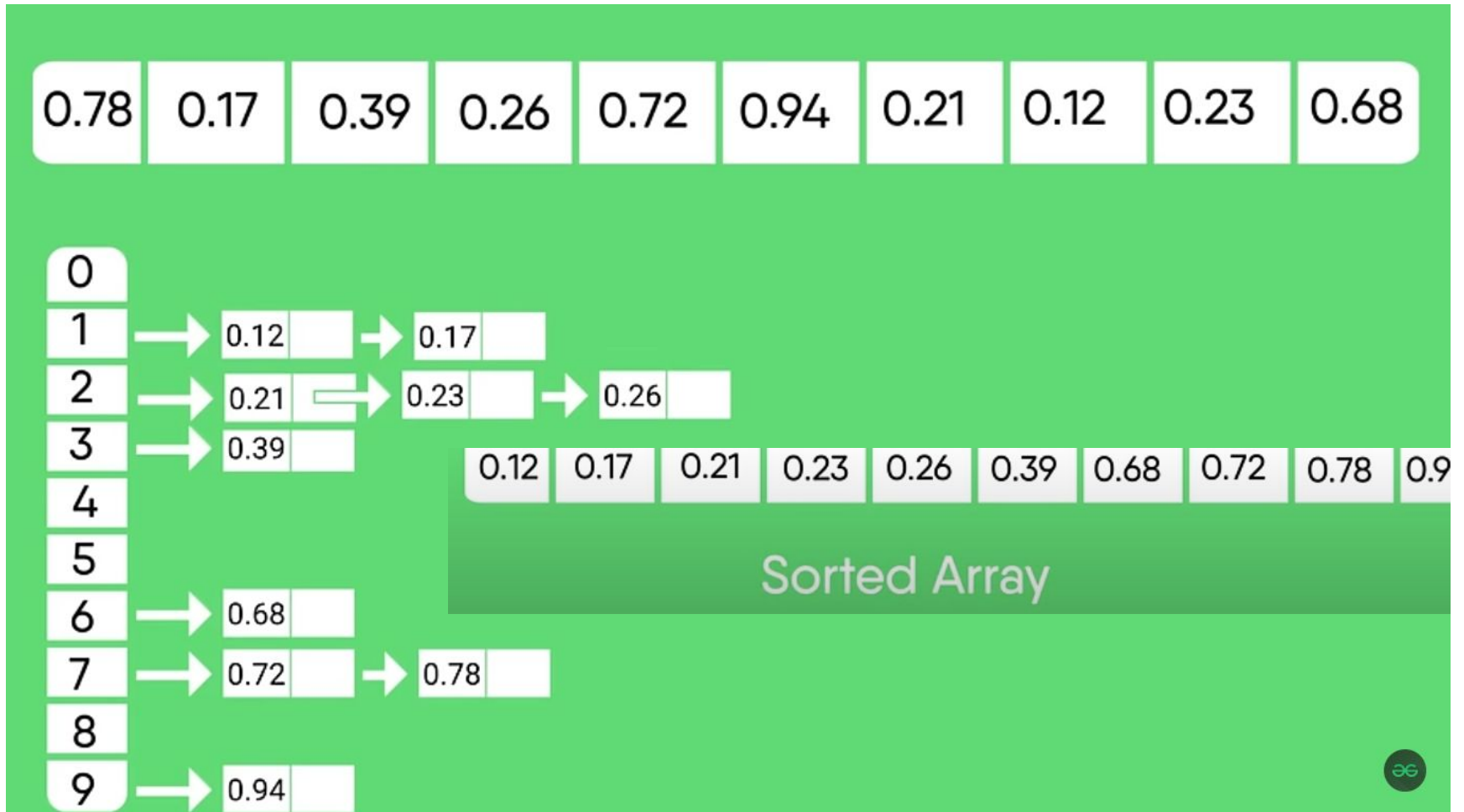
<https://www.youtube.com/watch?v=VuXbEb5ywrU>

In this sorting algorithm buckets are created to put elements into them.

Then we apply some sorting algorithm (insertion sort) to sort the elements in each bucket.

Finally take out and join them to get sorted array.

Bucket sort

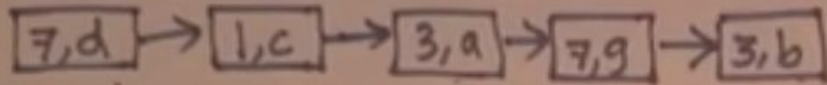


Bucket sort

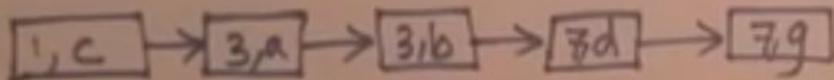
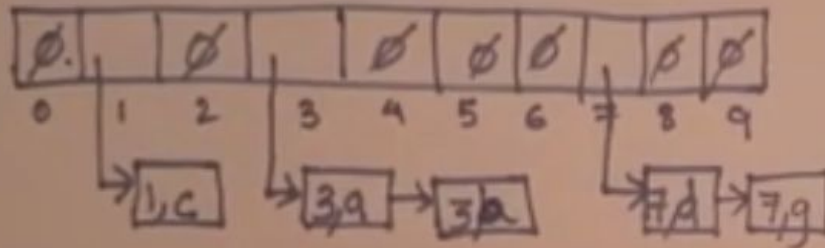
- **bucket sort:** A sort on integers in a known range where elements are "sorted" by tallying counts of occurrences of each unique value.
- For an input array a of integers from $[0 \dots M)$, create an array of counters C of size M .
- For each element in a , if its value is k , increment $C[k]$.
- Use the counters in C to place a 's contents back in sorted order.
- This sort can be even faster than quick sort!
 - It takes advantage of additional information about the data, namely, that it is a collection of integers in a specific range.
 - With some kinds of data, can be even more clever:
 - if all values in a are unique, C can be `boolean[]` instead of `int[]`
 - can use a *BitSet* to save memory (C will be smaller)

Bucket sort

BUCKET SORT



We know that range $0 \rightarrow 9$



Stable Sort?

If there is more than 1 tuple of the same key, the initial order is retained

```

BucketSort (S, N, B)
    ↑      ↑      ↑
    seq  range bucket array

{
    while (S is not empty)
    {
        object(key, e) = S.removefirst();
        B[key] = addlast(object(key, e));
    }
    PHASE 1

    for (i = 0 → i = N-1)
    {
        while (B[i] is not empty)
        {
            object(key, e) = B[i].removefirst();
            S.addlast(object);
        }
        PHASE 2
    }
    return S;
}

Phase 1:  $O(n)$  — no. of tuples
Phase 2:  $O(N)$  — Range
Bucket Sort is  $O(n + N)$ 
    
```

Bucket sort example

- input array a :

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
value	7	4	1	2	5	4	10	9	1	4	7	8	9	8	9	4	4

- create array of tallies:

index	0	1	2	3	4	5	6	7	8	9	10
value	0	2	1	0	5	1	0	2	2	3	1

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
value	1	1	2	4	4	4	4	4	5	7	7	8	8	9	9	9	10

- use tallies to generate sorted contents of a

Bucket sort code

```
// Sorts the contents of a using bucket sort algorithm.  
// Precondition: all elements in a are in range 0 .. 9999999  
public static void bucketSort(int[] a) {  
    int[] counters = new int[10000000];  
    for (int k : a) {  
        counters[k]++; // tally the counters  
    }  
    // put the counter information back into a  
    int i = 0;  
    for (int j = 0; j < counters.length; j++) {  
        for (int k = 0; k < counters[j]; k++) {  
            a[i] = j;  
            i++;  
        }  
    }  
}
```

Bucket sort code v2

// Sorts the contents of a using bucket sort algorithm.

// Works for any range of integers in a.

```
public static void bucketSort(int[] a) {  
    int min = Integer.MAX_VALUE; // find range of values  
    int max = Integer.MIN_VALUE; // stored in a  
    for (int k : a) {  
        max = Math.max(max, k);  
        min = Math.min(min, k);  
    }  
    int[] counters = new int[max - min + 1];  
    for (int k : a) {  
        counters[k - min]++;  
    }  
    int i = 0;  
    for (int j = 0; j < counters.length; j++) {  
        for (int k = 0; k < counters[j]; k++) {  
            a[i] = j + min;  
            i++;  
        }  
    }  
}
```

Bucket sort runtime

```
bucketSort(a, length=N):
```

```
    C = new int[M] //  $O(M)$ 
```

```
    for (k : a) {C[k]++} //  $O(N)$ 
```

```
    for (j : C) { //  $T(N/2)$ 
```

```
        for (C[j]) times: a[i] = j. //  $O(M)$ 
```

- $O(M + N)$
- $\sim O(N)$ time when $N \gg M$
- *linear runtime on sorting!* woot

Useful Links

<http://www.csanimated.com/animation.php?t=Quicksort>

<http://www.hakansozzer.com/category/c/>

<http://www.nczonline.net/blog/2012/11/27/computer-science-in-javascript-quicksort/>

<http://www.sorting-algorithms.com/shell-sort>

Design Methods

We have discussed examples of the following algorithm design principles:

- Divide-and-Conquer Paradigm
(<https://www.youtube.com/watch?v=PfEmF07S4hw>)
- Greedy Paradigm
(<https://www.youtube.com/watch?v=JxmwhAGcuxk>)
- Dynamic Programming Paradigm
(<https://www.youtube.com/watch?v=FiHRNzkMADo>)

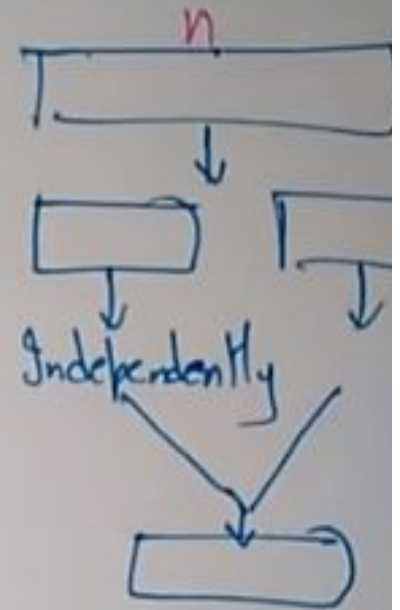
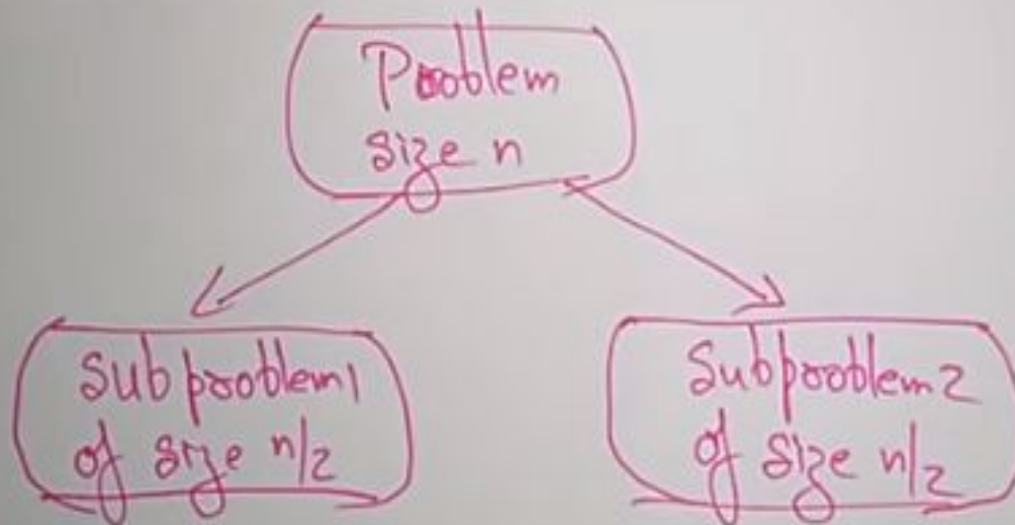
Main Question

When can one successfully use one of these algorithm design paradigms to solve a problem?

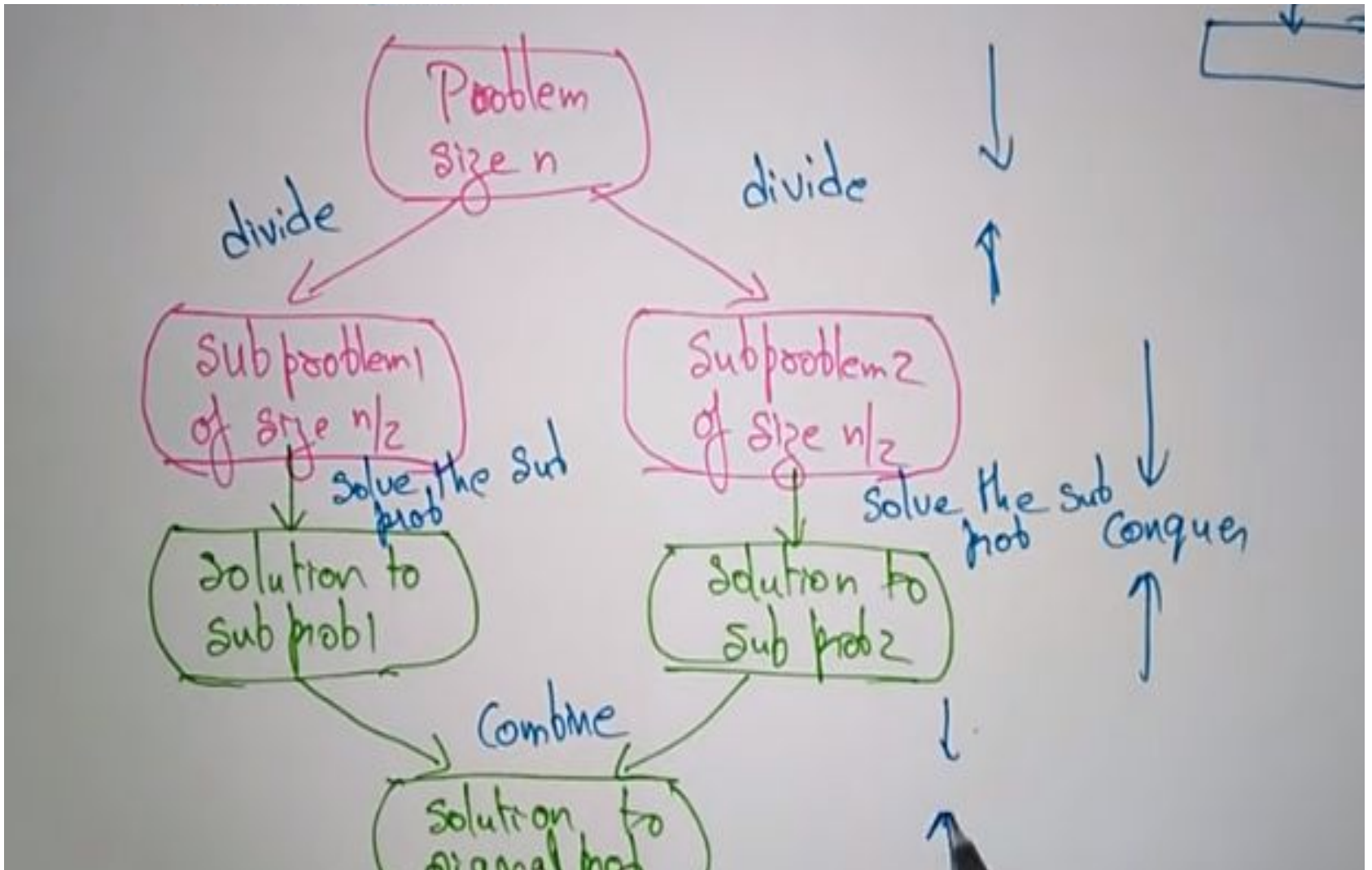
Divide-and-Conquer

Divide-and-Conquer

- ① Divide problem into smaller parts
- ② Independently solve the parts
- ③ Combine these solution to get overall solution

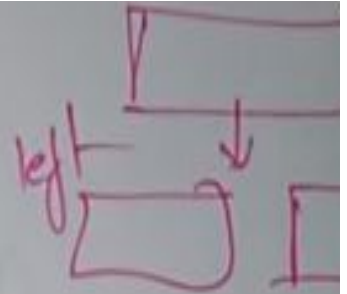


Divide-and-Conquer



Divide-and-Conquer

Idea 1 : Divide the array into 2 halves,
recursively sort left & right halves
then merge two halves { merge sort }



Idea 2 : Partition array into small items & large items
then recursively sort the two sets { Quick sort }

Divide-and-Conquer

Examples!

- ① Searching :- binary search
- ② Sorting :- merge sort, quick sort
- ③ tree traversals
- ④ Matrix multiplication :- strassen's alg

Divide-and-Conquer

Divide-and-conquer:

- Break the problem into smaller sub-problems
- Solve each of the sub-problems
- Combine the solutions to obtain the solution to the original problem

Key detail:

- We keep breaking the sub-problems into smaller and smaller, until the problem is transformed into something entirely different.
 - – At this point, we “conquered” the original problem.

Divide-and-Conquer

Divide-and-conquer:

- Example: In the merge sort, we have the (complicated) task of sorting a bunch of values.
 - However, sorting a set of one value, however, is a completely different problem (a trivial one, in this case).
 - Sorting a set of two values is not as trivial, but it's still a completely different problem — one that can be described as one conditional swap.

Divide-and-Conquer

Divide-and-conquer:

- An interesting example that we have not covered earlier in the course: Karatsuba multiplication algorithm:
 - Useful for multiplying polynomials and integer number (based on a binary representation)
- Originally proposed as an efficient technique to implement binary multiplication in hardware (i.e., an efficient design of the multiplication circuitry in an ALU)

Divide-and-Conquer

Divide-and-conquer:

- The key detail in this case being: multiplying two numbers of n bits is a complicated problem — but multiplying two single-bit numbers is just a logical AND gate.
- The algorithm is nowadays used in cryptographic applications (among others), where arithmetic with very large numbers (in the order of up to thousands of bits) is required:
 - In this context, the difference being: multiply a number of n bits where $n > \text{CPU register width}$ vs. multiply two register-size numbers.

Greedy Algorithms

Greedy Algorithms

Simplest & straight forward approach

⇒ The decision is taken on basis of current available info. without worrying abt the effect of current decision in future.

⇒ feasible sol that may or may not optimal
↓ ↓
Any subset that satisfy the condition best or most favourable sol

Greedy Algorithms

Characteristics & features:

① To construct the sol in an optimal way, Alg maintains 2 sets

- one contains chosen items &
- other contains rejected items

② Greedy alg make good local choices

- an optimal sol
- A feasible sol

Greedy Algorithms

Components of Greedy Algorithm..

- ① A candidate set:- A sol is created from this set
- ② A selection function:- used to choose the best candidate to be added to the sol
- ③ A feasibility function:- used to determine whether a candidate can be used to contribute to the sol
- ④ A objective function:- used to assign value to a sol or partial sol
- ⑤ A solution function:- used to indicate whether a complete sol has been reached.

Greedy Algorithms

⇒ Areas of Applications:

- ① finding shortest path
- ② finding MST
- ③ Job sequencing with deadline
- ④ fractional knapsack prob

Greedy Algorithms

Pseudo code for GA:

Algorithm Greedy(a, n)

```
{
  Solution := 0;
  for  $i := 1$  to  $n$  do
  {
     $x := \text{select}(a)$ ;
    if feasible(Solution,  $x$ ) then
      Solution := union(Solution,  $x$ );
  }
  return Solution;
}
```


Greedy Algorithms

- These are **iterative algorithms** that at each iteration the criterion used is to **maximize some “gain” or some objective function**.
- The term “greedy” refers to the fact that the algorithms do this in a “**short sighted**” way; they try to maximize immediate gain, disregarding the big picture (“get the most I can get now”).
 - For this reason, they can fail to determine a global maximum or minimum (they could “fall in a trap” and converge to some local maximum)
 - **Example: finding the maximum of a function going by steps of fixed size.** (example drawn in the board)
- **Dijkstra's, Prim's, and Kruskal's algorithms are all greedy algorithms**

Greedy Algorithms

- The development of a greedy algorithm can be separated into the following steps:
 1. Cast the optimization problem as one in which we make a choice and are left with **one subproblem to solve**.
 2. Prove that there is always an **optimal solution** to the original problem that makes the greedy choice, so that the **greedy choice is always safe**.
 3. Demonstrate that, having made the greedy choice, what remains is a subproblem with the property that if we **combine an optimal solution to the subproblem** with the greedy choice that we have made, we arrive at an **optimal solution to the original problem**.

Greedy-Choice Property

- The greedy choice property is that a globally optimal solution can be arrived at by making a locally optimal (=greedy) choice.

Optimal Substructure

- A problem exhibits optimal substructure **if and only if an optimal solution** to the problem contains within it optimal solutions to subproblems.

Optimal Substructure

- A classical example of this technique is the problem of producing change — or in general, expressing a given amount of money in coins, minimizing the number of coins required.
- What's a greedy algorithm to compute this result?
- At each iteration, put as many coins of the highest denomination that fits (so that at this iteration we minimize the number of coins used).
 - But.... Does it minimize the number of coins in the final outcome of the algorithm?

Optimal Substructure

- You're probably used to the idea that it does (because with Canadian coins / denominations, it does work)
- But it can indeed fail. Consider denominations 10, 8, and 1; and try to break 17 into those.
- The optimal is obviously 3 coins ($2 \times 8 + 1$); but the greedy algorithm would tell us that we require 8 coins! ($10 + 7 \times 1$)

Optimal Substructure

- The greedy algorithm can work (as in, produce always the correct output), depending on the set of denominations:
 - If every denomination is at least twice the next one (“next one” in decreasing order, that is).
 - Alternatively, if every denomination is greater than or equal to the sum of all smaller denominations.
 - Canadian coins meet these requirements; in cents, we have: 200, 100, 25, 10, 5, and 1

Optimal Substructure

- So, if these greedy / short-sighted algorithms can fail, why do we bother? Why are they attractive?
- They tend to be efficient — at each iteration, we look only at the immediate scenario, no need to look globally.
- If we can find a greedy algorithm that works, then that's good news!
 - On the other hand, it could be an example of a solution to a problem like an NP-complete problem, or a problem believed intractable, and a greedy algorithm might be an efficient alternative that finds a near-optimal solution.

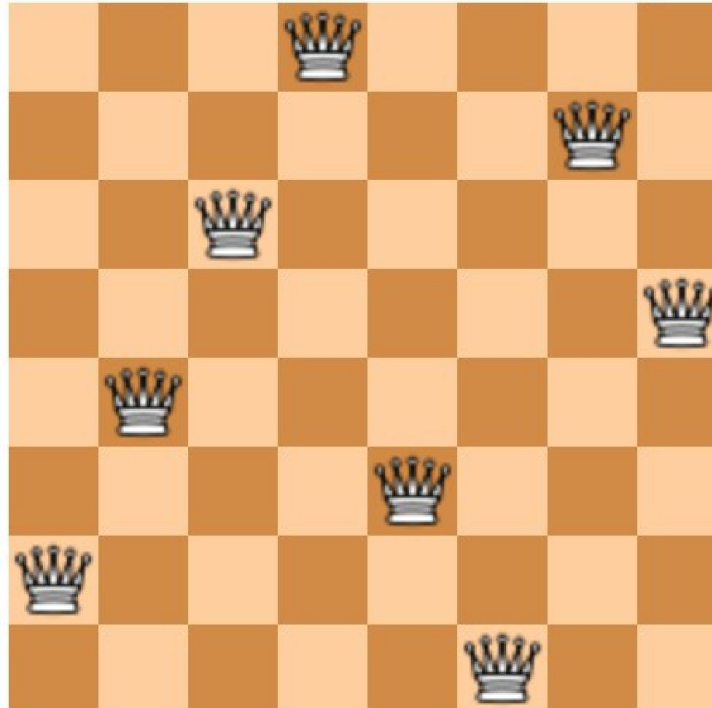
Backtracking algorithms

Backtracking algorithms

- Backtracking algorithms can be seen as an optimized form of exhaustive search algorithms:
 - Try all possible combinations of things, *with early elimination* of paths that lead to no solution.
 - A good visual analogy is: Think of the possibilities represented by a tree, and we do a depth-first traversal, but we have the advantage that in many cases, before reaching the leaf node, we'll know that this path will lead us nowhere, so we just “backtrack” and try the next possibility at the previous level.

Backtracking algorithms

- One classical example is the famous eight queens Chess puzzle:
 - Find a placement for eight queens in a chess board such that neither one falls under threat of capture.



Backtracking algorithms

- A backtracking algorithm places one queen at a time (say, column by column).
 - At each column (each “level” of the search), we have eight possibilities, so we try one and move to the next level.
 - If at some level we find that all eight possibilities fail, then we *backtrack* — go back to the previous level, reporting that this one failed; at the previous level, we now move to the next possibility and keep trying.
 - The key detail is that most of the time, we're going to discover *early* that the current path won't work (e.g., at the third or fourth column we'll find that it won't work)

Dynamic Programming

Dynamic Programming

It is a strategy for designing Alg which is used when problem breaks down into recurring small problems



⇒ It is typically applied to optimization problems.

⇒ In such prob there can be many sol., Each sol has a value & we wish to find a sol with the optimal value.

Dynamic Programming

① Simple subproblems:

We should be able to break the original prob to small subproblems that have the same structure.

② Optimal substructure of prob:

The optimal sol to the problem contains within optimal sol to its subproblems

③ Overlapping subproblems:

There exists some places where we solve the same subproblem more than once.

Dynamic Programming

- Steps to design DP Algorithm:

- 1) characterize optimal substructure
- 2) Recursively define the value of an optimal sol
- 3) compute the value bottom up
- 4) (if needed) construct an optimal sol.

Applications *

- ⇒ Matrix chain Multiplication
- ⇒ optimal binary search tree
- ⇒ All pairs shortest path problem
- ⇒ Traveling salesperson prob
- ⇒ 0/1 knapsack prob
- ⇒ Reliability design

Dynamic Programming

- The basic idea with Dynamic programming is to work with a problem that is broken into slightly smaller sub-problems, where some of these sub-problems overlap.
 - A straightforward recursive solution is inefficient since we solve the same sub-problems over and over again.
- The main detail with a dynamic programming solution is that we store the solutions for sub -problems that we already solved!

Dynamic Programming

- The development of a dynamic programming algorithm can be subdivided into the following steps:
 1. Characterize the structure of an optimal solution
 2. Recursively define the value of an optimal solution
 3. Compute the value of an optimal solution in a bottom-up fashion
 4. Construct an optimal solution from computed information

Dynamic Programming

- A classical example is that of a recursive implementation of a function to determine a Fibonacci number — which can be “neatly” implemented as:

```
int fibonacci (int n)
{
    if (n <= 2)
    {
        return 1;
    }
    return fibonacci(n-1) + fibonacci(n-2);
}
```

Dynamic Programming

Two approaches:

- **Top to bottom**

- – Set up an associated array where each time that the function is requested to compute one value, it first checks if that value is in the corresponding location in the array:
 - If it is, use the value
 - If it's not, compute the value and store the obtained result in the array.

- **Bottom to top**

- Start at the “base case”, and explicitly go up, calculating the “upper” values in terms of the already-calculated “bottom” values.

Dynamic Programming

There are many applications. Examples are:

- Finding the optimal order of multiplications for a sequence of matrices of different sizes
 - – Matrix multiplication is associative; obtaining $ABCD$ can be done as $A(BCD)$, or $(AB)(CD)$, or $A(BC)D$, etc.
 - – When considering all possible arrangements, some of the arrangements for smaller sequences repeat, so we would end up redundantly doing these computations

Dynamic Programming

- Finding common substrings, or aligning strings where differences are small omissions on either side.
- Alignment by stretching sub-sequences:
 - This has very interesting applications in speech recognition and in rhythm recognition/matching for music search applications.

Key Question

- When can we apply the dynamic programming paradigm?

Optimal Substructure

- A problem exhibits optimal substructure if and only if an optimal solution to the problem contains within it optimal solutions to subproblems.
- Whenever a problem exhibits optimal substructure, it is an indication that a dynamic programming or greedy strategy might apply.

Overlapping Subproblems

- A second indication that dynamic programming might be applicable is that the space of subproblems must be small, meaning that a recursive algorithm for the problem solves the same subproblems over and over.
- Typically, the total number of distinct subproblems is a polynomial in the input size.

-

Overlapping Subproblems

- When a recursive algorithm revisits the same problem over and over again, then we say that the optimization problem has overlapping subproblems.
- Here two subproblems are called overlapping if and only if they really are the same subproblem that occurs as a subproblem of different problems.

Note

If a recursive algorithm solving the problem creates always new subproblems, then this is an indication that divide-and-conquer methods rather than dynamic programming might apply.

END